

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

ĐÀO CÔNG HẬU
PHẠM NGỌC HẢI

KHÓA LUẬN TỐT NGHIỆP
NGHIÊN CỨU THIẾT KẾ VÀ HIỆN THỰC LỖI IP
PHÁT HIỆN ÂM THANH KHẨN CẤP TRONG MÔI
TRƯỜNG CÔNG CỘNG

RESEARCH, DESIGN AND IMPLEMENTATION OF AN IP
CORE FOR EMERGENCY SOUND DETECTION IN PUBLIC
ENVIRONMENT

CỬ NHÂN NGÀNH KỸ THUẬT MÁY TÍNH

TP. HỒ CHÍ MINH, 2026

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

ĐÀO CÔNG HẬU – 22520408

PHẠM NGỌC HẢI – 22520389

KHÓA LUẬN TỐT NGHIỆP
NGHIÊN CỨU THIẾT KẾ VÀ HIỆN THỰC LỖI IP
PHÁT HIỆN ÂM THANH KHẨN CẤP TRONG MÔI
TRƯỜNG CÔNG CỘNG

RESEARCH, DESIGN AND IMPLEMENTATION OF AN IP
CORE FOR EMERGENCY SOUND DETECTION IN PUBLIC
ENVIRONMENT

CỬ NHÂN NGÀNH KỸ THUẬT MÁY TÍNH

GIẢNG VIÊN HƯỚNG DẪN
THS. TRƯƠNG VĂN CƯỜNG

TP. HỒ CHÍ MINH, 2026

THÔNG TIN HỘI ĐỒNG CHẤM KHÓA LUẬN TỐT NGHIỆP

Hội đồng chấm khóa luận tốt nghiệp, thành lập theo Quyết định số 710/QĐ-ĐHCNTT ngày 19 tháng 06 năm 2026 của Hiệu trưởng Trường Đại học Công nghệ Thông tin.

LỜI CẢM ƠN

Trong quá trình học tập và thực hiện khóa luận tốt nghiệp, nhóm em đã nhận được nhiều sự quan tâm, hướng dẫn và hỗ trợ quý báu từ quý thầy cô, gia đình và bạn bè.

Trước hết, nhóm em xin chân thành cảm ơn Ban Giám hiệu Trường Đại học Công nghệ Thông tin và quý thầy cô Khoa Kỹ thuật Máy tính đã tạo điều kiện thuận lợi, đồng thời trang bị những kiến thức chuyên môn cần thiết trong suốt quá trình học tập tại trường.

Đặc biệt, nhóm em xin bày tỏ lòng biết ơn sâu sắc đến thầy Trương Văn Cương, giảng viên hướng dẫn – người đã tận tình định hướng nghiên cứu, đóng góp nhiều ý kiến chuyên ngành và luôn động viên nhóm trong quá trình thực hiện đề tài “Nghiên cứu, thiết kế và hiện thực lõi IP phát hiện âm thanh khẩn cấp trong môi trường công cộng”.

Nhóm em cũng xin cảm ơn các thầy cô, anh chị, bạn bè và gia đình đã luôn hỗ trợ, chia sẻ kinh nghiệm và tạo điều kiện để nhóm hoàn thành khóa luận.

Mặc dù đã cố gắng hoàn thành khóa luận song khó tránh khỏi những sai sót do hạn chế về thời gian và kinh nghiệm. Nhóm chúng em rất mong nhận được những ý kiến đóng góp quý báu từ quý thầy cô và các bạn để đề tài được hoàn thiện hơn.

Nhóm em xin chân thành cảm ơn!

Thành phố Hồ Chí Minh, tháng 7 năm 2026

Sinh viên thực hiện

Đào Công Hậu

Phạm Ngọc Hải

MỤC LỤC

Chương 1.	MỞ ĐẦU	5
1.1.	Lý do chọn đề tài	5
1.2.	Mục tiêu nghiên cứu	5
1.3.	Đối tượng và phạm vi nghiên cứu	6
1.4.	Phương pháp nghiên cứu	7
Chương 2.	TÌM HIỂU TỔNG QUAN	9
2.1.	Cơ sở lý thuyết.....	9
2.1.1.	Tín hiệu âm thanh số	9
2.1.2.	Biến đổi Fourier rời rạc và biến đổi Fourier nhanh.....	9
2.1.3.	Đặc trưng Log-Mel Spectrogram	10
2.1.3.1.	Công suất của phổ	10
2.1.3.2.	Mel filterbank.....	10
2.1.3.3.	Log-Mel	11
2.1.4.	Mạng nơ ron tích chập (CNN).....	11
2.1.4.1.	Nguyên lý tích chập hai chiều.....	12
2.1.4.2.	Padding và Stride	13
2.1.4.3.	Batch Normalization	13
2.1.4.4.	Hàm kích hoạt ReLU	14
2.1.4.5.	MaxPooling.....	14
2.1.4.6.	Fully Connected Layer.....	15
2.1.4.7.	Fixed-point và lượng tử hóa trong CNN.....	15
2.2.	Các công trình nghiên cứu liên quan	16
2.2.1.	Triển khai và tăng tốc mạng CNN trên hệ thống nhúng	16

2.2.2.	Nhận diện âm thanh thời gian thực trên nền tảng SoC-FPGA.....	16
2.2.3.	So sánh với các công trình nghiên cứu có liên quan	17
Chương 3.	THIẾT KẾ HỆ THỐNG.....	19
3.1.	Kiến trúc tổng quan của hệ thống trên phần cứng.....	19
3.2.	Thiết kế khối FFT	20
3.3.	Thiết kế khối Log-Mel Spectrogram	24
3.3.1.	Mô-đun power_unit.....	26
3.3.2.	Mô-đun mac_unit	29
3.3.3.	Mô-đun log_unit.....	34
3.4.	Bộ đệm FIFO	38
3.5.	Thiết kế khối CNN.....	39
3.5.1.	Khối tích chập (convolution).....	41
3.5.2.	Khối Maxpool.....	48
3.5.3.	Khối Fully Connected (FC1).....	51
3.5.4.	Khối Fully Connected (FC2).....	57
Chương 4.	THIẾT KẾ KIỂM THỬ'	65
4.1.	Tổng quan kiểm thử trên phần mềm.....	65
4.2.	Thiết kế mô phỏng phần mềm	66
4.2.1.	Chuẩn bị dataset	66
4.2.2.	Tiền xử lý dữ liệu	67
4.2.3.	Mô hình CNN	68
4.2.4.	Huấn luyện mô hình	69
4.3.	Nền tảng phần cứng thực nghiệm (Kit FPGA ZCU104).....	72
Chương 5.	KẾT QUẢ MÔ PHỎNG VÀ KIỂM THỬ'	74

5.1.	Mô phỏng Behavioral/RTL (pre-synthesis).....	74
5.1.1.	Khối FFT	74
5.1.2.	Khối Log-Mel.....	75
5.1.3.	Khối CNN.....	77
5.1.4.	Kết quả mô phỏng của toàn hệ thống (FFT + Log-Mel + CNN)	78
5.2.	Tổng hợp thiết kế (Synthesis).....	83
5.2.1.	Ước lượng mức sử dụng tài nguyên phần cứng	83
5.2.2.	Mô phỏng chức năng sau tổng hợp	84
5.3.	Hiện thực hóa phần cứng (Implementation).....	87
5.3.1.	Tài nguyên sử dụng thực tế	87
5.3.2.	Phân tích công suất tiêu thụ (Power).....	88
5.3.3.	Phân tích thời gian và tần số hoạt động (Timing)	89
5.4.	So sánh tài nguyên hệ thống với các công trình nghiên cứu liên quan ...	90
Chương 6.	TỔNG KẾT	92
6.1.	Kết luận.....	92
6.2.	Khó khăn.....	93
6.3.	Hướng phát triển.....	93

DANH MỤC HÌNH

Hình 3.1: Sơ đồ khối hệ thống	20
Hình 3.2: Top-module của FFT	21
Hình 3.3: Sơ đồ khối của FFT.....	23
Hình 3.4: Top-module của Log-Mel Spectrogram.....	24
Hình 3.5: Sơ đồ khối Log-Mel Spectrogram	25
Hình 3.6: Mô-đun power_unit.....	27
Hình 3.7: Sơ đồ khối của power_unit	28
Hình 3.8: Mô-đun mac_unit.....	30
Hình 3.9: Sơ đồ khối mac_unit	31
Hình 3.10: Mô-đun log_unit	35
Hình 3.11: Sơ đồ khối log_unit.....	36
Hình 3.12: Mô-đun FIFO	38
Hình 3.13: Kiến trúc tổng quan khối CNN	39
Hình 3.14: Top-module của khối CNN.....	40
Hình 3.15: Sơ đồ khối của khối tích chập.....	43
Hình 3.16: Ảnh minh họa về padding = 0.....	45
Hình 3.17: Lưu đồ FSM của khối tích chập.....	48
Hình 3.18: Sơ đồ khối của maxpool lớp thứ nhất	49
Hình 3.19: Sơ đồ khối của FC1.....	53
Hình 3.20: Lưu đồ FSM của FC1.....	56
Hình 3.21: Lưu đồ FSM của FC2.....	64
Hình 4.1: Quy trình thiết kế kiểm thử.....	65
Hình 4.2: Kết quả training mô hình CNN	71
Hình 4.3: Ảnh kit Zynq UltraScale+ MPSoC ZCU104	72
Hình 5.1: Kết quả mô phỏng FFT	74
Hình 5.2: Kết quả kiểm thử FFT	75
Hình 5.3: Kết quả mô phỏng Log-Mel.....	76
Hình 5.4: Kết quả kiểm thử Log-Mel.....	76

Hình 5.5: Kết quả mô phỏng CNN (nạp dữ liệu).....	77
Hình 5.6: Kết quả mô phỏng CNN (xuất kết quả)	77
Hình 5.7: Kết quả kiểm thử CNN	78
Hình 5.8: Kết quả nhận diện của hệ thống trên phần mềm.....	79
Hình 5.9: Kết quả mô phỏng hệ thống.....	79
Hình 5.10: Kết quả kiểm thử hệ thống.....	80
Hình 5.11: Kết quả mô phỏng hệ thống.....	80
Hình 5.12: Kết quả kiểm thử hệ thống.....	80
Hình 5.13: Kết quả mô phỏng hệ thống.....	80
Hình 5.14: Kết quả kiểm thử hệ thống.....	81
Hình 5.15: Biểu đồ so sánh giá trị logit các testcase lớp CỨU.....	81
Hình 5.16: Biểu đồ so sánh giá trị logit các testcase lớp CUỐP	82
Hình 5.17: Biểu đồ so sánh giá trị logit các testcase của lớp UNKNOWN.....	83
Hình 5.18: Bảng kết quả ước lượng mức sử dụng tài nguyên (Synthesis)	83
Hình 5.19: Kết quả mô phỏng sau tổng hợp của khối Log-Mel	84
Hình 5.20: Kết quả kiểm thử sau tổng hợp của khối Log-Mel	85
Hình 5.21: Kết quả mô phỏng khối CNN sau tổng hợp.....	85
Hình 5.22: Kết quả kiểm thử sau tổng hợp của khối CNN	86
Hình 5.23: Kết quả kiểm thử sau tổng hợp của khối FFT.....	86
Hình 5.24: Bảng kết quả sử dụng tài nguyên (Implementation)	87
Hình 5.25: Ảnh báo cáo công suất tiêu thụ	88
Hình 5.26: Ảnh báo cáo phân tích thời gian	89

DANH MỤC BẢNG

Bảng 2.1: So sánh các công trình nghiên cứu	17
Bảng 3.1: Các port của FFT	21
Bảng 3.2: Các port của Log-Mel Spectrogram	25
Bảng 3.3: Các port của power_unit.....	27
Bảng 3.4: Các port của mac_unit	29
Bảng 3.5: Cấu trúc dữ liệu của mỗi từ nhớ trong ROM	31
Bảng 3.6: Các port của log_unit.....	34
Bảng 3.7: Các port của model CNN	40
Bảng 3.8: Các port của khối tích chập	42
Bảng 3.9: Bảng các trạng thái của FSM.....	47
Bảng 3.10: Các port của khối Maxpool	48
Bảng 3.11: Các port của FC1	52
Bảng 3.12: Các trạng thái của FSM	56
Bảng 3.13: Các port của FC2	58
Bảng 3.14: Các trạng thái của controller.....	62
Bảng 4.1: Thống kê tập dữ liệu.....	67
Bảng 4.2: Thông số của khối tiền xử lý	67
Bảng 4.3: Các thành phần của mô hình	69
Bảng 4.4: Các thông số giá trị.....	70
Bảng 4.5: Các chỉ số huấn luyện.....	71
Bảng 4.6: Thông số tài nguyên của chip FPGA SoC XCZU7EV	73
Bảng 5.1: Thông số kiểm thử FFT	74
Bảng 5.2: Thông số kiểm thử Log-Mel.....	76
Bảng 5.3: Thông số kiểm thử model CNN	78
Bảng 5.4: So sánh kết quả sử dụng tài nguyên	87
Bảng 5.5: So sánh tài nguyên hệ thống giữa các công trình nghiên cứu	90

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Từ viết đầy đủ
ACC	Accumulator (Thanh ghi tích lũy)
BRAM	Block Random Access Memory
CNN	Convolutional Neural Network
DFT	Discrete Fourier Transform
DSP	Digital Signal Processing / Digital Signal Processor
FFT	Fast Fourier Transform
FIFO	First In, First Out
FPGA	Field Programmable Gate Arrays
FSM	Finite State Machine
HDL	Hardware Description Language
IP	Intellectual Property (core)
LSB	Least Significant Bit
LUT	Look-Up Table
MAC	Multiply-Accumulate
MSB	Most Significant Bit
PCM	Pulse-Code Modulation
ReLU	Rectified Linear Unit

TÓM TẮT KHÓA LUẬN

Trong bối cảnh nhu cầu giám sát an ninh tại các khu vực công cộng ngày càng tăng, đặc biệt vào ban đêm hoặc tại những khu vực có điều kiện quan sát hạn chế, việc phát hiện sớm các tình huống bất thường đóng vai trò quan trọng trong việc hỗ trợ cảnh báo và xử lý sự cố. Các hệ thống camera giám sát truyền thống chủ yếu dựa vào hình ảnh, tuy nhiên hiệu quả có thể bị suy giảm trong các trường hợp thiếu sáng, góc khuất, vật cản hoặc chất lượng hình ảnh thấp. Trong khi đó, âm thanh là một nguồn thông tin bổ sung quan trọng, có khả năng phản ánh các sự kiện khẩn cấp như tiếng kêu cứu, tiếng la hét, tiếng va chạm hoặc các âm thanh bất thường khác. Vì vậy, khóa luận này tập trung nghiên cứu, thiết kế và hiện thực lõi IP phát hiện âm thanh khẩn cấp trong môi trường công cộng nhằm hỗ trợ các hệ thống giám sát an ninh hoạt động hiệu quả hơn.

Đề tài xây dựng hướng tiếp cận dựa trên xử lý tín hiệu âm thanh và học sâu. Tín hiệu âm thanh đầu vào được chuẩn hóa ở tần số lấy mẫu 16 kHz, sau đó được chia thành các frame ngắn để trích xuất đặc trưng. Chuỗi xử lý đặc trưng bao gồm các bước: biến đổi Fourier nhanh FFT để chuyển tín hiệu từ miền thời gian sang miền tần số, tính phổ công suất, áp dụng Mel filterbank và thực hiện phép logarithm để tạo đặc trưng Log-Mel Spectrogram. Đặc trưng Log-Mel thu được có dạng ma trận hai chiều, thể hiện sự phân bố năng lượng âm thanh theo thời gian và tần số, phù hợp làm đầu vào cho mạng nơ-ron tích chập CNN.

Bên cạnh việc xây dựng mô hình phần mềm, khóa luận còn tập trung thiết kế các khối xử lý đặc trưng dưới dạng IP phần cứng, bao gồm FFT, Log-Mel Spectrogram và CNN. Các khối IP được thiết kế theo hướng có giao diện rõ ràng, có các tín hiệu đồng bộ, từ đó tạo điều kiện thuận lợi cho việc tích hợp vào hệ thống lớn hơn và hướng đến khả năng triển khai trên FPGA.

Quá trình kiểm chứng được thực hiện bằng cách kết hợp giữa mô phỏng phần cứng và so sánh với phần mềm tham chiếu. Mô hình CNN được huấn luyện và đánh

giá trên tập dữ liệu âm thanh nhằm hướng đến mục tiêu đạt độ chính xác phân loại từ 90% trở lên.

Kết quả kỳ vọng của khóa luận là xây dựng được một pipeline xử lý âm thanh có khả năng trích xuất đặc trưng Log-Mel từ tín hiệu âm thanh 16 kHz, model CNN nhận diện được từ khóa và có khả năng tích hợp trên nền tảng phần cứng FPGA. Hệ thống đề xuất góp phần tạo nền tảng cho các ứng dụng giám sát an ninh thông minh, trong đó âm thanh được khai thác như một nguồn thông tin hỗ trợ phát hiện sớm các tình huống khẩn cấp trong môi trường công cộng.

ABSTRACT

With the increasing demand for security surveillance in public areas, particularly during nighttime or in environments with limited visibility, the early detection of abnormal situations has become essential for timely warning and incident response. Conventional surveillance systems primarily rely on visual information captured by cameras; however, their performance may degrade under low-light conditions, occlusions, obstructed views, or poor image quality. In contrast, audio provides an important complementary source of information capable of reflecting emergency events such as cries for help, screams, collisions, and other abnormal sounds. Therefore, this thesis focuses on the research, design, and implementation of an intellectual property (IP) core for emergency sound detection in public environments to enhance the effectiveness of security surveillance systems.

The proposed approach is based on audio signal processing and deep learning. The input audio signal is first standardized to a sampling rate of 16 kHz and segmented into short frames for feature extraction. The feature extraction pipeline consists of Fast Fourier Transform (FFT) to convert the signal from the time domain to the frequency domain, power spectrum computation, Mel filterbank processing, and logarithmic transformation to generate Log-Mel Spectrogram features. The resulting Log-Mel Spectrogram is represented as a two-dimensional matrix that describes the distribution of acoustic energy over time and frequency, making it well suited as the input to a Convolutional Neural Network (CNN).

In addition to developing the software model, this thesis emphasizes the hardware design of the feature extraction pipeline as reusable IP cores, including FFT, Log-Mel Spectrogram, and CNN modules. These hardware IP cores are designed with well-defined interfaces and synchronized control signals, facilitating integration into larger systems and enabling deployment on FPGA platforms.

The proposed design is verified through hardware simulation and comparison with a software reference model. The CNN model is trained and evaluated on an audio dataset with the objective of achieving a classification accuracy of at least 90%.

The expected outcome of this thesis is a complete audio processing pipeline capable of extracting Log-Mel Spectrogram features from 16 kHz audio signals, employing a CNN model to recognize emergency-related keywords, and supporting integration on FPGA hardware platforms. The proposed system provides a foundation for intelligent security surveillance applications by exploiting audio as a complementary source of information for the early detection of emergency situations in public environments.

Chương 1. MỞ ĐẦU

1.1. Lý do chọn đề tài

Trong bối cảnh đô thị hóa ngày càng phát triển, nhu cầu giám sát an ninh tại các khu vực công cộng như đường phố, bãi giữ xe, khu dân cư hoặc các khu vực ít người qua lại vào ban đêm ngày càng trở nên quan trọng. Các hệ thống camera giám sát hiện nay chủ yếu dựa vào hình ảnh để phát hiện sự kiện bất thường. Tuy nhiên, trong điều kiện ban đêm, thiếu sáng, góc khuất, vật cản hoặc chất lượng hình ảnh thấp, việc nhận diện sự cố chỉ dựa trên hình ảnh có thể gặp nhiều hạn chế.

Âm thanh là một nguồn thông tin quan trọng có thể hỗ trợ phát hiện các tình huống bất thường trong môi trường thực tế. Một số sự kiện liên quan đến an ninh như tiếng la hét, tiếng kêu cứu, tiếng va chạm, tiếng kính vỡ, tiếng cãi vã hoặc các âm thanh đột ngột thường xuất hiện trước hoặc đồng thời với sự cố. Vì vậy, việc khai thác tín hiệu âm thanh kết hợp với hệ thống camera giám sát có thể giúp nâng cao khả năng phát hiện sớm các tình huống nguy hiểm, đặc biệt trong môi trường ban đêm.

Từ thực tế trên, đề tài “Phát hiện âm thanh bất thường trong camera công cộng” được lựa chọn nhằm nghiên cứu và xây dựng một hướng tiếp cận sử dụng xử lý tín hiệu âm thanh và học sâu để nhận diện các âm thanh có khả năng liên quan đến sự cố an ninh.

1.2. Mục tiêu nghiên cứu

Mục tiêu chính của đề tài là nghiên cứu và xây dựng một hệ thống có khả năng phát hiện âm thanh bất thường trong môi trường công cộng, đặc biệt trong các tình huống cần sự giám sát an ninh vào ban đêm. Hệ thống hướng đến khả năng xử lý tín hiệu âm thanh theo thời gian thực, trích xuất đặc trưng phù hợp và phân loại âm thanh bất thường với độ chính xác cao.

Các mục tiêu cụ thể của đề tài bao gồm:

Thứ nhất, xây dựng mô hình phát hiện âm thanh bất thường dựa trên đặc trưng Log-Mel Spectrogram và mạng nơ-ron tích chập CNN, với mục tiêu đạt độ chính xác phân loại từ 90% trở lên trên tập dữ liệu kiểm thử.

Thứ hai, thiết kế quy trình xử lý tín hiệu âm thanh với tần số lấy mẫu 16 kHz. Tín hiệu âm thanh đầu vào được chia thành các frame, đưa qua các bước xử lý gồm FFT, tính phổ công suất, Mel filterbank và logarithm để tạo đặc trưng Log-Mel Spectrogram.

Thứ ba, thiết kế và tích hợp các khối xử lý đặc trưng âm thanh dưới dạng IP phần cứng, bao gồm FFT, Log-Mel và CNN. Các IP này cần có giao diện rõ ràng, chuẩn hóa tín hiệu dữ liệu hoặc các tín hiệu đồng bộ tương ứng, nhằm thuận tiện cho việc tích hợp vào hệ thống lớn hơn.

Thứ tư, kiểm chứng khả năng hoạt động của các khối IP qua mô phỏng RTL, đối chiếu với kết quả tham chiếu từ phần mềm Python để đánh giá độ chính xác của pipeline xử lý đặc trưng. Trên cơ sở đó, các khối IP được tổng hợp và hiện thực hóa trên nền tảng FPGA, khả năng đáp ứng ràng buộc thời gian và ước lượng công suất tiêu thụ. Cuối cùng, thiết kế được triển khai và kiểm chứng hoạt động thực tế trên board FPGA. Qua đó, đề tài hướng đến việc xây dựng một nền tảng xử lý âm thanh hoàn chỉnh, từ mô phỏng đến triển khai thực tế trên phần cứng, phù hợp với các ứng dụng giám sát an ninh yêu cầu tốc độ xử lý nhanh và độ trễ thấp.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là tín hiệu âm thanh trong môi trường giám sát an ninh, đặc biệt là các âm thanh có khả năng liên quan đến tình huống bất thường xảy ra vào ban đêm. Các âm thanh này bao gồm tiếng kêu cứu, tiếng kêu cướp,.....

Đề tài tập trung nghiên cứu quy trình xử lý tín hiệu âm thanh có tần số lấy mẫu 16 kHz. Tín hiệu âm thanh đầu vào được chia thành các frame ngắn, sau đó trích xuất đặc trưng bằng chuỗi xử lý gồm FFT; tính phổ công suất, Mel filterbank và phép logarithm để tạo ra đặc trưng Log-Mel Spectrogram. Đặc trưng này được sử dụng

làm đầu vào cho mô hình CNN nhằm phân loại âm thanh bình thường và âm thanh bất thường.

Về phạm vi thuật toán, đề tài tập trung vào phương pháp trích xuất đặc trưng Log-Mel và mô hình CNN cho bài toán phân loại âm thanh. Mục tiêu đánh giá của hệ thống là đạt độ chính xác phân loại từ 90% trở lên trên tập dữ liệu kiểm thử. Tuy nhiên, phạm vi đề tài chưa hướng đến việc nhận diện toàn bộ mọi loại âm thanh trong thực tế, mà tập trung vào một số nhóm âm thanh đại diện có liên quan đến bài toán giám sát an ninh.

Về phạm vi phần cứng, đề tài tập trung thiết kế và kiểm chứng các khối xử lý đặc trưng âm thanh dưới dạng IP, bao gồm FFT; Power Unit, Mel/MAC Unit và Log Unit tạo thành IP đặc trưng Log-Mel. Các IP này được định hướng có giao diện chuẩn hóa để thuận tiện tích hợp vào hệ thống lớn hơn. Quy trình kiểm chứng được thực hiện qua ba giai đoạn: (1) mô phỏng RTL ở mức module và mức tích hợp, đối chiếu với kết quả tham chiếu từ Python; (2) tổng hợp (synthesis) và hiện thực hóa (implementation) thiết kế trên công cụ FPGA, đánh giá các chỉ số tài nguyên gồm area, timing và power; (3) triển khai thiết kế lên board FPGA thật để kiểm chứng khả năng hoạt động trong điều kiện thực tế và đáp ứng yêu cầu xử lý thời gian thực.

1.4. Phương pháp nghiên cứu

Đề tài được thực hiện theo phương pháp kết hợp giữa nghiên cứu lý thuyết, xây dựng mô hình phần mềm, thiết kế phần cứng và kiểm chứng thực nghiệm.

Trước tiên, đề tài nghiên cứu các kiến thức nền tảng liên quan đến xử lý tín hiệu âm thanh và học sâu, bao gồm biến đổi Fourier nhanh FFT, phổ công suất, Mel filterbank, Logarithm và mạng nơ-ron tích chập CNN. Các nội dung này là cơ sở để xây dựng pipeline chuyển đổi âm thanh thô thành đặc trưng phù hợp cho bài toán phát hiện âm thanh bất thường.

Tiếp theo, hệ thống phần mềm được xây dựng bằng Python nhằm xử lý dữ liệu âm thanh, tạo đặc trưng Log-Mel Spectrogram, huấn luyện và đánh giá mô hình CNN.

Tín hiệu âm thanh được chuẩn hóa ở tần số lấy mẫu 16 kHz để thống nhất với định hướng xử lý thời gian thực của hệ thống. Kết quả từ phần mềm được dùng làm dữ liệu tham chiếu khi kiểm chứng các khối xử lý phần cứng.

Song song với phần mềm, các khối xử lý đặc trưng được thiết kế bằng Verilog dưới dạng các IP phần cứng. Lõi FFT 1024 điểm được sử dụng để chuyển đổi tín hiệu từ miền thời gian sang miền tần số. Power Unit tính phổ công suất từ phần thực và phần ảo của FFT. Mel/MAC Unit thực hiện phép nhân - cộng dồn với các trọng số Mel filterbank. Log Unit thực hiện phép logarithm để tạo đặc trưng Log-Mel hoàn chỉnh. Cuối cùng khối CNN được thiết kế chi tiết với các lớp tương ứng. Các khối này được thiết kế theo hướng có giao diện rõ ràng, sử dụng các tín hiệu điều khiển như clock, reset, enable, valid và các tín hiệu đồng bộ cần thiết để thuận tiện cho việc tích hợp.

Quá trình kiểm chứng được thực hiện theo ba mức:

Ở mức từng module, các khối FFT, Power Unit, Mel/MAC Unit, Log Unit, CNN được kiểm tra riêng bằng testbench để xác nhận chức năng hoạt động đúng.

Ở mức tích hợp, toàn bộ pipeline FFT $\rightarrow$ Power $\rightarrow$ Mel/MAC $\rightarrow$ Log $\rightarrow$ CNN được mô phỏng và so sánh với kết quả tham chiếu từ Python.

Ở mức phần cứng thực tế, thiết kế được tổng hợp và hiện thực hóa trên nền tảng FPGA, các chỉ số sau bao gồm: area, timing và power được trích xuất và đánh giá, sau đó thiết kế được nạp lên board để kiểm chứng khả năng hoạt động trong điều kiện thực tế.

Cuối cùng, hệ thống được đánh giá dựa trên các tiêu chí chính gồm: độ chính xác phân loại của mô hình CNN, độ đúng của các khối IP khi so sánh với phần mềm tham chiếu, các chỉ số tài nguyên phần cứng (area, timing, power) sau synthesis/implementation, và khả năng hoạt động ổn định khi triển khai trên board FPGA thật. Qua đó, đề tài đánh giá mức độ phù hợp của kiến trúc đề xuất đối với bài toán phát hiện âm thanh bất thường trong hệ thống giám sát an ninh.

Chương 2. TÌM HIỂU TỔNG QUAN

2.1. Cơ sở lý thuyết

2.1.1. Tín hiệu âm thanh số

Tín hiệu âm thanh trong môi trường thực tế là tín hiệu liên tục theo thời gian. Để xử lý trên máy tính hoặc phần cứng số, tín hiệu này cần được lấy mẫu và lượng tử hóa thành chuỗi các giá trị rời rạc. Trong đề tài khóa luận, tín hiệu âm thanh được chuẩn hóa về tần số lấy mẫu 16 kHz, tương ứng với 16000 mẫu trong 1 giây.

Do đặc tính của âm thanh thay đổi theo thời gian, tín hiệu được chia thành các đoạn ngắn gọi là frame. Mỗi frame được xem là gần ổn định trong một khoảng thời gian ngắn và được xử lý riêng để theo dõi sự thay đổi của phổ tần số.

2.1.2. Biến đổi Fourier rời rạc và biến đổi Fourier nhanh

Biến đổi Fourier rời rạc, hay DFT, dùng để chuyển một dãy tín hiệu rời rạc từ miền thời gian sang miền tần số. Với một dãy tín hiệu đầu vào gồm N mẫu: $x[0], x[1], x[2], \dots, x[N-1]$ DFT tạo ra N hệ số tần số: $X[0], X[1], X[2], \dots, X[N-1]$

Công thức DFT thuận được biểu diễn như sau:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi \frac{k}{N} n} \quad (2.1)$$

- $x[n]$: Mẫu tín hiệu ở miền thời gian
- $X[k]$: Hệ số phổ tại bin tần số k
- N : Số điểm FFT
- k : Số bin tần số
- j : Đơn vị ảo

Giá trị $X[k]$ là một số phức gồm phần thực và phần ảo:

$$X[k] = \text{Re}[k] + j\text{Im}[k] \quad (2.2)$$

Nếu tính trực tiếp DFT, độ phức tạp tính toán là $O(N^2)$. Với N lớn, cách tính này không phù hợp cho xử lý thời gian thực. FFT là thuật toán tối ưu để tính DFT với độ phức tạp $O(N\log_2 N)$. Trong đề tài, FFT 1024 điểm được sử dụng để phân tích phổ của từng frame âm thanh.

Với tần số lấy mẫu $F_s = 16 \text{ kHz}$ và $N = 1024$, độ phân giải tần số của FFT là:

$$\Delta f = F_s / N = 16000 / 1024 = 15.625 \text{ Hz/bin}$$

Điều này có nghĩa là mỗi bin FFT cách nhau 15.625 Hz. Ví dụ: tín hiệu sin 1000 Hz sẽ có đỉnh phổ gần bin 64.

2.1.3. Đặc trưng Log-Mel Spectrogram

2.1.3.1. Công suất của phổ

Kết quả FFT là số phức gồm phần thực (Re) và phần ảo (Im). Tuy nhiên, trong bài toán trích xuất đặc trưng âm thanh, hệ thống cần biết mức năng lượng của từng thành phần tần số. Do đó, phổ công suất được tính theo công thức:

$$P[k] = \text{Re}[k]^2 + \text{Im}[k]^2 \quad (2.3)$$

Trong đó, $P[k]$ là năng lượng phổ tại bin k . Phổ công suất là đầu vào cho khối Mel filterbank. Việc sử dụng công suất của phổ giúp hệ thống biểu diễn mức mạnh yếu của từng vùng tần số thay vì xử lý trực tiếp phần thực và phần ảo của FFT đồng thời là dữ liệu đầu vào cho khối bộ lọc Mel (Mel filterbank).

2.1.3.2. Mel filterbank

Mel filterbank là phương pháp gom các bin tần số của FFT thành các dải tần theo thang Mel. Thang Mel được xây dựng dựa trên cách con người cảm nhận âm thanh, trong đó tai người không cảm nhận các thay đổi tần số theo thang tuyến tính tuyệt đối. Trong hệ thống đề xuất, 512 bin đầu của FFT 1024 điểm được gom thành 40 dải Mel. Mỗi dải Mel được tính bằng phép nhân phổ công suất với trọng số Mel tương ứng và cộng dồn:

$$\text{Mel}[m] = \sum P[k] \times W[m,k] \quad (2.4)$$

Trong đó, $P[k]$ là công suất tại bin k , $W[m,k]$ là trọng số Mel của filter m tại bin k , và $Mel[m]$ là năng lượng của dải Mel thứ m .

2.1.3.3. Log-Mel

Sau khi tính Mel energy, các giá trị năng lượng có thể có dải động rất lớn. Nếu đưa trực tiếp vào mô hình CNN, sự chênh lệch quá lớn giữa các giá trị có thể làm mô hình khó học. Do đó, hệ thống áp dụng phép logarithm để nén biên độ:

$$\text{LogMel}[m] = \log_{10}(\text{Mel}[m]) \quad (2.5)$$

Kết quả Log-Mel Spectrogram là ma trận hai chiều biểu diễn năng lượng âm thanh theo thời gian và dải Mel. Trong đề tài, mỗi frame tạo ra 40 giá trị Log-Mel. Khi gom nhiều frame liên tiếp, hệ thống tạo ra ma trận đặc trưng, ví dụ kích thước 40×64 , phù hợp làm đầu vào cho mạng CNN.

2.1.4. Mạng nơ ron tích chập (CNN)

CNN là mô hình học sâu có khả năng tự động học và trích xuất các đặc trưng hình học cục bộ trên cấu trúc dữ liệu hai chiều. Khi Log-Mel Spectrogram được xem như một bức ảnh xám, mạng CNN có thể nhận diện các mẫu cấu trúc như vùng năng lượng tăng đột ngột, dải tần kéo dài hoặc sự thay đổi dạng phổ đặc trưng của từng loại từ khóa âm thanh.

Tại một nơ-ron cơ bản, phép toán thực thi bao gồm tổ hợp tuyến tính và hàm kích hoạt:

$$z = \sum_{i=1}^n (x_i w_i) + b \quad (2.6)$$

$$y = f(z) \quad (2.7)$$

Trong đó:

- z : Đầu ra tuyến tính (giá trị trước khi đưa qua hàm kích hoạt)
- $x_i(\text{input})$: Các giá trị đầu vào của dữ liệu (ví dụ: đặc trưng của một bức ảnh)

- w_i (Weights): Trọng số biểu thị mức độ quan trọng của đầu vào x_i đối với nơ-ron. Trọng số càng lớn, đầu vào đó càng ảnh hưởng đến kết Z
- b (Bias): Độ lệch (hoặc hằng số): Nó cho phép nơ-ron dịch chuyển đường ranh giới quyết định sang trái hoặc phải, giúp mô hình linh hoạt hơn trong việc khớp với dữ liệu thực tế
- f : Hàm kích hoạt
- y : Đầu ra của nơ-ron

Trong mạng học sâu, nhiều nơ-ron được ghép lại thành từng lớp. Các lớp đầu học đặc trưng đơn giản, còn các lớp sâu hơn học các đặc trưng phức tạp hơn. Đối với bài toán âm thanh, các nơ-ron trong CNN không xử lý trực tiếp sóng âm thô, mà xử lý ảnh đặc trưng Log-Mel Spectrogram.

2.1.4.1. Nguyên lý tích chập hai chiều

Tích chập hai chiều là phép toán quan trọng nhất trong CNN. Trong phép tích chập, một bộ lọc nhỏ gọi là kernel trượt trên dữ liệu đầu vào. Tại mỗi vị trí, các phần tử của kernel được nhân với vùng dữ liệu tương ứng, sau đó cộng lại để tạo ra một giá trị đầu ra. Với ảnh đầu vào X và kernel W kích thước $K \times K$, đầu ra tại vị trí hàng r , cột c được tính như sau:

$$Y[r, c] = \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} X[r + i, c + j] * W[i, j] \quad (2.8)$$

Trong đó:

- X : Feature map đầu vào
- W : Trọng số kernel
- i, j : Chỉ số trong kernel
- r, c : Vị trí không gian trên feature map
- Y : Feature map đầu ra

Trong đề tài này, các lớp convolution sử dụng kernel 3×3 . Kernel 3×3 giúp mô hình học các đặc trưng cục bộ trên vùng nhỏ của ảnh Log-Mel. Ví dụ, một vùng

3×3 có thể biểu diễn sự thay đổi năng lượng âm thanh trong một khoảng thời gian ngắn và một nhóm dải tần gần nhau.

2.1.4.2. Padding và Stride

Khi kernel trượt trên ảnh đầu vào, kích thước đầu ra phụ thuộc vào kích thước input, kích thước kernel, stride và padding. Công thức tính kích thước đầu ra theo một chiều là:

$$Output = \frac{Input + 2 \times \text{Padding} - \text{Kernel}}{\text{Stride}} + 1 \quad (2.9)$$

Trong mô hình CNN của đề tài, các lớp convolution sử dụng kernel 3×3 , stride = 1 và padding = 1. Khi đó, kích thước không gian của feature map được giữ nguyên sau convolution. Việc giữ nguyên kích thước sau convolution giúp mô hình không bị mất thông tin biên quá sớm, đồng thời thuận tiện khi ghép với tầng MaxPooling phía sau.

2.1.4.3. Batch Normalization

Batch Normalization là kỹ thuật chuẩn hóa dữ liệu trung gian trong mạng nơ-ron. Công thức Batch Normalization:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \quad (2.10)$$

$$y = \gamma \hat{x} + \beta \quad (2.11)$$

Trong đó:

- x : giá trị đầu vào.
- μ : Giá trị trung bình
- σ^2 : Phương sai tính toán
- $\hat{x}$: dữ liệu sau khi chuẩn hóa.
- ϵ : Hằng số nhỏ tránh chia cho 0
- γ : Hệ số tỉ lệ
- β : Hệ số dịch chuyển

- y : đầu ra của lớp Batch Normalization.

Trong quá trình huấn luyện, BatchNorm giúp mô hình hội tụ ổn định hơn và có thể giảm hiện tượng overfitting. Trong quá trình suy luận trên phần cứng, BatchNorm có thể được gộp vào trọng số và bias của lớp convolution để giảm số phép toán.

2.1.4.4. Hàm kích hoạt ReLU

Sau các lớp convolution hoặc fully connected, mạng nơ-ron cần một hàm kích hoạt phi tuyến. Nếu không có hàm kích hoạt, toàn bộ mạng nhiều lớp vẫn chỉ tương đương với một phép biến đổi tuyến tính. ReLU là hàm kích hoạt đơn giản và phổ biến trong CNN. Công thức ReLU:

$$\text{ReLU}(x) = \max(0, x) \quad (2.12)$$

Hàm ReLU giữ lại các giá trị dương và đưa các giá trị âm về 0. Đối với phần cứng FPGA, ReLU rất thuận lợi vì chỉ cần kiểm tra bit dấu của số signed. Nếu giá trị âm thì output bằng 0, nếu giá trị dương thì giữ nguyên.

2.1.4.5. MaxPooling

MaxPooling là tầng giảm kích thước không gian của feature map. Khối MaxPooling kích thước cửa sổ 2×2 với bước dịch bằng 2 thực hiện trích xuất giá trị lớn nhất trong vùng không gian. Công thức:

$$Y[r, c] = \max \left\{ \begin{array}{l} X[2r, 2c], \\ X[2r, 2c + 1], \\ X[2r + 1, 2c], \\ X[2r + 1, 2c + 1] \end{array} \right\} \quad (2.13)$$

Trong kiến trúc mạng của đề tài, kích thước không gian của các ma trận đặc trưng sẽ giảm đi một nửa sau mỗi tầng MaxPool: $40 \times 64 \rightarrow 20 \times 32 \rightarrow 10 \times 16 \rightarrow 5 \times 8$. Việc giảm kích thước này giúp giảm số lượng phép toán cho phần classifier, đồng thời giữ lại các đặc trưng mạnh nhất trong ảnh Log-Mel.

2.1.4.6. Fully Connected Layer

Tầng liên kết đầy đủ thực hiện phép nhân ma trận - vector cơ bản. Công thức của Fully Connected Layer:

$$y_j = \sum_{i=0}^{N-1} (x_i \times w_{j,i}) + b_j \quad (2.14)$$

Trong đó:

- $x[i]$: Giá trị đầu vào thứ i
- $w[j, i]$: Trọng số nối từ input i đến output j
- $b[j]$: Bias của node j
- $y[j]$: Giá trị đầu ra của node j

Sau ba tầng tích chập và MaxPooling tuần tự, tensor đặc trưng cuối cùng có kích thước phẳng là:

$$64 \times 5 \times 8 = 2560 \text{ phần tử}$$

Dữ liệu này được làm phẳng (flatten) thành một vector một chiều và đưa qua hai tầng phân lớp:

$$2560 \rightarrow 128 \rightarrow 3$$

Ba giá trị đầu ra thu được là các giá trị logits tương ứng với ba phân lớp âm thanh đích: “cứu”, “cướp” và “unknown”. Lớp dự đoán cuối cùng được xác định bằng phép toán tìm chỉ số có giá trị logit lớn nhất.

2.1.4.7. Fixed-point và lượng tử hóa trong CNN

Trong phần mềm, CNN thường được huấn luyện bằng số thực floating-point. Tuy nhiên, khi triển khai lên FPGA, floating-point tốn nhiều tài nguyên hơn fixed-point. Vì vậy, mô hình cần được lượng tử hóa. Lượng tử hóa là quá trình chuyển một giá trị số thực sang dạng số nguyên fixed-point. Công thức tổng quát:

$$X_{\text{fixed}} = \text{round}(X_{\text{float}} \times 2^F) \quad (2.15)$$

Trong đó:

- F : số bit thập phân

Khi cần chuyển ngược về số thực:

$$X_{\text{float}} \approx X \frac{x_{\text{fixed}}}{2^F} \quad (2.16)$$

Trong thiết kế này, dữ liệu Log-Mel đầu vào sử dụng định dạng số Q6.10 ($F=10$ bit thập phân), trong khi phần lớn trọng số kết nối và dữ liệu trung gian trong mạch CNN được chuẩn hóa về định dạng Q8.8 ($F=8$ bit thập phân). Sau mỗi phép nhân tích lũy (MAC), dữ liệu kết quả sẽ được xử lý làm tròn, dịch phải bit thích hợp và áp dụng mạch bão hòa (saturation) để đưa gọn về độ rộng thanh ghi chuẩn 16-bit, tránh hiện tượng tràn vòng (wrap-around) gây sai lệch dữ liệu.

2.2. Các công trình nghiên cứu liên quan

2.2.1. Triển khai và tăng tốc mạng CNN trên hệ thống nhúng

Vandendriessche và các cộng sự [1] đã nghiên cứu việc triển khai mạng CNN cho bài toán nhận dạng âm thanh môi trường trên các hệ thống nhúng, đặc biệt tập trung vào nền tảng phần cứng FPGA. Tín hiệu âm thanh được chuyển đổi sang biểu diễn phổ Spectrogram hai chiều. Nghiên cứu này đánh giá chuyên sâu hiệu năng của các công cụ tổng hợp phần cứng tự động, bao gồm hls4ml và lõi xử lý đa năng Vitis AI DPU. Kết quả thực nghiệm chỉ ra rằng các công cụ tự động thường gặp giới hạn về bùng nổ tài nguyên (đối với hls4ml) hoặc phải đánh đổi về độ trễ xử lý (đối với DPU). Công trình này là cơ sở quan trọng khẳng định sự cần thiết của việc tự thiết kế và tối ưu hóa phần cứng ở mức thanh ghi (RTL) thủ công để đạt được sự cân bằng tốt nhất giữa tài nguyên và tốc độ xử lý phần cứng.

2.2.2. Nhận diện âm thanh thời gian thực trên nền tảng SoC-FPGA

Da Silva và các cộng sự [2] đề xuất một hệ thống phát hiện tiếng chim thời gian thực sử dụng kiến trúc mạng học sâu lai CNN-RNN (Lightweight GRU) được

triển khai trên nền tảng SoC-FPGA. Công trình này giải quyết bài toán bằng cách trích xuất đặc trưng Log-Mel Spectrogram trực tiếp làm đầu vào và sử dụng phương pháp High-Level Synthesis (HLS) để tổng hợp ra mạch cứng.

Khóa luận này kế thừa hướng tiếp cận sử dụng đặc trưng Log-Mel Spectrogram trên FPGA. Tuy nhiên, thay vì sử dụng các công cụ tổng hợp cấp cao (HLS) vốn tạo ra kiến trúc cứng nhắc, khóa luận tập trung hiện thực hóa toàn bộ luồng xử lý từ tín hiệu thô (bao gồm FFT, Mel-Filterbank, Logarithm và cấu trúc mạch CNN) hoàn toàn bằng ngôn ngữ mô tả phần cứng Verilog HDL. Thiết kế thủ công kết hợp với lượng tử hóa số cố định (fixed-point) giúp hệ thống tối ưu hóa triệt để tài nguyên, đáp ứng yêu cầu hoạt động thời gian thực cho bài toán đặc thù nhận diện tiếng kêu khẩn cấp.

2.2.3. So sánh với các công trình nghiên cứu có liên quan

Bảng 2.1: So sánh các công trình nghiên cứu

Nội dung	Vandendriessche et al. [1]	da Silva et al. [2]	Khóa luận
Mục tiêu	Đánh giá và so sánh hiệu năng triển khai CNN trên hệ thống nhúng (sử dụng hls4ml và Vitis AI DPU) cho phân loại âm thanh môi trường.	Xây dựng hệ thống phát hiện tiếng chim thời gian thực trên SoC-FPGA sử dụng mạng học sâu lai CNN-RNN.	Thiết kế các lõi IP cần thiết cho bộ nhận diện tiếng kêu khẩn cấp (FFT, Log-Mel, AI model,...) và kiểm chứng, tích hợp trên FPGA.
Đặc trưng đầu vào	Spectrogram (xử lý dưới dạng ảnh tĩnh 2D).	Đặc trưng Log-Mel Spectrogram.	Đặc trưng Log-Mel Spectrogram có kích thước 40×64 .

Phương pháp phân loại	Mạng CNN 2D, kiến trúc phần cứng được sinh tự động (HLS, DPU).	Mạng CNN-RNN (GRU), kiến trúc phần cứng được sinh tự động (HLS).	CNN gồm 3 tầng Conv-ReLU-MaxPooling, 2 lớp Fully Connected.
Đầu ra mô hình	Phân loại 50 lớp âm thanh môi trường (tập dữ liệu ESC-50).	Phân loại nhị phân (Xác định có hoặc không có tiếng chim).	Ba giá trị logits tương ứng với các lớp: Cứu, Cướp, Unknown.
Triển khai trên phần cứng	Đã triển khai trên FPGA (ZCU104) thông qua luồng thiết kế tự động của Xilinx.	Đã triển khai trên SoC-FPGA (ZU3CG) sử dụng công cụ High-Level Synthesis.	Toàn bộ pipeline xử lý được hiện thực thủ công cấp thanh ghi (RTL), mô phỏng, kiểm thử và tổng hợp đánh giá tài nguyên FPGA (ZCU104).
Biểu diễn số	Fixed-point (QAT 8-bit).	Fixed-point (16-bit).	Fixed-point 16-bit.
Phạm vi hiện thực	Chỉ tập trung xây dựng và đánh giá lỗi tăng tốc CNN.	Chỉ tập trung xây dựng lõi phân loại CNN-RNN.	Tích hợp hoàn chỉnh Pipeline phần cứng từ tín hiệu âm thanh thô: FFT 1024 điểm, tính phổ công suất, Mel filterbank, Log-Mel Spectrogram và mạng CNN

Chương 3. THIẾT KẾ HỆ THỐNG

Chương này trình bày quá trình thiết kế và hiện thực phần cứng cho lõi IP phát hiện âm thanh khẩn cấp trong môi trường công cộng. Trọng tâm của chương là kiến trúc RTL, cách tổ chức các khối xử lý, cơ chế truyền dữ liệu dạng streaming và các kỹ thuật tối ưu để đáp ứng yêu cầu triển khai trên FPGA.

Hệ thống phần cứng được xây dựng theo hướng phân rã thành các mô-đun độc lập, bao gồm khối FFT, khối trích xuất đặc trưng Log-Mel Spectrogram và khối CNN suy luận. Các mô-đun được kết nối theo mô hình pipeline để tăng thông lượng xử lý, đồng thời sử dụng số cố định nhằm giảm chi phí tài nguyên và phù hợp với đặc thù của phần cứng số.

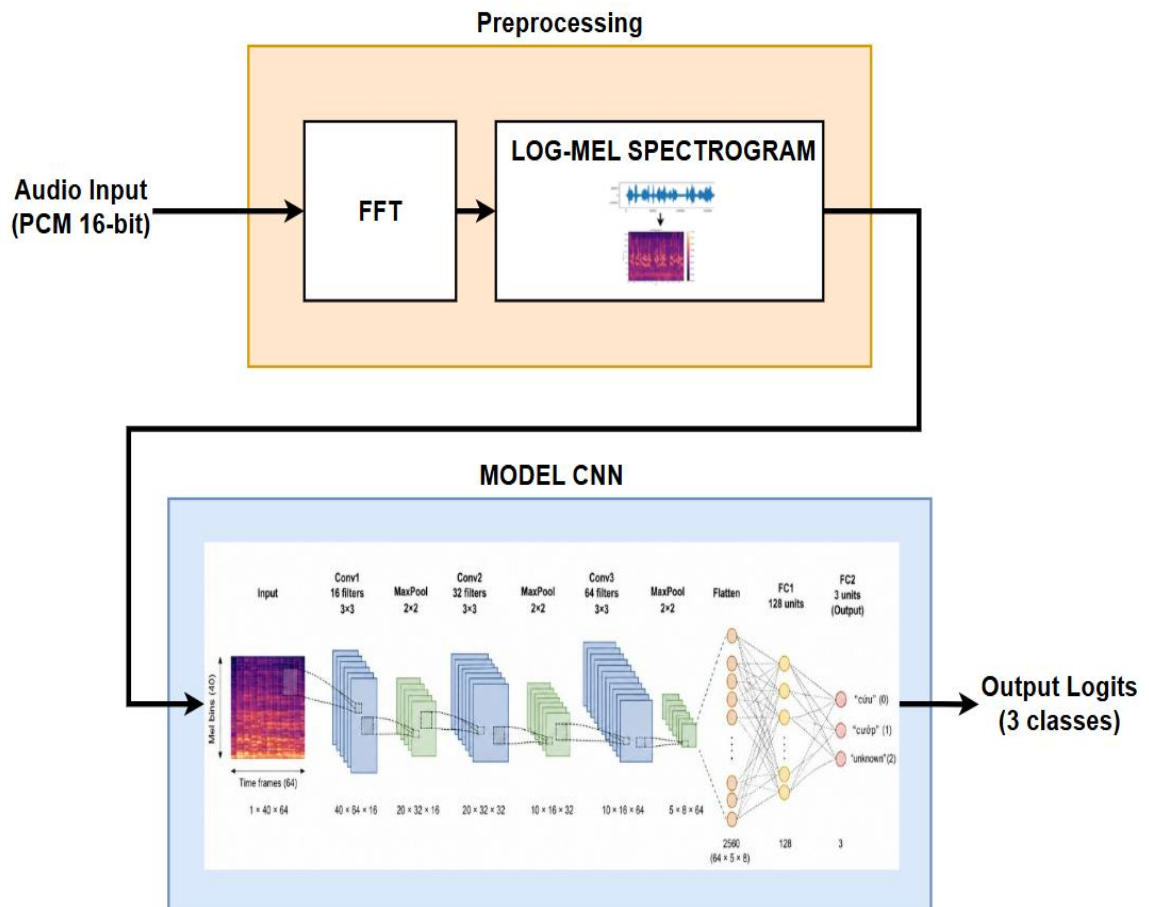
Nội dung của chương tập trung vào kiến trúc tổng thể của hệ thống trên phần cứng, mô tả chi tiết các khối RTL chính, giao thức đồng bộ dữ liệu và phương thức tích hợp các mô-đun thành một lõi IP hoàn chỉnh.

3.1. Kiến trúc tổng quan của hệ thống trên phần cứng

Hệ thống phần cứng được xây dựng theo hướng phân chia thành các khối IP độc lập, trong đó mỗi khối đảm nhiệm một chức năng xử lý riêng biệt và giao tiếp với nhau thông qua giao diện truyền dữ liệu dạng luồng (streaming). Cách tiếp cận này giúp đơn giản hóa quá trình thiết kế, kiểm chứng và tích hợp hệ thống, đồng thời cho phép tái sử dụng từng khối IP trong các ứng dụng xử lý tín hiệu âm thanh khác.

Kiến trúc tổng thể của hệ thống bao gồm ba khối chức năng chính: FFT, Log-Mel Spectrogram và CNN. Dữ liệu được xử lý theo mô hình pipeline, trong đó đầu ra của khối trước sẽ trở thành đầu vào của khối kế tiếp. Nhờ cơ chế xử lý tuần tự này, các khối có thể hoạt động tương đối độc lập, giúp nâng cao khả năng mở rộng và thuận lợi cho việc tối ưu tài nguyên phần cứng. Trong quá trình xử lý, tốc độ sinh dữ liệu output từ khối Log-Mel Spectrogram và tốc độ xử lý dữ liệu của model CNN giống nhau. Do đó, cần có một bộ đệm FIFO được thiết kế và bố trí giữa khối Log-

Mel Spectrogram và khối CNN nhằm thực hiện chức năng tích lũy và đồng bộ dữ liệu tính toán, nhằm tránh gây xung đột dữ liệu.



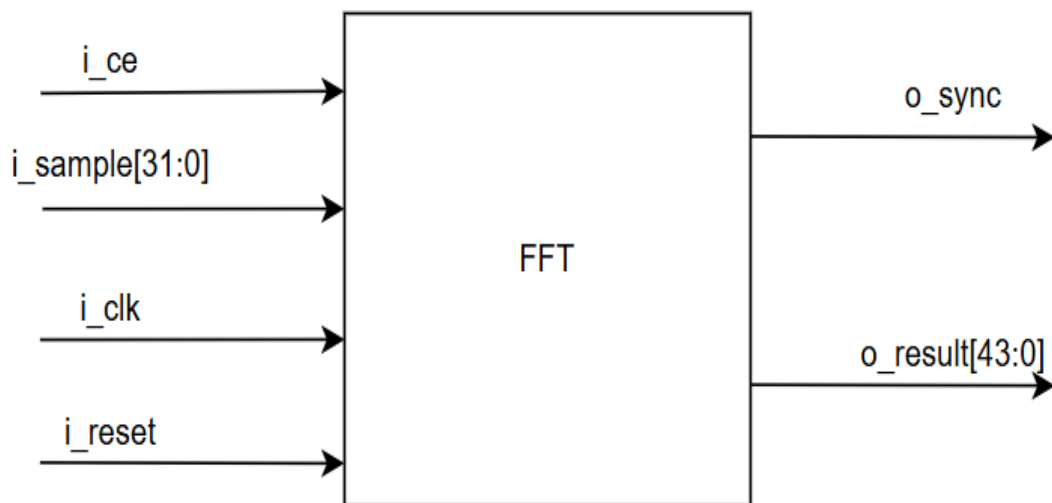
Hình 3.1: Sơ đồ khối hệ thống

Bên cạnh tính mô-đun, hệ thống còn được thiết kế theo kiến trúc pipeline nhằm khai thác khả năng xử lý song song của phần cứng. Nhờ đó, nhiều giai đoạn xử lý được thực hiện đồng thời trong cùng khoảng thời gian hoạt động của hệ thống thay vì phải chờ hoàn thành tuần tự từng bước. Việc áp dụng cơ chế pipeline giúp tăng thông lượng xử lý, cho phép các mẫu dữ liệu đầu vào được đưa vào hệ thống liên tục theo từng chu kỳ xung nhịp.

3.2. Thiết kế khối FFT

Khối FFT là thành phần đầu tiên trong chuỗi xử lý số của hệ thống phần cứng và đóng vai trò chuyển đổi tín hiệu âm thanh từ miền thời gian sang miền tần số.

Trong đề tài này, khối FFT được xây dựng dựa trên lõi mã nguồn mở dblockfft của ZipCPU, sau đó được cấu hình lại theo yêu cầu của hệ thống để phù hợp với kiến trúc xử lý một luồng dữ liệu vào và một luồng dữ liệu ra. Việc lựa chọn FFT dạng pipeline giúp khối có khả năng tiếp nhận dữ liệu liên tiếp theo từng chu kỳ clock, đồng thời tạo nền tảng thuận lợi cho việc tích hợp vào toàn bộ chuỗi xử lý Log-Mel và CNN sau đó.



Hình 3.2: Top-module của FFT

Khối FFT trong đề tài được cấu hình với kích thước cố định 1024 điểm và xử lý tín hiệu phức theo dạng bù hai (two's complement). Dữ liệu đầu vào có độ rộng 16 bit cho phần thực và 16 bit cho phần ảo, tương ứng với tổng cộng 32 bit cho mỗi mẫu phức. Đầu ra của FFT được mở rộng lên 22 bit cho mỗi thành phần thực và ảo, tức tổng cộng 44 bit, nhằm bảo đảm đủ biên độ số học trong quá trình xử lý trung gian (hạn chế nguy cơ tràn số và mất mát thông tin trong các phép toán).

Bảng 3.1: Các port của FFT

Tên tín hiệu	I/O	Độ rộng	Mô tả
i_clk	Input	1 bit	Tín hiệu xung nhịp hệ thống, đồng bộ toàn bộ hoạt động của khối FFT.

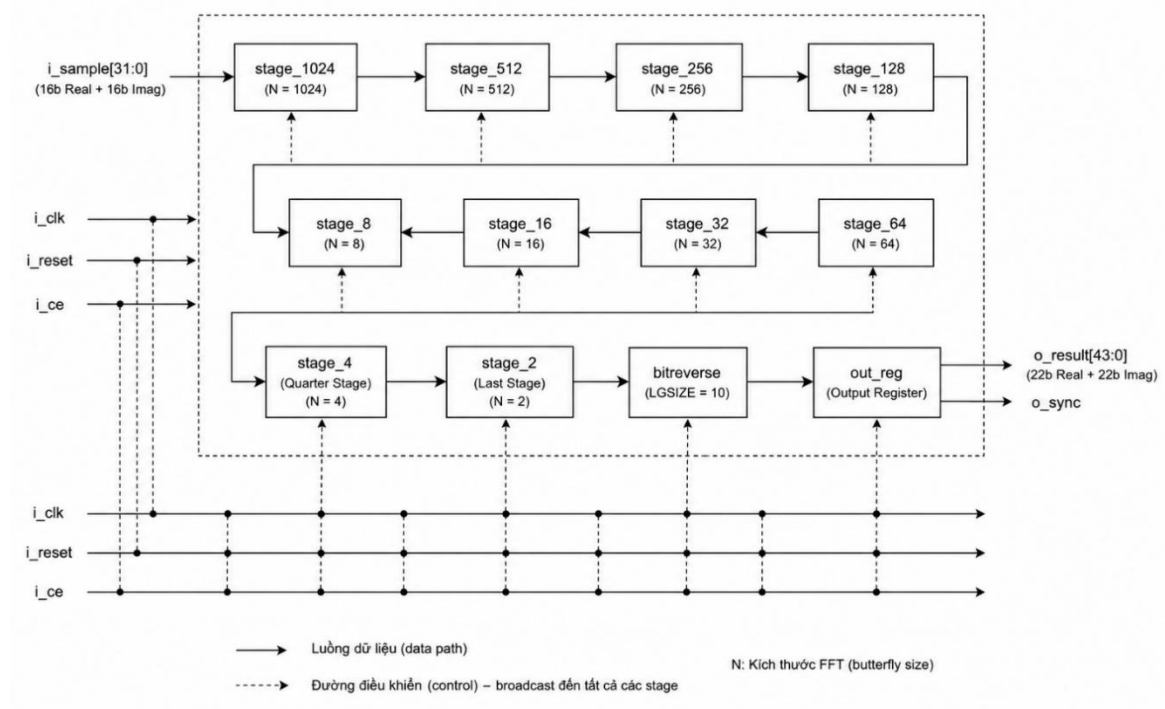
i_reset	Input	1 bit	Tín hiệu reset đồng bộ mức cao, dùng để khởi tạo lại các thanh ghi và trạng thái bên trong khối FFT.
i_ce	Input	1 bit	Tín hiệu cho phép xử lý (Clock Enable). Khi ở mức '1', khối FFT tiếp nhận và xử lý dữ liệu đầu vào.
i_sample[31:0]	Input	32 bit	Dữ liệu đầu vào dạng số phức, gồm 16 bit phần thực và 16 bit phần ảo theo định dạng bù hai (two's complement).
o_sync	Output	1 bit	Tín hiệu đồng bộ đầu ra, đánh dấu mẫu đầu tiên của một khung FFT và cho biết dữ liệu đầu ra hợp lệ.
o_result[43:0]	Output	44 bit	Kết quả FFT đầu ra dạng số phức, gồm 22 bit phần thực và 22 bit phần ảo.

Về mặt tổ chức logic, FFT được hiện thực theo cấu trúc pipeline nhiều tầng. Tín hiệu đầu vào đi lần lượt qua các giai đoạn xử lý theo kích thước cánh bướm giảm dần từ stage $1024 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2$, sau đó qua các tầng xử lý cuối để hoàn tất phép biến đổi. Trong module top, các tầng này được ghép nối tuần tự thông qua các khối fftstage, qtrstage, laststage và cuối cùng là khối bitreverse. Cấu trúc này cho phép toàn bộ FFT hoạt động liên tục trên luồng dữ liệu streaming, mỗi chu kỳ clock có thể tiếp nhận một mẫu mới trong khi các mẫu trước đó đang được xử lý ở các tầng phía sau.

Một điểm quan trọng của khối FFT trong đề tài là cách xử lý bit-reversal ở cuối chuỗi biến đổi. Thay vì xuất kết quả theo thứ tự tự nhiên của các bước xử lý trung gian, module bitreverse được dùng để sắp xếp lại phổ đầu ra về đúng thứ tự tần số trước khi chuyển sang khối Log-Mel Spectrogram. Điều này giúp đảm bảo dữ liệu

FFT đưa sang bước trích xuất đặc trưng phù hợp với yêu cầu của Mel filterbank ở giai đoạn sau.

Về phương diện thiết kế số học, khối FFT sử dụng cơ chế làm tròn và mở rộng độ rộng dữ liệu theo từng tầng để hạn chế tràn số trong quá trình cộng và nhân phức. Các hệ số xoay pha được lưu trong các file ROM hệ số riêng cho từng tầng, ví dụ cmem_1024.hex, cmem_512.hex, cmem_256.hex, ... Nhờ đó, FFT không cần tính toán hệ số cosine/sine động trong thời gian chạy mà chỉ đọc từ bộ nhớ hệ số đã sinh sẵn, giúp giảm độ phức tạp phần cứng và tăng tính ổn định khi triển khai trên FPGA.



Hình 3.3: Sơ đồ khối của FFT

Trong hệ thống của đề tài, khối FFT không được thiết kế như một module xử lý độc lập, mà được tích hợp vào kiến trúc tổng thể của lõi IP. Mỗi mẫu âm thanh đầu vào sau khi được đưa vào khối FFT sẽ lần lượt tạo ra phổ tần số để làm cơ sở cho các khối Log-Mel Spectrogram và CNN phía sau. Như vậy, FFT vừa là khối mở đầu của toàn bộ chuỗi xử lý, vừa là điểm nối giữa miền tín hiệu âm thanh thô và miền đặc trưng dùng cho mô hình học sâu.

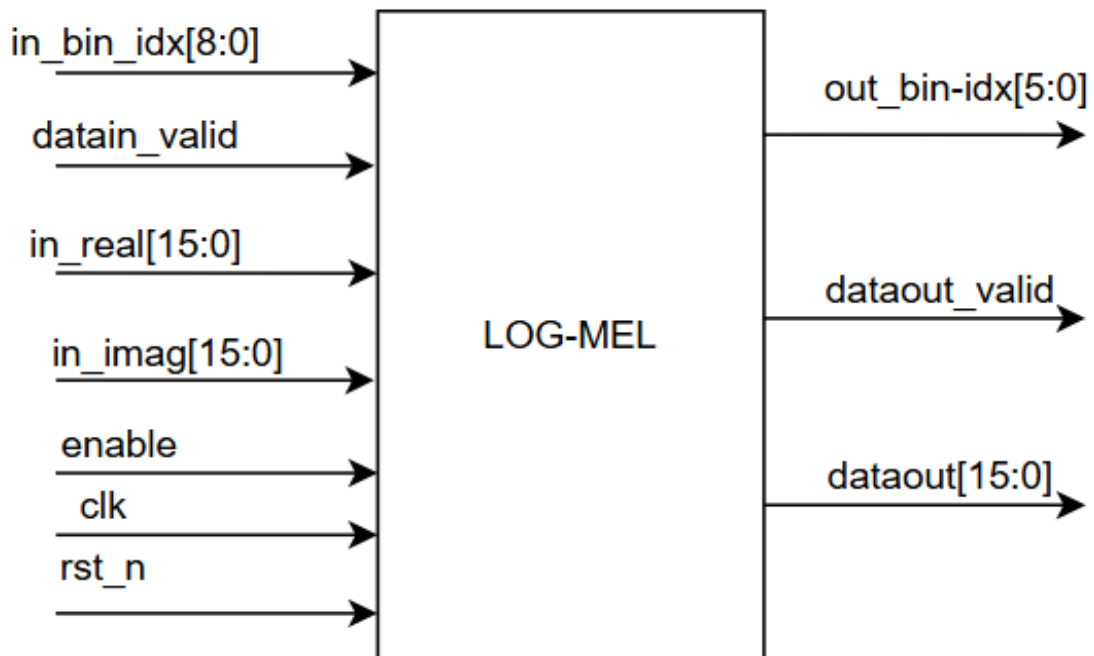
3.3. Thiết kế khối Log-Mel Spectrogram

Khối Log-Mel Spectrogram có nhiệm vụ chuyển đổi dữ liệu phổ tần số thu được từ khối FFT thành đặc trưng Log-Mel Spectrogram trước khi đưa vào mạng CNN. Đây là bước trích xuất đặc trưng quan trọng, giúp biểu diễn tín hiệu âm thanh dưới dạng gần với cơ chế cảm nhận tần số của thính giác con người, đồng thời làm giảm kích thước dữ liệu đầu vào cho mô hình phân loại.

Trong đề tài này, khối Log-Mel Spectrogram được thiết kế hoàn toàn bằng phần cứng RTL và được tổ chức theo kiến trúc pipeline gồm ba khối xử lý chính:

- Khối tính công suất phổ (Power Unit);
- Khối Mel filterbank (MAC Unit);
- Khối Logarithm (Log Unit).

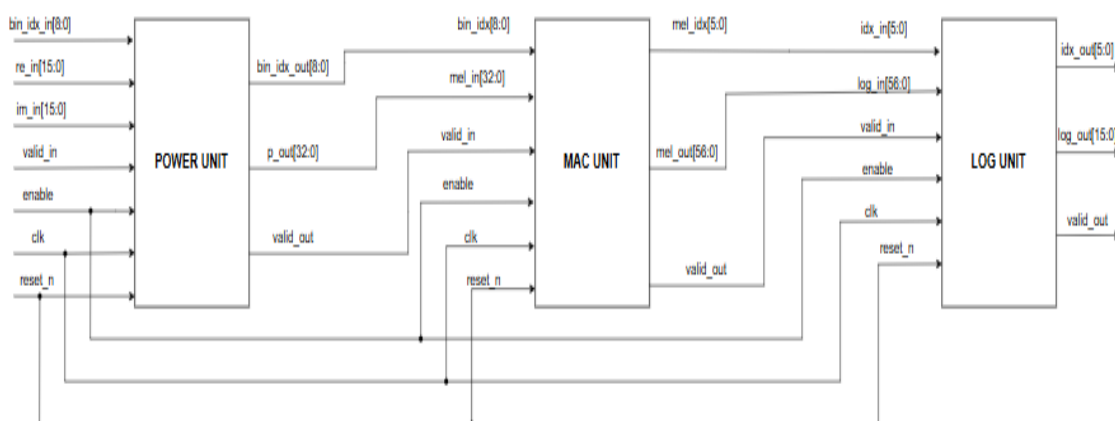
Dữ liệu đầu vào của khối là kết quả FFT dưới dạng phần thực và phần ảo của từng bin tần số. Các giá trị này lần lượt đi qua ba giai đoạn xử lý để tạo ra đặc trưng Log-Mel tương ứng.



Hình 3.4: Top-module của Log-Mel Spectrogram

Bảng 3.2: Các port của Log-Mel Spectrogram

Tên tín hiệu	Hướng	Độ rộng	Mô tả
clk	Input	1 bit	Tín hiệu clock hệ thống.
reset_n	Input	1 bit	Tín hiệu reset tích cực mức thấp.
enable	Input	1 bit	Cho phép toàn bộ khối Log-Mel Spectrogram hoạt động.
datain_valid	Input	1 bit	Báo hiệu dữ liệu FFT đầu vào hợp lệ.
in_real	Input	16 bit	Thành phần thực của hệ số FFT.
in_imag	Input	16 bit	Thành phần ảo của hệ số FFT.
in_bin_idx	Input	9 bit	Chỉ số bin tần số FFT tương ứng với dữ liệu đầu vào.
dataout_valid	Output	1 bit	Báo hiệu dữ liệu Log-Mel đầu ra hợp lệ.
dataout	Output	16 bit	Giá trị Log-Mel Spectrogram sau khi tính toán.
out_bin_idx	Output	6 bit	Chỉ số Mel bin tương ứng với dữ liệu đầu ra.



Hình 3.5: Sơ đồ khối Log-Mel Spectrogram

Luồng dữ liệu trong hệ thống được thực hiện theo kiểu streaming. Khi một mẫu FFT hợp lệ xuất hiện tại đầu vào, dữ liệu sẽ được xử lý liên tục qua các tầng pipeline mà không cần lưu toàn bộ phổ FFT trước khi tính toán. Cơ chế này cho phép nhiều khối hoạt động song song trên các tập dữ liệu khác nhau trong cùng một thời điểm, qua đó nâng cao thông lượng xử lý và giảm độ trễ toàn hệ thống.

Tại tầng đầu tiên, khối Power Unit thực hiện tính năng lượng của từng bin FFT. Kết quả năng lượng phổ sau đó được đưa tới khối MAC Unit để thực hiện phép nhân – cộng tích lũy với các hệ số Mel filterbank được lưu trong ROM. Quá trình này tạo ra các hệ số năng lượng Mel tương ứng với từng bộ lọc. Tiếp theo, các giá trị Mel Energy được đưa qua khối Log Unit để thực hiện phép biến đổi logarithm cơ số 2 bằng phương pháp LUT kết hợp chuẩn hóa dữ liệu. Kết quả cuối cùng là các giá trị Log-Mel Spectrogram được lượng tử hóa cố định 16 bit, phù hợp với định dạng dữ liệu đầu vào của mô hình CNN trên phần cứng.

Để đảm bảo khả năng đồng bộ dữ liệu giữa các tầng pipeline, mỗi khối đều sử dụng các tín hiệu điều khiển gồm clk, enable, valid_in và valid_out. Ngoài dữ liệu đặc trưng, chỉ số bin FFT (bin_idx) và chỉ số Mel Filter (mel_idx) cũng được truyền xuyên suốt giữa các tầng nhằm phục vụ việc điều khiển truy cập bộ nhớ và đồng bộ hóa luồng xử lý.

3.3.1. Mô-đun power_unit

Khối Power Unit là tầng xử lý đầu tiên của module Log-Mel Spectrogram, có nhiệm vụ chuyển đổi kết quả FFT từ miền số phức sang miền năng lượng. Đầu vào của khối là thành phần thực và thành phần ảo của từng hệ số FFT, trong khi đầu ra là công suất phổ tương ứng của mỗi bin tần số.

Về mặt toán học, công suất phổ được tính theo công thức:

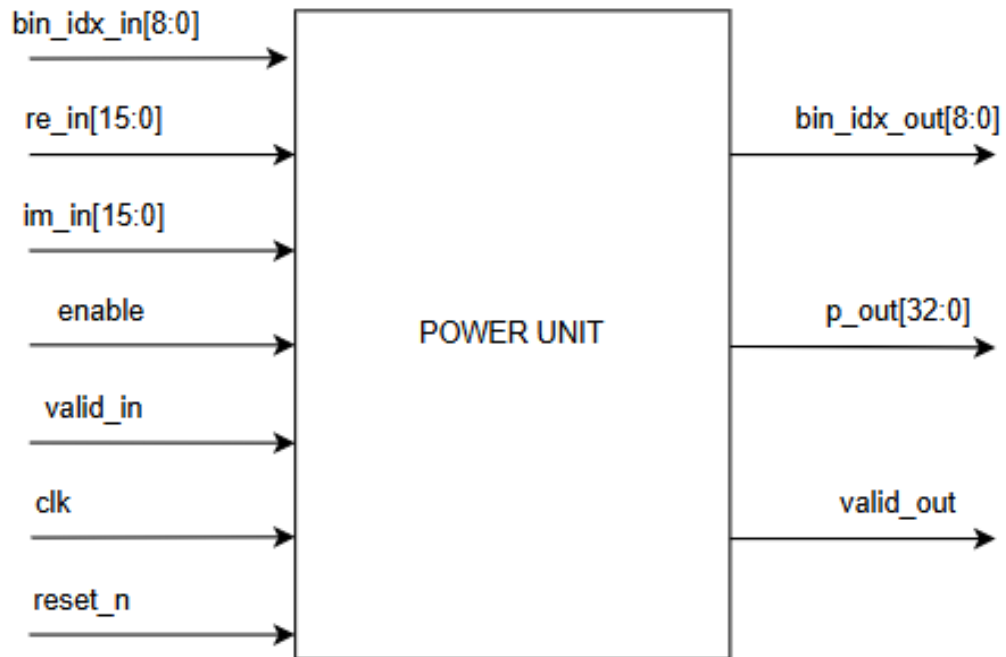
$$P(k) = \text{Re}(k)^2 + \text{Im}(k)^2 \quad (3.1)$$

Trong đó:

- $\text{Re}(k)$ là phần thực của hệ số FFT.

- $\text{Im}(k)$ là phần ảo của hệ số FFT.
- $P(k)$ là năng lượng của bin tần số thứ k .

Giá trị công suất phổ thu được sẽ được sử dụng làm đầu vào cho bộ lọc Mel ở tầng xử lý tiếp theo.

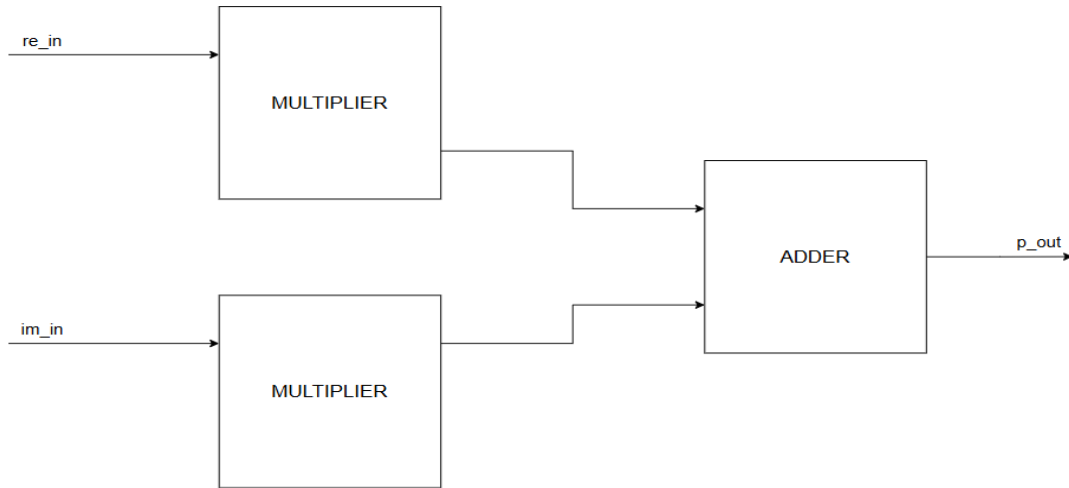


Hình 3.6: Mô-đun power_unit

Bảng 3.3: Các port của power_unit

Tín hiệu	I/O	Độ rộng	Mô tả
clk	Input	1 bit	Clock hệ thống
reset_n	Input	1 bit	Reset tích cực mức thấp
enable	Input	1 bit	Cho phép khởi hoạt động
re_in	Input	16 bit	Thành phần thực của FFT
im_in	Input	16 bit	Thành phần ảo của FFT
valid_in	Input	1 bit	Dữ liệu đầu vào hợp lệ
bin_idx_in	Input	9 bit	Chỉ số FFT bin tương ứng

p_out	Output	33 bit	Công suất phổ $\text{Re}^2 + \text{Im}^2$
valid_out	Output	1 bit	Cờ dữ liệu đầu ra hợp lệ
bin_idx_out	Output	9 bit	Chỉ số FFT bin sau pipeline



Hình 3.7: Sơ đồ khối của power_unit

Khối Power Unit được xây dựng theo kiến trúc pipeline hai tầng nhằm tăng tần số hoạt động và giảm độ dài đường dữ liệu tổ hợp.

❖ Giai đoạn 1: Bình phương tín hiệu FFT

Tại tầng đầu tiên, phần thực và phần ảo của hệ số FFT được đưa vào hai bộ nhân song song để tính giá trị bình phương: Re^2 và Im^2 .

Do dữ liệu đầu vào sử dụng độ rộng 16 bit có dấu, kết quả của mỗi phép nhân được mở rộng lên 32 bit nhằm tránh mất dữ liệu trong quá trình tính toán. Đồng thời tín hiệu điều khiển valid_in và chỉ số bin FFT (bin_idx_in) cũng được lưu qua các thanh ghi pipeline để đảm bảo đồng bộ dữ liệu giữa các tầng.

❖ Giai đoạn 2: Tính công suất phổ

Ở tầng thứ hai, hai kết quả bình phương được cộng lại để tạo thành năng lượng của bin tần số: $P(k) = \text{Re}(k)^2 + \text{Im}(k)^2$

Do tổng của hai số 32 bit có thể phát sinh bit nhớ (carry), đầu ra được mở rộng thành 33 bit nhằm đảm bảo không xảy ra tràn số trong quá trình cộng.

Sau khi phép cộng hoàn tất, khối xuất ra:

- Giá trị công suất phổ p_out .
- Chỉ số bin FFT tương ứng bin_idx_out .
- Tín hiệu xác nhận dữ liệu hợp lệ $valid_out$.

3.3.2. Mô-đun mac_unit

Khối MAC Unit có nhiệm vụ thực hiện phép biến đổi từ miền phổ công suất FFT sang miền Mel bằng cách áp dụng bộ lọc Mel filterbank. Đây là khối trung tâm của module Log-Mel Spectrogram, chịu trách nhiệm tính toán năng lượng của các Mel bin từ dữ liệu phổ đầu vào. Về nguyên lý, năng lượng của Mel bin thứ m được xác định bởi:

$$S(m) = \sum_{k=0}^{N-1} P(k)W_m(k) \quad (3.2)$$

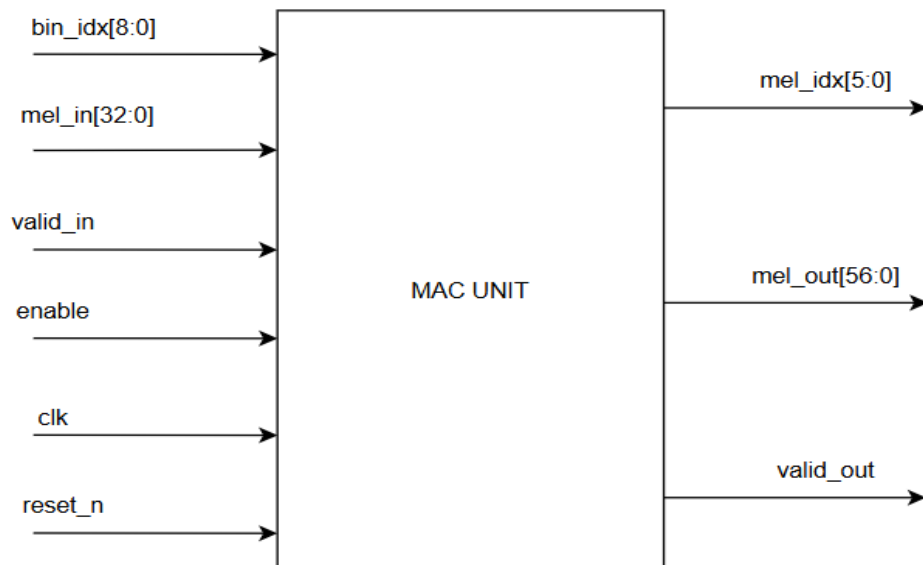
Trong đó:

- $P(k)$ là công suất của FFT bin thứ k ;
- $W_m(k)$ là hệ số của bộ lọc Mel thứ m ;
- $S(m)$ là năng lượng Mel thu được sau quá trình tích lũy.

Bảng 3.4: Các port của mac_unit

Tín hiệu	I/O	Độ rộng	Mô tả
clk	Input	1 bit	Clock hệ thống
reset_n	Input	1 bit	Reset tích cực mức thấp
enable	Input	1 bit	Cho phép khối hoạt động
valid_in	Input	1 bit	Cờ dữ liệu đầu vào hợp lệ

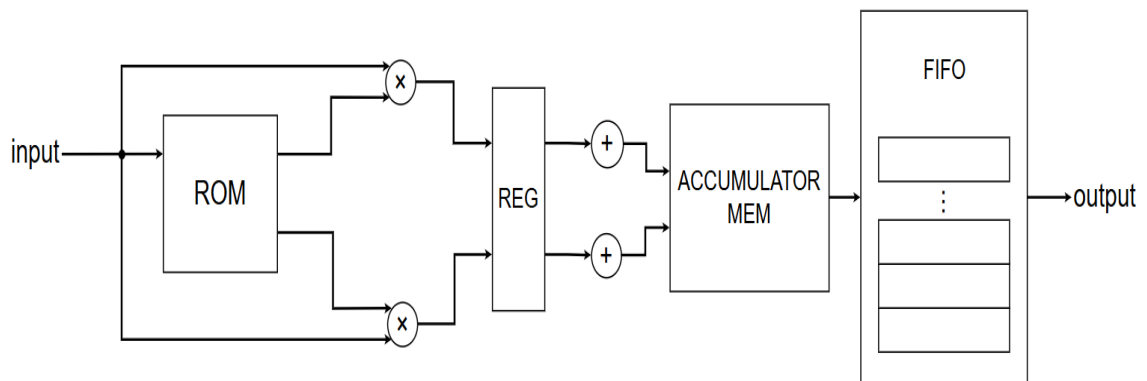
mel_in	Input	33 bit	Giá trị năng lượng phổ từ Power Unit
bin_idx	Input	9 bit	Chỉ số FFT bin tương ứng
valid_out	Output	1 bit	Cờ dữ liệu đầu ra hợp lệ
mel_out	Output	58 bit	Năng lượng tích lũy của bộ lọc Mel
mel_idx	Output	6 bit	Chỉ số bộ lọc Mel (0–39)



Hình 3.8: Mô-đun mac_unit

Khác với cách hiện thực truyền thống lưu toàn bộ ma trận Mel filterbank, hệ thống sử dụng một bộ nhớ ROM chứa thông tin ánh xạ giữa các FFT bin và các Mel filter tương ứng. Mỗi địa chỉ ROM chứa tối đa hai hệ số trọng số cùng hai chỉ số Mel filter tương ứng. Nghĩa là với mỗi giá trị năng lượng FFT bin chỉ có thể tương ứng tối đa với hai chỉ số tam giác Mel filter, các giá trị trong Mel filter tồn tại tam giác chỉ thuộc một vùng bin nhất định chứ không trải dài hoàn toàn frame, các giá trị ngoài vùng tam giác đều bằng 0. Cấu trúc này tận dụng đặc tính của bộ lọc tam giác Mel,

trong đó một FFT bin chỉ đóng góp cho tối đa hai bộ lọc Mel lân cận, giúp giảm đáng kể dung lượng bộ nhớ cần thiết.



Hình 3.9: Sơ đồ khối mac_unit

Kiến trúc pipeline: MAC Unit được thiết kế theo kiến trúc pipeline gồm ba giai đoạn chính.

❖ Giai đoạn 1 – Truy xuất ROM

MAC Unit nhận giá trị công suất phổ từ Power Unit cùng với chỉ số FFT bin tương ứng. Chỉ số bin được sử dụng làm địa chỉ truy xuất bộ nhớ ROM chứa các hệ số Mel filterbank đã được sinh trước bằng phần mềm Python. Mỗi từ nhớ ROM có độ rộng 48 bit và được đóng gói theo cấu trúc:

Bảng 3.5: Cấu trúc dữ liệu của mỗi từ nhớ trong ROM

Trường dữ liệu	Độ rộng
w0	16 bit
w1	16 bit
f0	6 bit
f1	6 bit
done_f0	1 bit
done_f1	1 bit

Trong đó:

- w_0, w_1 là hai hệ số trọng số của Mel filterbank được lượng tử hóa theo định dạng Q0.15.
- f_0, f_1 là chỉ số của hai Mel filter mà FFT bin hiện tại đóng góp năng lượng.
- $done_f_0, done_f_1$ là các cờ đánh dấu FFT bin cuối cùng thuộc Mel filter tương ứng.

Do đặc tính của Mel filterbank dạng tam giác, mỗi FFT bin chỉ có thể thuộc tối đa hai bộ lọc Mel lân cận. Vì vậy ROM chỉ cần lưu tối đa hai hệ số và hai chỉ số Mel filter cho mỗi FFT bin, giúp giảm đáng kể dung lượng bộ nhớ so với việc lưu toàn bộ ma trận hệ số. Sau khi đọc ROM, toàn bộ dữ liệu được đưa vào các thanh ghi để đồng bộ với các giai đoạn nhân và tích lũy ở phía sau.

❖ Giai đoạn 2 – Nhân trọng số

Giá trị công suất phổ đầu vào được nhân đồng thời với hai hệ số trọng số lấy từ ROM là w_0 và w_1 . Hai phép nhân được thực hiện song song nhằm tăng thông lượng xử lý và tận dụng khả năng pipeline của hệ thống.

Kết quả của các phép nhân cùng với chỉ số Mel filter tương ứng được lưu lại trong các thanh ghi trung gian để phục vụ cho quá trình tích lũy.

❖ Giai đoạn 3 – Tích lũy năng lượng Mel

Kết quả nhân được cộng dồn vào bộ nhớ tích lũy tương ứng với từng Mel filter. Mỗi Mel filter được cấp phát một vùng nhớ tích lũy riêng trong mảng `acc_mem`. Khi một giá trị mới được sinh ra, kết quả sẽ được cộng vào giá trị đã tích lũy trước đó để hình thành tổng năng lượng cuối cùng của Mel filter.

Do các FFT bin được xử lý liên tục theo dạng streaming nên khả năng xảy ra hiện tượng đọc sau ghi (Read-After-Write Hazard) là rất cao. Để giải quyết vấn đề này, module sử dụng cơ chế forwarding. Nếu một Mel filter vừa được cập nhật ở chu kỳ trước và tiếp tục được truy cập ở chu kỳ hiện tại, giá trị mới nhất sẽ được lấy trực tiếp từ thanh ghi forwarding thay vì đọc lại từ bộ nhớ tích lũy. Nhờ đó kết quả tích lũy luôn chính xác ngay cả khi các truy cập liên tiếp xảy ra trên cùng một Mel filter.

Cơ chế quản lý frame bằng Phase

Để hỗ trợ xử lý liên tục nhiều frame FFT liên tiếp, MAC Unit sử dụng cơ chế phase accumulation. Mỗi Mel filter được gắn thêm một bit phase lưu trong mảng acc_phase. Khi FFT frame mới bắt đầu, hệ thống chỉ cần đảo giá trị phase hiện hành thay vì xóa toàn bộ bộ nhớ tích lũy.

Trong quá trình cập nhật, nếu phase của ô nhớ khác với phase hiện tại của hệ thống thì giá trị mới sẽ được ghi đè trực tiếp. Ngược lại, dữ liệu sẽ tiếp tục được cộng dồn như bình thường.

Giải pháp này loại bỏ hoàn toàn thời gian khởi tạo lại bộ nhớ giữa các frame và cho phép hệ thống hoạt động liên tục theo cơ chế streaming.

Cơ chế phát hiện hoàn thành Mel bin

Trong ROM, mỗi FFT bin cuối cùng của một Mel filter được đánh dấu bằng tín hiệu Done Flag. Khi kết quả tích lũy cuối cùng của một Mel filter được tạo ra, giá trị hoàn chỉnh sẽ được lưu vào các thanh ghi done_data cùng với chỉ số Mel filter tương ứng.

Nhờ cơ chế này, hệ thống không cần đợi xử lý hết toàn bộ 512 FFT bin mới xuất dữ liệu. Các Mel filter hoàn thành trước có thể được xuất trước, giúp giảm độ trễ toàn hệ thống.

Bộ đệm FIFO đầu ra

Các giá trị Mel đã hoàn thành được đưa vào FIFO nội bộ trước khi truyền sang khối Log Unit. FIFO đóng vai trò bộ đệm giữa khối tích lũy và khối xử lý logarithm, giúp:

- Hấp thụ chênh lệch tốc độ giữa các khối xử lý.
- Ngăn mất dữ liệu khi nhiều Mel filter hoàn thành liên tiếp.
- Duy trì luồng dữ liệu streaming liên tục.

Mỗi phần tử FIFO lưu trữ đồng thời:

- Chỉ số Mel filter.
- Giá trị năng lượng Mel đã tích lũy hoàn chỉnh.

Khối Output Stream đọc FIFO theo cơ chế tuần tự và phát ra các tín hiệu:

- mel_out: giá trị năng lượng Mel.
- mel_idx: chỉ số Mel filter tương ứng.
- valid_out: tín hiệu báo dữ liệu hợp lệ.

3.3.3. Mô-đun log_unit

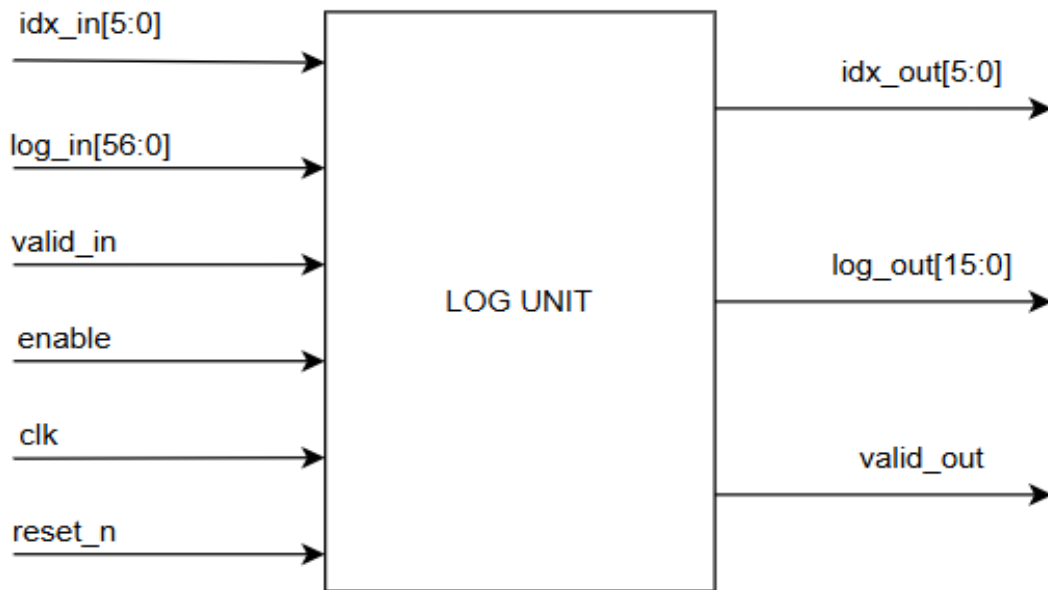
Khối Log Unit có nhiệm vụ chuyển đổi giá trị năng lượng Mel-Spectrogram từ khối MAC Unit sang miền logarit nhằm tạo ra đặc trưng Log-Mel Spectrogram phục vụ cho mạng CNN. Việc áp dụng phép logarit giúp nén dải động của tín hiệu, giảm sự chênh lệch giữa các thành phần năng lượng lớn và nhỏ, đồng thời mô phỏng gần hơn đặc tính cảm nhận âm thanh của hệ thính giác con người.

Trong các hệ thống xử lý tín hiệu số trên phần cứng, việc hiện thực trực tiếp hàm logarit thường đòi hỏi tài nguyên tính toán lớn và độ trễ cao. Vì vậy, đề tài sử dụng phương pháp MSB Detector kết hợp Look-Up Table (LUT) để xấp xỉ hàm logarit cơ số hai. Phương pháp này chỉ yêu cầu các phép dịch bit, so sánh và truy xuất bộ nhớ ROM, qua đó giảm đáng kể độ phức tạp phần cứng nhưng vẫn đảm bảo độ chính xác cần thiết.

Bảng 3.6: Các port của log_unit

Tên tín hiệu	Hướng	Độ rộng	Mô tả
clk	Input	1 bit	Tín hiệu clock hệ thống
reset_n	Input	1 bit	Reset tích cực mức thấp
enable	Input	1 bit	Cho phép khối hoạt động
valid_in	Input	1 bit	Báo hiệu dữ liệu đầu vào hợp lệ
log_in	Input	57 bit	Giá trị năng lượng Mel sau tích lũy từ mac_unit
idx_in	Input	6 bit	Chỉ số bộ lọc Mel tương ứng

log_out	Output	16 bit	Giá trị Log-Mel Spectrogram sau chuẩn hóa
valid_out	Output	1 bit	Báo hiệu dữ liệu đầu ra hợp lệ
idx_out	Output	6 bit	Chỉ số bộ lọc Mel tương ứng với dữ liệu đầu ra



Hình 3.10: Mô-đun log_unit

Nguyên lý tính toán: Giả sử giá trị năng lượng đầu vào là E , ta có thể biểu diễn:

$$E = 2^n \cdot (1 + f) \quad (3.3)$$

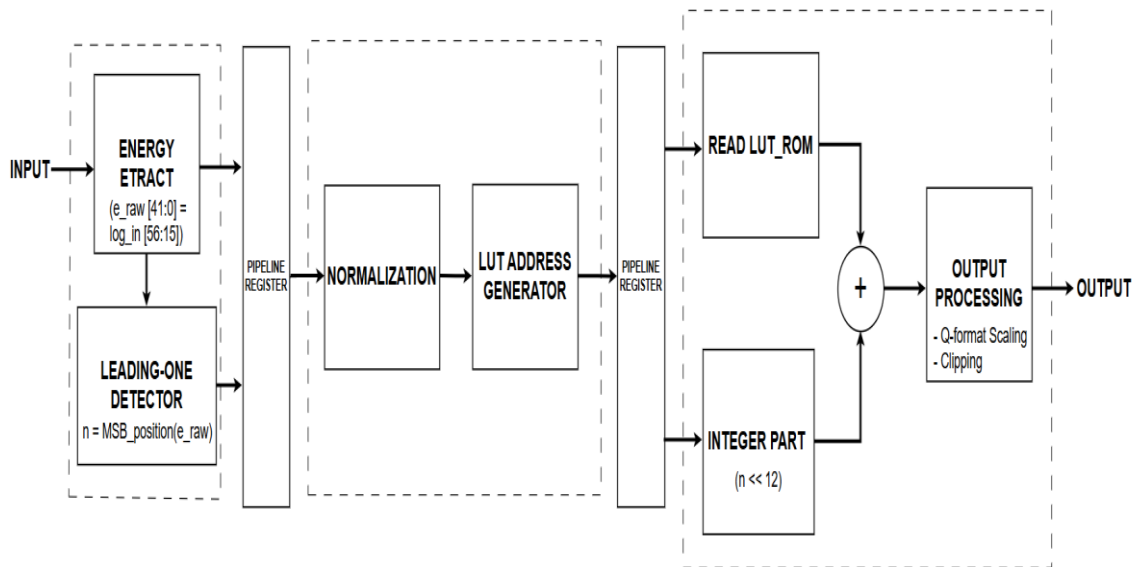
Trong đó:

- n là vị trí bit có trọng số lớn nhất (Most Significant Bit - MSB);
- f là phần thập phân đã được chuẩn hóa.

Khi đó:

$$\log_2(E) = n + \log_2(1 + f) \quad (3.4)$$

Trong thiết kế, phần nguyên (n) được xác định thông qua bộ dò bit có trọng số lớn nhất (MSB Detector), trong khi phần phân số $\log_2(1 + f)$ được tra cứu từ bộ nhớ LUT đã được tính toán trước. Toàn bộ quá trình được tổ chức theo kiến trúc pipeline gồm ba tầng xử lý liên tiếp nhằm bảo đảm khả năng xử lý dữ liệu dạng luồng (streaming).



Hình 3.11: Sơ đồ khối log_unit

❖ Giai đoạn 1: Xác định vị trí bit MSB

Đầu vào của khối Log Unit là giá trị năng lượng tích lũy từ khối MAC với độ rộng 57 bit. Trước tiên, dữ liệu được dịch phải một lượng bit cố định thông qua tham số $Q_SHIFT = 15$ nhằm đưa dữ liệu về đúng miền giá trị mong muốn cho quá trình tính logarit.

Để tránh trường hợp không xác định của hàm logarit khi đầu vào bằng 0, hệ thống thực hiện kiểm tra giá trị đầu vào và tự động thay thế giá trị 0 bằng 1. Cơ chế này bảo đảm pipeline hoạt động ổn định và tránh phát sinh lỗi tính toán.

Sau đó, bộ dò MSB tiến hành quét toàn bộ từ dữ liệu và xác định vị trí của bit có trọng số lớn nhất đang ở mức logic 1. Vị trí này tương ứng với phần nguyên của hàm logarit cơ số hai. Kết quả cùng tín hiệu điều khiển được lưu lại trong các thanh ghi của tầng thứ nhất.

❖ Giai đoạn 2: Chuẩn hóa dữ liệu và tạo địa chỉ LUT

Sau khi xác định được vị trí MSB, dữ liệu được chuẩn hóa bằng cách dịch trái một lượng tương ứng để đưa bit MSB lên vị trí cao nhất của từ dữ liệu. Các bit phía sau MSB được sử dụng để biểu diễn phần phân số của giá trị đầu vào.

Từ phần phân số này, hệ thống trích xuất 12 bit để tạo địa chỉ truy xuất bộ nhớ LUT. Trong thiết kế, LUT có kích thước 4096 phần tử, tương ứng với độ rộng địa chỉ 12 bit.

Mỗi phần tử trong LUT lưu giá trị: $\log_2(1 + f)$ ở định dạng số cố định với 12 bit phần thập phân. Các giá trị này được tạo trước bằng chương trình Python và lưu trong tệp ROM, sau đó được nạp vào phần cứng trong quá trình tổng hợp thiết kế. Thông tin địa chỉ LUT cùng vị trí MSB tiếp tục được chuyển sang tầng pipeline kế tiếp.

❖ Giai đoạn 3: Tra cứu LUT và hiệu chỉnh kết quả

Tại tầng cuối, giá trị phân số được đọc từ ROM LUT thông qua địa chỉ đã xác định ở giai đoạn trước. Kết quả tra cứu được cộng với phần nguyên do bộ dò MSB cung cấp để tạo thành giá trị logarit hoàn chỉnh.

Ban đầu kết quả được biểu diễn dưới dạng số cố định với 12 bit phần thập phân. Sau đó dữ liệu được dịch phải hai bit nhằm chuyển đổi về định dạng Q6.10 để đồng bộ với mô hình tham chiếu trên Python và mạng CNN sử dụng trong quá trình huấn luyện.

Để bù sai lệch phát sinh do cơ chế scale của khối FFT và các phép dịch bit trong pipeline phần cứng, hệ thống thực hiện thêm một bước hiệu chỉnh bằng cách trừ đi một hằng số thực nghiệm. Giá trị hiệu chỉnh này được xác định thông qua quá trình đối chiếu kết quả giữa mô hình Python và mô phỏng RTL nhằm bảo đảm dữ liệu Log-Mel Spectrogram sinh ra từ phần cứng có độ tương đồng cao với dữ liệu dùng trong quá trình huấn luyện.

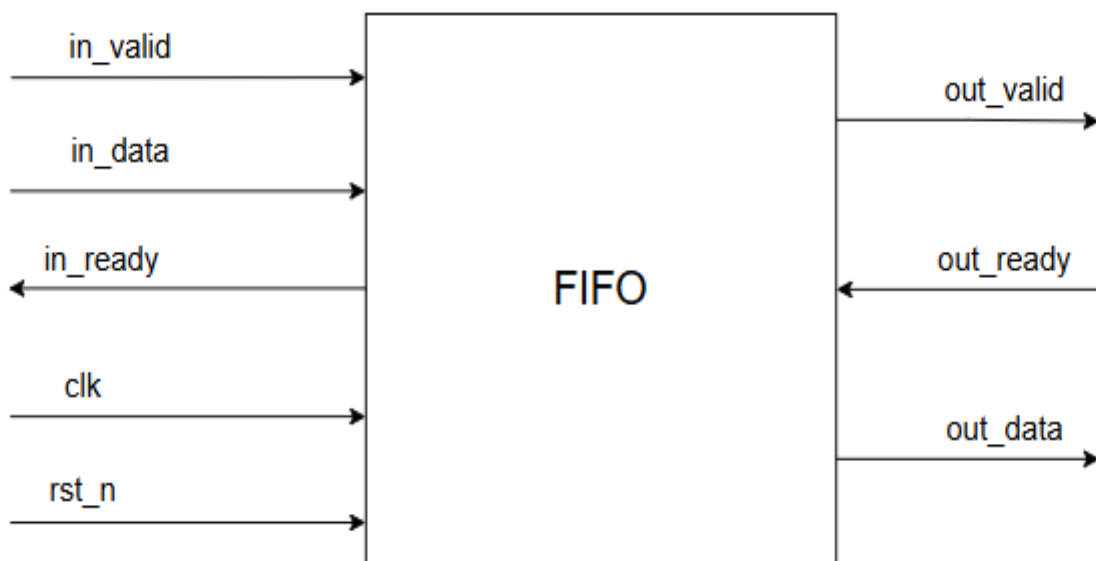
Sau bước hiệu chỉnh, dữ liệu được đưa qua bộ giới hạn ngưỡng (Clipper). Nếu giá trị logarit nhỏ hơn 0 thì đầu ra được gán bằng 0. Ngược lại, dữ liệu được giữ

nguyên và xuất ra ngoài. Cơ chế này giúp loại bỏ các thành phần năng lượng rất nhỏ và bảo đảm dữ liệu đầu ra luôn nằm trong miền không âm.

3.4. Bộ đệm FIFO

Đầu ra của khối Log-Mel Spectrogram được sinh ra theo dạng luồng dữ liệu (streaming), trong khi khối CNN yêu cầu dữ liệu được tổ chức thành các khung đặc trưng hoàn chỉnh trước khi thực hiện suy luận. Do đó, một bộ đệm FIFO được sử dụng để lưu trữ tạm thời dữ liệu giữa hai khối nhằm đảm bảo tính liên tục của luồng xử lý.

FIFO được thiết kế theo kiến trúc vòng tròn (Circular Buffer), bao gồm bộ nhớ lưu trữ dữ liệu, con trỏ ghi (write pointer), con trỏ đọc (read pointer) và bộ đếm số phần tử hiện có trong bộ đệm (count). Dữ liệu được ghi vào FIFO khi tín hiệu `in_valid` và `in_ready` đồng thời được kích hoạt. Tương tự, dữ liệu được đọc ra khỏi FIFO khi tín hiệu `out_valid` và `out_ready` ở mức logic cao.



Hình 3.11: Mô-đun FIFO

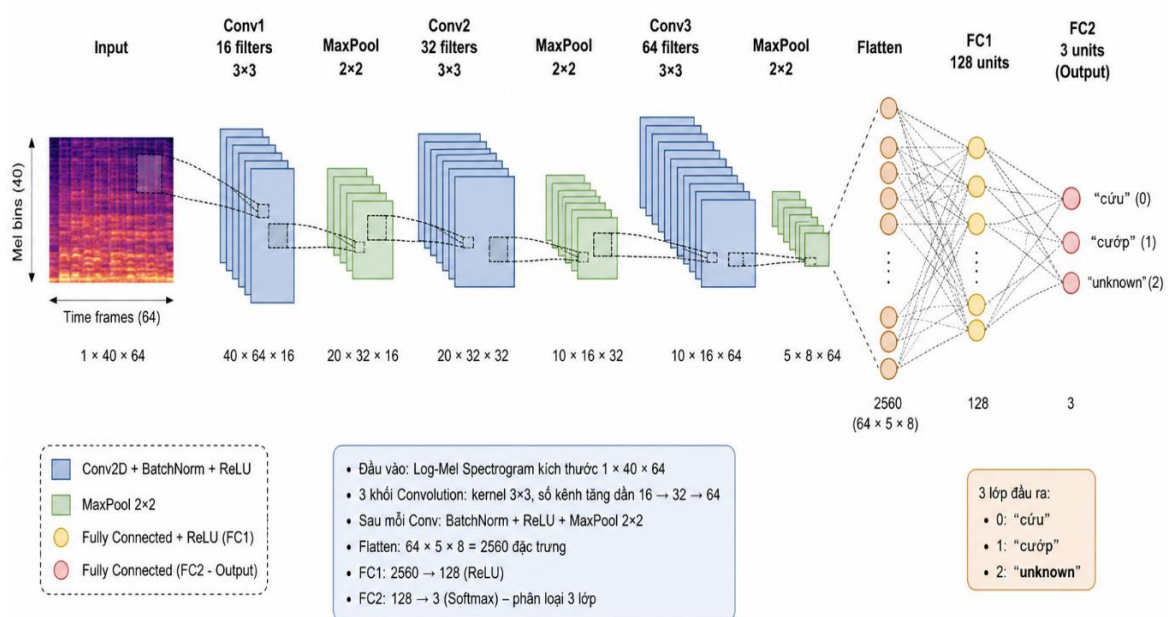
Khối FIFO sử dụng cơ chế bắt tay (handshake protocol) theo chuẩn valid-ready, cho phép tách biệt tốc độ xử lý giữa khối sinh dữ liệu và khối tiêu thụ dữ liệu. Khi bộ đệm đầy ($\text{count} = \text{DEPTH}$), tín hiệu `in_ready` được hạ xuống mức thấp

để ngăn ghi thêm dữ liệu. Ngược lại, khi FIFO rỗng ($\text{count} = 0$), tín hiệu out_valid được hạ xuống nhằm ngăn đọc dữ liệu không hợp lệ.

Trong đề tài này, FIFO có độ rộng dữ liệu 16 bit và độ sâu 32 phần tử. Cấu hình này cho phép lưu trữ tạm thời nhiều vector đặc trưng Log-Mel trước khi được đưa vào khối CNN, đồng thời hỗ trợ kiến trúc pipeline hoạt động ổn định khi xuất hiện chênh lệch độ trễ giữa các khối IP.

3.5. Thiết kế khối CNN

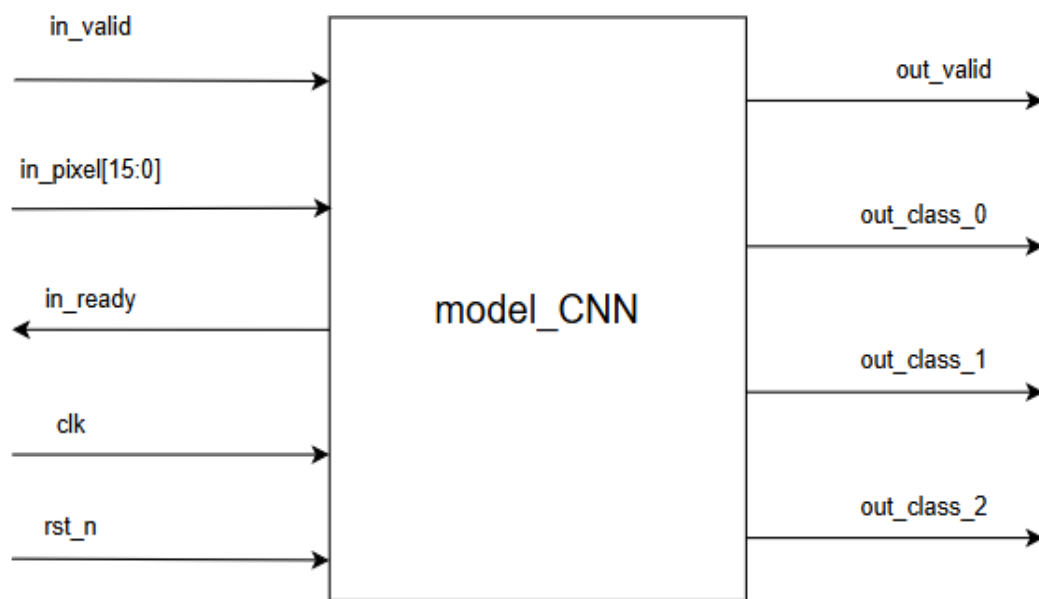
Khối CNN IP là phần quan trọng nhất của thiết kế, nó là nơi chứa các tầng của một mô hình học sâu, tùy thuộc vào mô hình sử dụng mà mỗi khối sẽ có cấu trúc khác nhau.



Hình 3.12: Kiến trúc tổng quan khối CNN

Đầu vào của mạng là ảnh đặc trưng Log-Mel Spectrogram có kích thước 40×64 , được tạo từ khối tiền xử lý tín hiệu âm thanh. Mô hình gồm hai phần chính: khối trích xuất đặc trưng (Feature Extraction) gồm các lớp Conv và Maxpool, khối phân loại (Classification) gồm các lớp Fully Connected. Trong đó, khối trích xuất đặc trưng bao gồm ba tầng tích chập liên tiếp kết hợp với các lớp chuẩn hóa Batch

Normalization, hàm kích hoạt ReLU và phép lấy mẫu cực đại Max Pooling. Các tầng tích chập có số lượng kênh đặc trưng lần lượt là 16, 32 và 64, với kích thước kernel 3×3 và cơ chế padding bằng 1 nhằm duy trì kích thước không gian của đặc trưng sau mỗi phép tích chập. Khối phân loại (Classification) có nhiệm vụ ánh xạ các đặc trưng đã được trích xuất sang các lớp từ khóa tương ứng. Khối này bao gồm một lớp Fully Connected 128 neuron kết hợp với hàm kích hoạt ReLU. Cuối cùng, một lớp Fully Connected đầu ra thực hiện phân loại và đưa ra xác suất dự đoán cho từng lớp từ khóa.



Hình 3.13: Top-module của khối CNN

Bảng 3.7: Các port của model CNN

Tên tín hiệu	I/O	Độ rộng	Mô tả
clk	Input	1 bit	Tín hiệu xung clock đồng bộ cho toàn bộ mô hình CNN.

rst_n	Input	1 bit	Tín hiệu reset mức thấp, dùng để khởi tạo trạng thái ban đầu của các thanh ghi và bộ đếm bên trong hệ thống.
in_valid	Input	1 bit	Báo hiệu dữ liệu đầu vào tại in_pixel là hợp lệ.
in_ready	Output	1 bit	Báo hiệu khối CNN sẵn sàng nhận dữ liệu đầu vào mới.
in_pixel	Input	16 bit	Dữ liệu đặc trưng Log-Mel Spectrogram đầu vào, định dạng số cố định 16 bit.
out_valid	Output	1 bit	Báo hiệu kết quả phân loại ở các ngõ ra đã hợp lệ.
out_class_0	Output	16 bit	Giá trị (điểm dự đoán) của lớp từ khóa thứ nhất (<i>cứu</i>).
out_class_1	Output	16 bit	Giá trị (điểm dự đoán) của lớp từ khóa thứ hai (<i>cướp</i>).
out_class_2	Output	16 bit	Giá trị (điểm dự đoán) của lớp từ khóa thứ ba (<i>unknown</i>).

3.5.1. Khối tích chập (convolution)

Khối tích chập chịu trách nhiệm điều phối quá trình tạo cửa sổ dữ liệu, đọc trọng số, thực hiện phép tích chập và xuất kết quả ra ngoài hệ thống. Kiến trúc được thiết kế theo mô hình xử lý luồng dữ liệu (streaming).

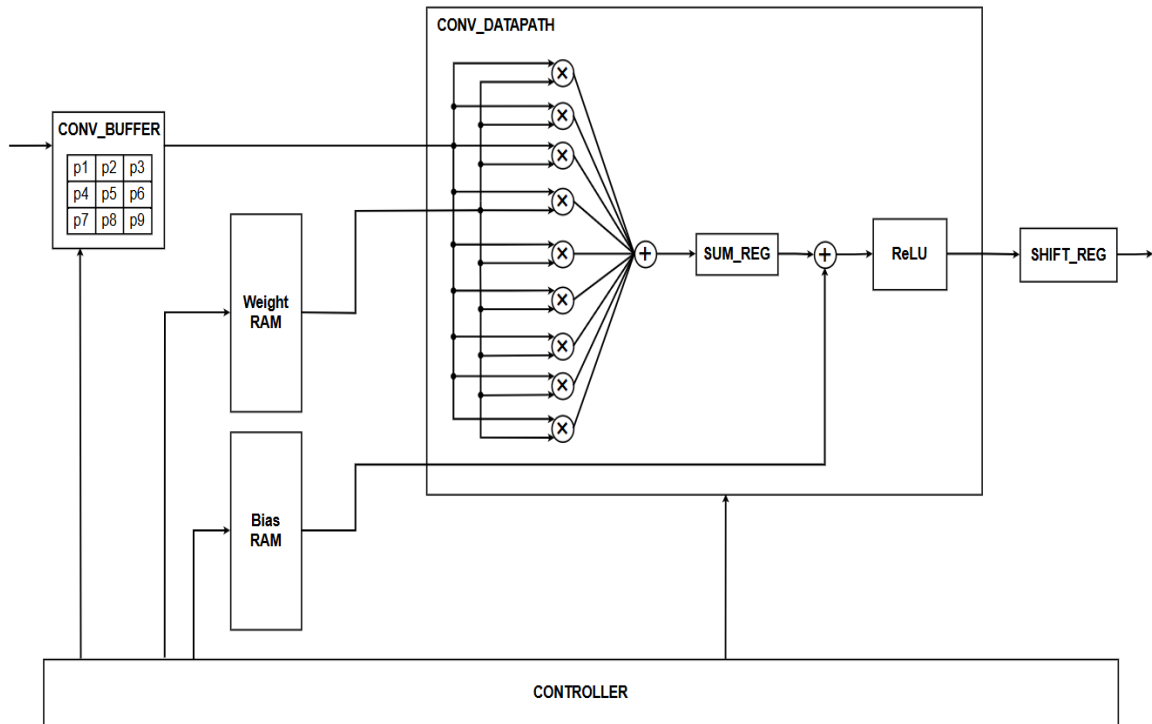
Hệ thống gồm ba thành phần chính:

- Khối tạo cửa sổ dữ liệu *conv_buffer*.
- Khối tính toán tích chập *conv_datapath*.
- Bộ điều khiển máy trạng thái hữu hạn (Finite State Machine – FSM).

Bảng 3.8: Các port của khối tích chập

Tên tín hiệu	I/O	Độ rộng	Mô tả
clk	Input	1 bit	Tín hiệu xung nhịp hệ thống.
rst_n	Input	1 bit	Tín hiệu reset tích cực mức thấp.
in_valid	Input	1 bit	Báo hiệu dữ liệu điểm ảnh đầu vào hợp lệ.
in_ready	Output	1 bit	Báo hiệu mô-đun sẵn sàng nhận dữ liệu đầu vào.
in_pixel	Input	DATA_WIDTH	Điểm ảnh đầu vào theo định dạng số dấu phẩy tĩnh.
out_ready	Input	1 bit	Tín hiệu xác nhận từ khối kế tiếp cho phép truyền dữ liệu đầu ra.
out_data	Output	DATA_WIDTH × OUT_CHANNELS	Vector chứa kết quả tích chập của toàn bộ các kênh đầu ra tại một vị trí cửa sổ.
out_valid	Output	1 bit	Báo hiệu dữ liệu đầu ra hợp lệ.

Các trọng số weight và hệ số bias của mạng CNN được lưu trữ trong bộ nhớ ROM nội bộ và được nạp từ các tệp HEX thông qua lệnh *\$readmemh*. Trong quá trình hoạt động, FSM lần lượt truy xuất từng bộ trọng số tương ứng với từng kênh đầu ra và đưa vào lõi tính toán.



Hình 3.14: Sơ đồ khối của khối tích chập

Ngoài ra, mô-đun còn tích hợp một thanh ghi đệm dữ liệu đầu ra (shift_reg) có nhiệm vụ lưu trữ tạm thời các kết quả tích chập được tạo ra từ từng bộ lọc. Do hệ thống sử dụng một lõi tính toán duy nhất để xử lý tuần tự các bộ lọc đầu ra, kết quả của mỗi lần tính toán sẽ được dịch vào thanh ghi này. Sau khi thu thập đủ 16 giá trị đầu ra tương ứng với 16 kênh đặc trưng (tương ứng 32 và 64 kênh cho các lớp sau), toàn bộ nội dung của thanh ghi sẽ được xuất ra cổng out_data dưới dạng một vector dữ liệu. Cơ chế này giúp đồng bộ dữ liệu đầu ra và đảm bảo rằng các đặc trưng thuộc cùng một vị trí không gian được truyền đi cùng lúc.

Thay vì sử dụng nhiều phần tử nhân – cộng (MAC) hoạt động song song cho tất cả các bộ lọc, hệ thống chỉ sử dụng một lõi tính toán duy nhất và thực hiện chia sẻ

tài nguyên theo thời gian (time-multiplexing). Với mỗi cửa sổ dữ liệu 3×3, lõi tích chập sẽ lần lượt tính toán cho các bộ lọc đầu ra. Các kết quả trung gian được lưu tuần tự trong thanh ghi đệm *shift_reg*, sau đó được đóng gói thành một từ dữ liệu rộng bit tương ứng với từng lớp trước khi truyền sang tầng xử lý tiếp theo. Điều này giúp giảm đáng kể tài nguyên phần cứng cần sử dụng so với phương án triển khai đồng thời nhiều lõi tích chập song song.

➤ **Thiết kế khối tạo cửa sổ dữ liệu *conv_buffer***

Khối *conv_buffer* có nhiệm vụ chuyển đổi luồng điểm ảnh đầu vào thành các cửa sổ dữ liệu 3×3 phục vụ cho phép tích chập.

Để giảm yêu cầu bộ nhớ, mô-đun sử dụng ba bộ đệm cột (column buffer) hoạt động theo cơ chế vòng tròn (circular buffering). Ba cột ảnh liên tiếp được lưu trữ trong mảng: *mem[0:2][*]*

Trong đó mỗi phần tử của mảng lưu trữ toàn bộ các điểm ảnh thuộc một cột của ảnh đầu vào. Con trỏ ghi *wr_ptr* được sử dụng để xác định cột đang được ghi, trong khi *mid_ptr* xác định cột trung tâm của cửa sổ hiện tại.

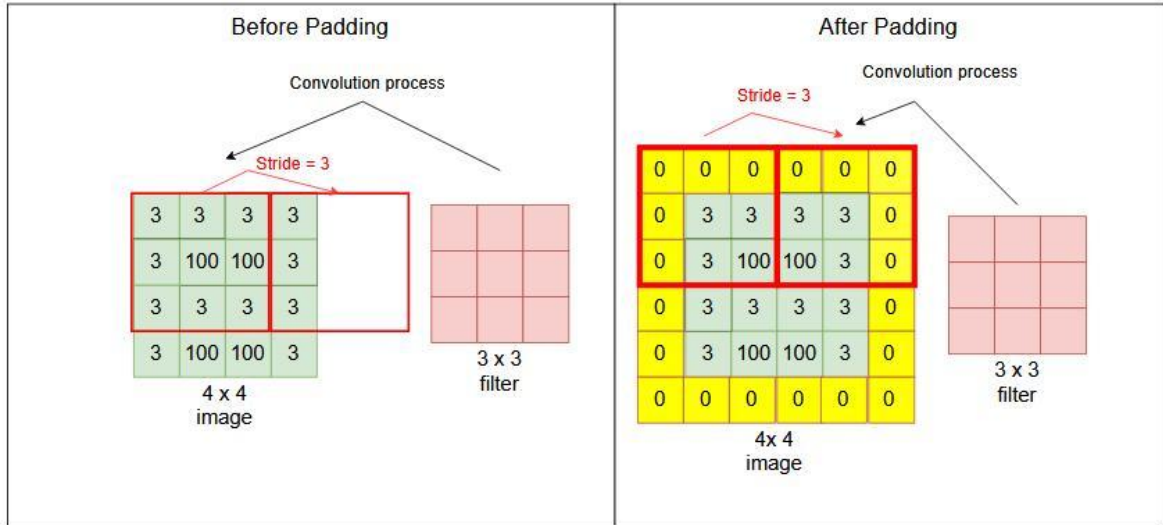
Khi tối thiểu ba cột dữ liệu đã được nạp vào bộ đệm, tín hiệu *active* được kích hoạt và hệ thống bắt đầu sinh các cửa sổ tích chập. Tại mỗi vị trí ((*out_r*, *out_c*)), khối *conv_buffer* đọc đồng thời 9 điểm ảnh tương ứng để tạo thành ma trận:

$$\begin{bmatrix} p0 & p1 & p2 \\ p3 & p4 & p5 \\ p6 & p7 & p8 \end{bmatrix}$$

Để duy trì kích thước đặc trưng đầu ra, hệ thống thực hiện kỹ thuật đệm biên bằng số không (zero-padding). Các điểm ảnh nằm ngoài phạm vi ảnh gốc được thay thế bằng giá trị 0 thông qua logic điều kiện khi sinh cửa sổ dữ liệu.

Ngoài ra, khối còn tích hợp cơ chế điều khiển luồng dữ liệu thông qua tín hiệu *buffer_in_ready*. Khoảng cách giữa cột đang ghi (*wr_col*) và cột đang đọc (*out_c*) được giám sát liên tục nhằm tránh trường hợp dữ liệu bị ghi đè trước khi quá trình tích chập hoàn tất.

Sau khi một cửa sổ dữ liệu được xử lý xong, tín hiệu next_window_en từ FSM sẽ yêu cầu bộ đệm dịch sang vị trí kế tiếp và sinh cửa sổ mới.



Hình 3.15: Ảnh minh họa về padding = 0

➤ Thiết kế khối tính toán tích chập conv_datapath

Khối conv_datapath thực hiện toàn bộ phép tính tích chập, cộng bias, kích hoạt ReLU và lượng tử hóa kết quả đầu ra.

Đối với mỗi cửa sổ 3×3, đầu ra được xác định theo công thức:

$$y = ReLU \left(\sum_{i=0}^8 (p_i \times w_i) + b \right) \quad (3.5)$$

Trong đó:

- (p_i) là các điểm ảnh trong cửa sổ dữ liệu.
- (w_i) là các trọng số weight của bộ lọc.
- (b) là hệ số bias đã được hợp nhất (fused bias).

Giai đoạn đầu tiên của khối tính toán bao gồm 9 bộ nhân hoạt động song song. Kết quả của từng phép nhân được lưu vào các thanh ghi mult_reg[0..8]. Đồng thời giá trị bias cũng được chốt vào thanh ghi bias_reg nhằm đồng bộ thời gian với dữ liệu sau phép nhân.

Sau khi các kết quả nhân được lưu trữ, mạch cộng tổ hợp thực hiện phép cộng toàn bộ các tích và cộng thêm bias:

$$sum = \sum_{i=0}^8 p_i w_i + b \quad (3.6)$$

Kết quả tiếp tục được đưa qua hàm kích hoạt ReLU nhằm loại bỏ các giá trị âm:

$$ReLU(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases} \quad (3.7)$$

Do hệ thống sử dụng biểu diễn số dấu phẩy tĩnh (fixed-point), số lượng bit phân thập phân sau phép nhân sẽ tăng lên bằng:

$$TOTAL_FRAC = PIXEL_FRAC + WEIGHT_FRAC \quad (3.8)$$

Trước khi xuất kết quả, hệ thống thực hiện làm tròn bằng cách cộng thêm hằng số:

$$ROUND_CONST = 2^{(SHIFT_RIGHT-1)} \quad (3.9)$$

Sau đó dữ liệu được dịch phải để đưa về định dạng đầu ra mong muốn. Để bảo vệ khỏi hiện tượng tràn số, mạch kiểm tra các bit có trọng số cao. Nếu phát hiện giá trị vượt quá phạm vi biểu diễn của dữ liệu 16 bit, đầu ra sẽ được bão hòa (saturation) tại giá trị cực đại:

$$16'H7FFF$$

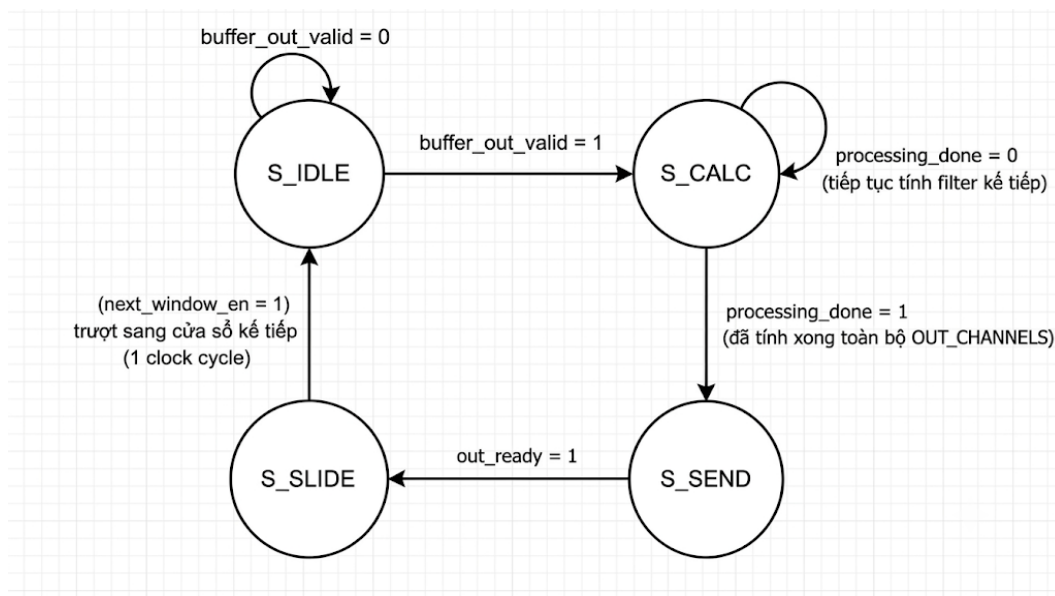
Ngược lại, dữ liệu sau lượng tử hóa sẽ được xuất trực tiếp ra cổng out_pixel.

➤ Bộ điều khiển FSM (Controller)

Quá trình hoạt động của hệ thống được điều khiển bởi máy trạng thái hữu hạn gồm bốn trạng thái.

Bảng 3.9: Bảng các trạng thái của FSM

Trạng thái	Điều kiện chuyển trạng thái	Chức năng
S_IDLE	buffer_out_valid = 1	Trạng thái khởi tạo hoặc sau khi hoàn thành xử lý một cửa sổ dữ liệu. Bộ điều khiển chờ buffer_out_valid từ conv_buffer. Trong trạng thái này, các thanh ghi điều khiển như filter_idx, shift_reg, processing_done được khởi tạo lại.
S_CALC	dp_partial_valid = 1 cho từng bộ lọc; chuyển sang S_SEND khi processing_done = 1	Bộ điều khiển lần lượt tạo yêu cầu xử lý cho từng bộ lọc thông qua request_en, cập nhật filter_idx để truy xuất weight/bias tương ứng, và ghi kết quả hợp lệ vào đúng vị trí trong shift_reg. Khi hoàn tất bộ lọc cuối cùng, processing_done được set để sang S_SEND.
S_SEND	out_ready = 1	Phát out_valid và xuất toàn bộ vector dữ liệu trong shift_reg ra cổng out_data. FSM giữ nguyên trạng thái này cho đến khi khối kế tiếp sẵn sàng nhận dữ liệu.
S_SLIDE	Không có điều kiện chờ; sau 1 chu kỳ chuyển thẳng về S_IDLE	Phát tín hiệu next_window_en để yêu cầu conv_buffer dịch sang cửa sổ dữ liệu kế tiếp. Trạng thái này chỉ kéo dài một chu kỳ clock.



Hình 3.16: Lưu đồ FSM của khối tích chập

3.5.2. Khối Maxpool

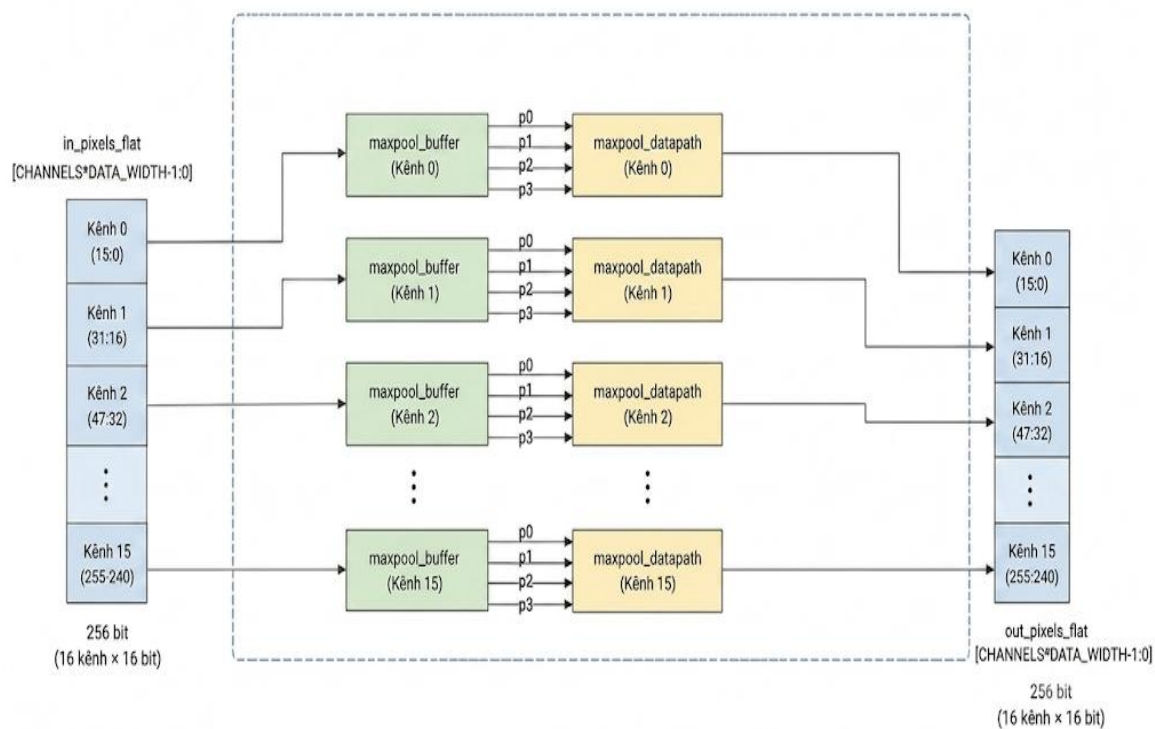
Khối maxpool_top là mô-đun gộp cực đại trong hệ thống xử lý ảnh. Đầu vào của khối này là kết quả đầu ra của tầng tích chập conv_top, được tổ chức dưới dạng vector song song gồm các kênh đặc trưng tương ứng, mỗi kênh có độ rộng 16 bit. Khối maxpool_top thực hiện quá trình gộp cực đại độc lập trên từng kênh đầu ra, sau đó ghép các kết quả lại thành một vector dữ liệu duy nhất để xuất sang tầng xử lý tiếp theo.

Bảng 3.10: Các port của khối Maxpool

Tên tín hiệu	I/O	Độ rộng	Mô tả
clk	Input	1 bit	Tín hiệu xung nhịp hệ thống.
rst_n	Input	1 bit	Tín hiệu reset tích cực mức thấp.

in_valid	Input	1 bit	Báo hiệu dữ liệu đầu vào hợp lệ từ khối tích chập conv_top.
in_pixels_flat	Input	CHANNELS × DATA_WIDTH	Vector chứa đồng thời toàn bộ các giá trị đặc trưng đầu ra từ khối tích chập.
out_pixels_flat	Output	CHANNELS × DATA_WIDTH	Vector chứa kết quả sau phép Max Pooling của tất cả các kênh đặc trưng.
out_valid	Output	1 bit	Báo hiệu dữ liệu đầu ra hợp lệ và sẵn sàng chuyển sang tầng xử lý tiếp theo.

Hình 3.17: Sơ đồ khối của maxpool lớp thứ nhất



Kiến trúc của khối maxpool_top được thiết kế theo mô hình song song theo kênh (channel-parallel). Cụ thể, hệ thống tạo ra 16 nhánh xử lý giống hệt nhau ở lớp đầu (tương ứng các lớp sau là 32 và 64), trong đó mỗi nhánh bao gồm một khối maxpool_buffer và một khối maxpool_datapath. Cách tổ chức này cho phép tất cả các kênh đặc trưng được xử lý đồng thời mà không cần cơ chế chia sẻ tài nguyên theo thời gian như ở khối tích chập.

➤ Thiết kế khối đệm dữ liệu maxpool_buffer

Khối maxpool_buffer có nhiệm vụ gom các điểm ảnh đầu vào thành cửa sổ dữ liệu 2×2 , phục vụ cho phép gộp cực đại. Trong kiến trúc này, dữ liệu đầu vào của mỗi kênh được xử lý theo chuỗi liên tiếp các điểm ảnh, và bộ đệm chỉ cần lưu trữ một cột dữ liệu trung gian trong mảng:

$$col_ram[0:IMG_H - 1]$$

Trong đó col_ram được dùng để lưu lại các điểm ảnh thuộc cột trước đó, còn các thanh ghi tạm top_left và top_right được sử dụng để giữ hai giá trị của hàng trên trong cửa sổ 2×2 .

Khối này sử dụng biến đếm hàng in_r để xác định vị trí điểm ảnh đang được nạp vào bộ nhớ. Ngoài ra, tín hiệu col_phase được dùng để phân biệt giữa hai pha cột: cột chẵn và cột lẻ. Khi col_phase = 0, dữ liệu đầu vào chỉ được ghi vào bộ đệm cột. Khi col_phase = 1, bộ đệm bắt đầu ghép cửa sổ dữ liệu từ cột trước và cột hiện tại để tạo thành một khối 2×2 . Cơ chế hoạt động của khối có thể được mô tả như sau:

- Ở cột chẵn, các điểm ảnh được ghi vào col_ram.
- Ở cột lẻ, với mỗi cặp hàng liên tiếp, khối sẽ:
 - ✓ lưu điểm ảnh của hàng chẵn vào top_left và top_right,
 - ✓ sau đó tại hàng lẻ, ghép bốn điểm ảnh thành cửa sổ 2×2 gồm:

$$\begin{pmatrix} p0 & p1 \\ p2 & p3 \end{pmatrix}$$

Khi đủ bốn điểm ảnh để tạo thành một cửa sổ hợp lệ, tín hiệu buffer_out_valid được kích hoạt để báo hiệu dữ liệu đầu vào cho khối tính toán đã sẵn sàng.

So với khối conv_buffer, kiến trúc của maxpool_buffer đơn giản hơn do không cần tạo cửa sổ 3×3 hay xử lý padding biên phức tạp. Khối này chủ yếu thực hiện chức năng gom dữ liệu theo khối 2×2 để phục vụ phép max pooling với bước trượt stride 2.

➤ **Thiết kế khối tính toán maxpool_datapath**

Khối maxpool_datapath thực hiện phép gộp cực đại trên cửa sổ 2×2. Với bốn giá trị đầu vào là (p_0, p_1, p_2, p_3), đầu ra được xác định theo công thức:

$$y = \max(p_0, p_1, p_2, p_3) \quad (3.10)$$

Để giảm độ dài đường truyền tới hạn (critical path), quá trình so sánh được chia thành hai bước độc lập trong mạch tổ hợp:

$$m_1 = \max(p_0, p_1)$$

$$m_2 = \max(p_2, p_3)$$

Sau đó kết quả cuối cùng được xác định bằng:

$$y = \max(m_1, m_2)$$

Kiến trúc này cho phép so sánh song song từng cặp phần tử trước khi thực hiện phép so sánh cuối cùng, giúp mạch hoạt động ổn định hơn ở tần số cao.

Kết quả cực đại sau đó được chột vào thanh ghi đầu ra max_out tại cạnh lên của xung nhịp. Tín hiệu valid_out được đồng bộ với đầu ra và có độ trễ một chu kỳ so với valid_in. Như vậy, khối maxpool_datapath có thể xem như một cấu trúc gồm một tầng tổ hợp và một tầng thanh ghi đầu ra.

3.5.3. Khối Fully Connected (FC1)

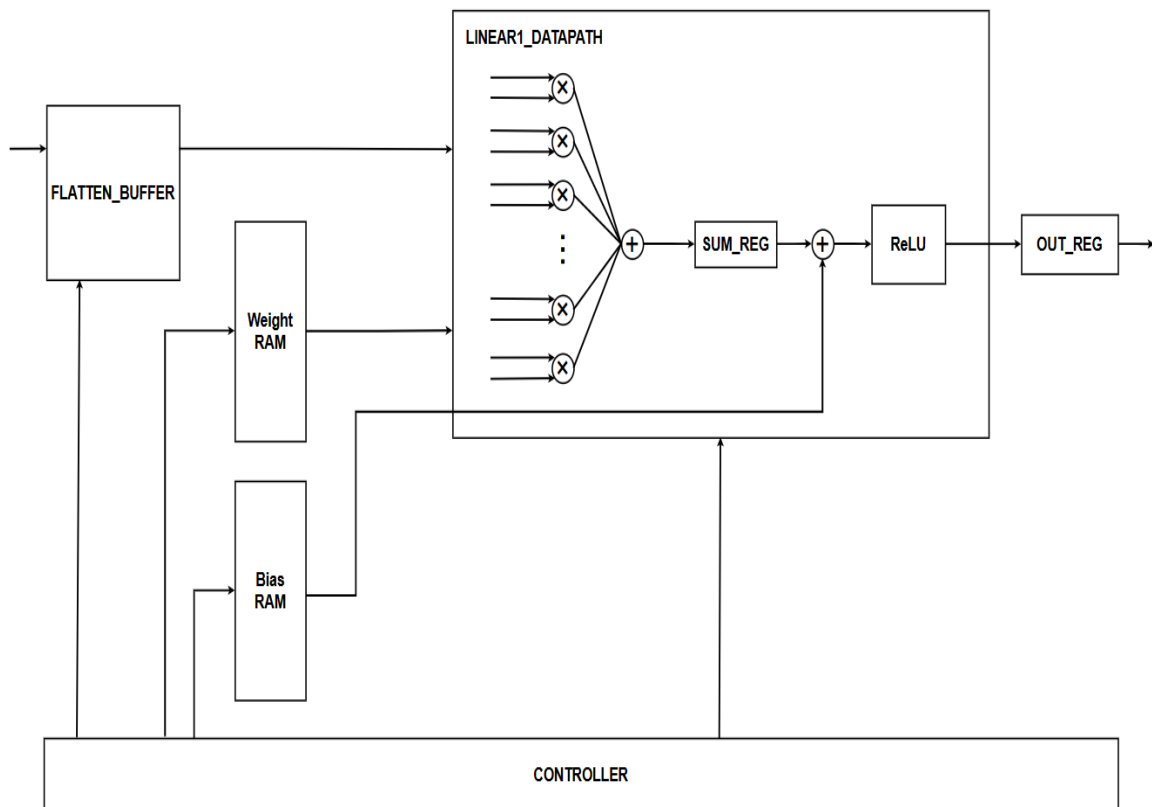
Khối FC1 là tầng Fully Connected thứ nhất của bộ phân loại, có nhiệm vụ nhận frame đặc trưng đầu vào từ tầng trước, thực hiện phép nhân – cộng tích lũy (Multiply-Accumulate – MAC) giữa dữ liệu đầu vào và các trọng số tương ứng của từng neuron, sau đó cộng thêm bias để tạo ra vector đặc trưng trung gian gồm 128 phần tử. Kết quả của tầng này được truyền tới tầng Fully Connected tiếp theo (FC2).

Bảng 3.11: Các port của FC1

Tên tín hiệu	I/O	Độ rộng	Mô tả
clk	Input	1 bit	Tín hiệu xung clock điều khiển hoạt động đồng bộ của toàn bộ module.
rst_n	Input	1 bit	Tín hiệu reset tích cực mức thấp, dùng để khởi tạo các thanh ghi và FSM.
in_valid	Input	1 bit	Báo hiệu dữ liệu đầu vào hợp lệ từ lớp trước.
in_ready	Output	1 bit	Báo hiệu module sẵn sàng nhận dữ liệu đầu vào mới.
in_data_flat	Input	1024 bit (64 × 16)	Bus dữ liệu đầu vào chứa 64 đặc trưng, mỗi đặc trưng 16 bit, được truyền trong từng chu kỳ xử lý.
out_ready	Input	1 bit	Tín hiệu xác nhận từ khối kế tiếp rằng dữ liệu đầu ra đã sẵn sàng được nhận.
out_valid	Output	1 bit	Báo hiệu dữ liệu đầu ra của toàn bộ lớp Fully Connected đã hợp lệ.
out_layer_data	Output	2048 bit (128 × 16)	Bus chứa toàn bộ 128 neuron đầu ra của lớp Fully Connected.

Khối này được thiết kế theo hướng xử lý tuần tự nhằm giảm chi phí phần cứng. Dữ liệu đầu vào gồm 40 chu kỳ, mỗi chu kỳ chứa 64 đặc trưng có độ rộng 16 bit.

Toàn bộ frame đầu vào được lưu trong bộ đệm flatten_buffer. Sau khi nhận đủ dữ liệu của một frame, bộ đệm sẽ phát lại (replay) dữ liệu nhiều lần để lần lượt tính toán cho từng neuron đầu ra. Với mỗi neuron, khối linear1_datapath thực hiện các phép nhân và cộng tích lũy với bộ trọng số tương ứng được lưu trong bộ nhớ ROM, sau đó cộng thêm giá trị bias để tạo ra kết quả của neuron đó. Sau khi hoàn tất tính toán cho 128 neuron, toàn bộ vector đầu ra được ghép lại và xuất sang tầng phân loại tiếp theo.



Hình 3.18: Sơ đồ khối của FC1

Cách tổ chức này cho phép tái sử dụng cùng một frame dữ liệu đầu vào cho nhiều neuron đầu ra, nhờ đó giảm số lượng phần tử nhân cộng cần sử dụng và tiết kiệm tài nguyên phần cứng.

➤ Khối flatten_buffer

Khối flatten_buffer đóng vai trò bộ nhớ đệm trung gian giữa tầng đặc trưng và lớp Fully Connected. Module lưu trữ toàn bộ một frame đầu vào gồm 40 vector đặc trưng, mỗi vector có độ rộng 1024 bit. Sau khi bộ nhớ được lấp đầy, tín hiệu

frame_ready được kích hoạt để thông báo cho bộ điều khiển bắt đầu quá trình phân loại.

Khác với FIFO thông thường, flatten_buffer cho phép phát lại cùng một frame nhiều lần. Cơ chế này được sử dụng để tái sử dụng tập đặc trưng đầu vào cho toàn bộ 128 neuron của lớp Fully Connected mà không cần truyền lại dữ liệu từ tầng trước.

Bộ điều khiển FSM sử dụng ba tín hiệu điều khiển gồm restart_replay, next_cycle_en và clear_frame. Trong đó:

- restart_replay đưa con trỏ đọc về cycle 0 để bắt đầu tính toán cho một neuron mới.
- next_cycle_en cho phép chuyển sang vector kế tiếp trong frame.
- clear_frame xóa toàn bộ trạng thái bộ đệm sau khi hoàn thành quá trình xử lý.

Trong quá trình replay, module sinh ra tín hiệu last_cycle khi đang phát vector cuối cùng của frame. Tín hiệu này được sử dụng bởi FSM để xác định thời điểm kết thúc xử lý dữ liệu cho một neuron và chuyển sang giai đoạn nhận kết quả từ datapath.

➤ Khối linear_datapath

Đây là khối thực hiện tính toán chính của tầng classifier thứ nhất. Datapath được chia thành 3 giai đoạn:

Giai đoạn 1: Nhân từng phần tử

Ở mỗi chu kỳ, datapath nhận:

- một vector đầu vào 64 phần tử,
- một vector trọng số 64 phần tử tương ứng,
- bias của nút hiện tại.

Mỗi phần tử đầu vào được nhân với phần tử trọng số tương ứng. Kết quả 64 phép nhân được lưu vào mult_reg[].

Giai đoạn 2: Cộng dồn 64 tích và tích lũy qua 40 chu kỳ

Các tích trung gian sau đó được cộng lại để tạo tổng của một chu kỳ. Tổng này tiếp tục được cộng dồn qua 40 chu kỳ đầu vào để tạo ra giá trị tuyến tính hoàn chỉnh cho một nút đầu ra.

Về mặt toán học, khối đang thực hiện:

$$accum = \sum_{k=0}^{39} \left(\sum_{c=0}^{63} x_{k,c} \cdot w_{j,k,c} \right) \quad (3.11)$$

Trong đó mỗi k là một chu kỳ đầu vào của frame. Ở chu kỳ đầu tiên ($cycle_idx = 0$), accumulator được nạp trực tiếp bằng tổng hiện tại. Từ các chu kỳ sau, tổng mới được cộng nối tiếp vào $accum_reg$.

Giai đoạn 3: Cộng bias, ReLU, làm tròn và bão hòa

Sau khi cộng dồn đủ 40 chu kỳ, datapath thực hiện các bước hậu xử lý:

- Cộng bias,
- Căn chỉnh số cố định theo Q-format,
- Áp dụng hàm kích hoạt ReLU,
- Làm tròn,
- Bão hòa nếu giá trị vượt quá miền biểu diễn 16 bit.

Cụ thể:

$$final_sum = accum + bias \quad (3.12)$$

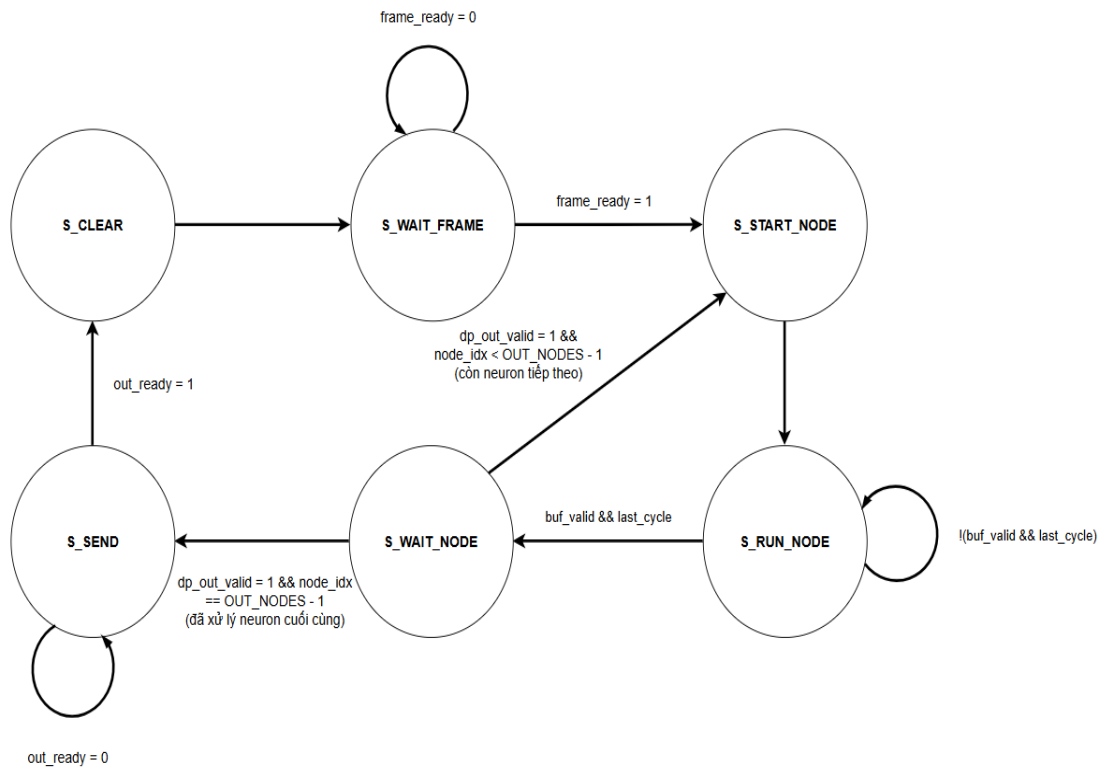
$$relu_{out} = \max(0, final_sum)$$

Sau đó giá trị được dịch bit để đưa về độ rộng dữ liệu đầu ra 16 bit. Nếu kết quả vượt miền biểu diễn của DATA_WIDTH, đầu ra sẽ được giới hạn ở giá trị cực đại:

$$16'h7FFF$$

Nhờ đó đầu ra ổn định và phù hợp với kiến trúc số cố định trên phần cứng.

➤ Bộ điều khiển trạng thái FSM



Hình 3.19: Lưu đồ FSM của FC1

Bảng 3.12: Các trạng thái của FSM

Trạng thái	Chức năng	Điều kiện chuyển trạng thái	Tín hiệu điều khiển chính
S_WAIT_FRAME	Chờ flatten_buffer nhận đủ 40 vector đặc trưng đầu vào và hoàn thành một frame.	frame_ready = 1 → S_START_NODE	Không kích hoạt tín hiệu điều khiển nào.
S_START_NODE	Khởi tạo quá trình tính toán cho một neuron mới. Yêu	Chuyển ngay sang S_RUN_NODE ở chu kỳ kế tiếp.	restart_replay = 1

	cầu buffer phát lại dữ liệu từ cycle 0.		
S_RUN_NODE	Thực hiện tính toán cho neuron hiện tại. Buffer lần lượt phát 40 vector đặc trưng, datapath thực hiện phép nhân và cộng dồn.	Khi buf_valid = 1 và last_cycle = 1 → S_WAIT_NODE	next_cycle_en = buf_valid
S_WAIT_NODE	Chờ datapath hoàn thành toàn bộ phép tính của neuron hiện tại và xuất kết quả.	Nếu dp_out_valid = 1: - node_idx < OUT_NODES - 1 → S_START_NODE - node_idx = OUT_NODES - 1 → S_SEND	Không phát tín hiệu điều khiển mới.
S_SEND	Xuất toàn bộ vector kết quả gồm 128 neuron ra cổng out_layer_data.	out_ready = 1 → S_CLEAR	out_valid = 1
S_CLEAR	Xóa dữ liệu frame cũ trong buffer, chuẩn bị nhận frame mới.	Chuyển ngay về S_WAIT_FRAME.	clear_frame = 1

3.5.4. Khối Fully Connected (FC2)

Khối FC2 được thiết kế là lớp phân loại cuối cùng của hệ thống nhận dạng, có nhiệm vụ nhận vector đặc trưng đầu ra từ khối FC1, thực hiện phép nhân – cộng tích

lũy giữa vector đặc trưng với các trọng số đã được huấn luyện trước, sau đó tạo ra ba giá trị đầu ra tương ứng với ba lớp cần phân loại của hệ thống. Các giá trị đầu ra này đại diện cho độ tin cậy của từng lớp và được sử dụng để đưa ra quyết định phân loại cuối cùng.

Đầu vào của khối là một vector gồm 128 phần tử dữ liệu ở định dạng fixed-point 16 bit. Đầu ra của khối bao gồm ba giá trị out_class_0, out_class_1 và out_class_2, tương ứng với ba lớp cần nhận dạng. Tín hiệu out_valid được sử dụng để báo hiệu kết quả đầu ra đã hợp lệ.

Để đạt được sự cân bằng giữa tốc độ xử lý và tài nguyên phần cứng, kiến trúc của khối được xây dựng theo hướng xử lý song song kết hợp với cơ chế pipeline nhiều tầng.

Bảng 3.13: Các port của FC2

Tên tín hiệu	I/O	Độ rộng	Mô tả
clk	Input	1 bit	Tín hiệu xung clock đồng bộ cho toàn bộ module.
rst_n	Input	1 bit	Tín hiệu reset tích cực mức thấp, dùng để khởi tạo các thanh ghi nội bộ.
in_valid	Input	1 bit	Báo hiệu dữ liệu đầu vào từ Layer 1 hợp lệ.
in_ready	Output	1 bit	Báo hiệu module sẵn sàng nhận vector dữ liệu mới (chỉ lên mức cao khi FSM đang rảnh và chưa có dữ liệu nào đang chờ xử lý); kết hợp với in_valid tạo thành cơ chế bắt tay valid/ready.
in_layer1_data	Input	2048 bit (128 × 16)	Bus chứa toàn bộ vector đặc trưng đầu ra của Layer 1 gồm 128 phần tử, mỗi

			phần tử 16 bit định dạng fixed-point Q8.8.
out_valid	Output	1 bit	Báo hiệu dữ liệu đầu ra phân loại hợp lệ.
out_class_0	Output	16 bit	Giá trị logit của lớp phân loại thứ nhất, định dạng signed fixed-point Q8.8.
out_class_1	Output	16 bit	Giá trị logit của lớp phân loại thứ hai, định dạng signed fixed-point Q8.8.
out_class_2	Output	16 bit	Giá trị logit của lớp phân loại thứ ba, định dạng signed fixed-point Q8.8.

Khối FC2 bao gồm các thành phần chính tương tự khối FC1, thay vào khối flatten_buffer sẽ là một buffer có chức năng khác. Do vector đầu vào gồm 128 phần tử nhưng số lượng bộ nhân được sử dụng là 64, quá trình tính toán được chia thành hai lần xử lý, mỗi lần thực hiện trên một nhóm gồm 64 phần tử. Khối buffer thay thế cho flatten_buffer là một thiết kế để chọn dữ liệu từng nhóm 64 phần tử làm đầu vào cho Datapath xử lý. Với ba lớp đầu ra, tổng cộng cần thực hiện sáu lượt tính toán để hoàn thành quá trình phân loại. Khối Datapath thực hiện xử lý các chứng năng tính toán sau:

Stage 1: Nhân song song từng nhóm 64 phần tử cho 3 lớp

Khối MAC Stage 1 gồm 64 bộ nhân hoạt động song song. Ở mỗi chu kỳ xử lý, 64 phần tử dữ liệu đầu vào được nhân với 64 hệ số trọng số tương ứng để tạo ra các tích riêng lẻ. Phép nhân thứ (i) được biểu diễn như sau:

$$P_i = x_i \times w_i, \quad i = 0, 1, \dots, 63 \quad (3.13)$$

Trong đó:

- (x_i): phần tử dữ liệu đầu vào thứ (i);
- (w_i): hệ số trọng số tương ứng;
- (P_i): kết quả của phép nhân giữa dữ liệu đầu vào và trọng số.

Các tích thu được được lưu vào mảng thanh ghi mult_reg để phục vụ cho giai đoạn cộng tích lũy ở tầng kế tiếp.

Việc thực hiện đồng thời 64 phép nhân giúp nâng cao đáng kể thông lượng xử lý của hệ thống, đồng thời tận dụng hiệu quả các khối DSP tích hợp trên FPGA.

Stage 2: Cộng tích lũy các giá trị nhân

Sau khi các tích được tạo ra ở Stage 1, khối MAC Stage 2 thực hiện cộng toàn bộ 64 giá trị để tạo thành tổng của một batch. Giá trị này được xác định theo biểu thức:

$$tree_sum = \sum_{i=0}^{63} P_i \quad (3.14)$$

Nếu batch hiện tại là batch đầu tiên, giá trị tree_sum sẽ được ghi trực tiếp vào thanh ghi tích lũy của lớp tương ứng. Nếu là batch thứ hai, giá trị này sẽ được cộng với kết quả đã lưu trước đó.

Do đó, sau hai lần xử lý liên tiếp, thanh ghi tích lũy của lớp thứ (j) sẽ chứa giá trị tích vô hướng giữa vector đặc trưng đầu vào và vector trọng số tương ứng:

$$ACC_j = \sum_0^{127} x_k w_{j,k} \quad (3.15)$$

Trong đó:

- (x_k) là phần tử đầu vào thứ (k);
- (w_{j,k}) là trọng số thứ (k) của lớp thứ (j);
- (ACC_j) là giá trị tích lũy cuối cùng của lớp thứ (j).

Các thanh ghi tích lũy được thiết kế với độ rộng 40 bit nhằm tránh hiện tượng tràn số khi cộng nhiều giá trị tích 32 bit.

Stage 3: Cộng bias, làm tròn và bão hòa

Sau khi hoàn thành phép nhân – cộng tích lũy, kết quả tiếp tục được đưa qua khối hậu xử lý nhằm tạo ra giá trị đầu ra cuối cùng.

Trước tiên, giá trị bias của từng lớp được cộng vào kết quả tích lũy. Do dữ liệu được biểu diễn ở dạng fixed-point Q8, giá trị bias được dịch trái thêm ($Q_{\{FRAC\}}$)

bit để đưa về cùng miền biểu diễn với accumulator. Giá trị sau khi cộng bias được xác định bởi:

$$S_j = ACC_j + (Bias_j \ll Q_{FRAC}) \quad (3.16)$$

Trong đó:

- (ACC_j): giá trị tích lũy của lớp thứ (j);
- (Bias_j): hệ số bias của lớp thứ (j);
- (Q_{FRAC}=8).

Tiếp theo, để giảm sai số lượng tử khi chuyển đổi về dữ liệu 16 bit, khối thực hiện phép làm tròn bằng cách cộng thêm một nửa giá trị của bit phần lẻ thấp nhất:

$$R_j = S_j + 2^{Q_{FRAC}-1} \quad (3.17)$$

Với (Q_{FRAC}=8), giá trị cộng thêm là:

$$2^{Q_{FRAC}-1} = 2^{8-1} = 127 \quad (3.18)$$

Sau khi làm tròn, kết quả được dịch phải (Q_{FRAC}) bit để đưa về miền số nguyên 16 bit. Cuối cùng, cơ chế bão hòa có dấu (signed saturation) được áp dụng nhằm giới hạn giá trị đầu ra trong miền biểu diễn của số nguyên có dấu 16 bit:

$$-2^{15} \leq y_j \leq 2^{15} - 1 \quad (3.19)$$

Hay tương đương:

$$-32768 \leq y_j \leq 32767 \quad (3.20)$$

Nếu giá trị vượt quá giới hạn trên hoặc dưới, đầu ra được gán bằng các giá trị biên -32768 hoặc 32767 tương ứng. Nhờ đó, hiện tượng tràn số được loại bỏ và kết quả đầu ra luôn nằm trong miền giá trị hợp lệ, đảm bảo độ ổn định của hệ thống khi triển khai trên phần cứng.

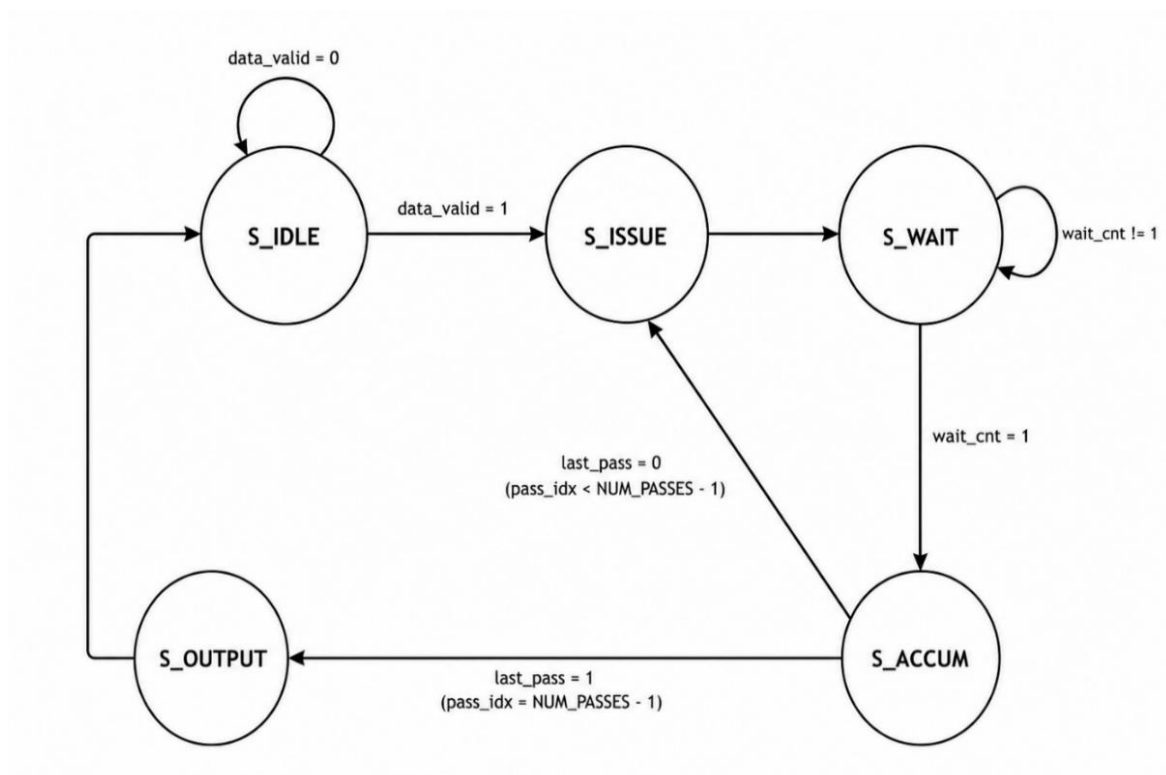
➤ Bộ điều khiển Controller

Bảng 3.14: Các trạng thái của controller

Trạng thái	Chức năng	Điều kiện chuyển trạng thái	Tín hiệu điều khiển chính
S_IDLE	Trạng thái chờ dữ liệu đầu vào từ khối trước.	Khi data_valid = 1 -> S_ISSUE; Ngược lại -> Giữ ở S_IDLE	in_ready = 1 (khi !data_valid), báo sẵn sàng nhận vector mới; pass_idx được giữ ở 0.
S_ISSUE	Phát lệnh bắt đầu tính toán cho batch hiện tại. Khối MAC Stage 1 được kích hoạt để thực hiện 64 phép nhân song song (kết quả ghi vào mult_reg ở chu kỳ kế tiếp).	Chuyển ngay sang S_WAIT ở chu kỳ kế tiếp.	mac_en = 1; wait_cnt được reset về 0.
S_WAIT	Chờ dữ liệu đi qua pipeline 2 tầng. Trong giai đoạn này, kết quả nhân (mult_reg) được cộng thành tree_sum và ghi/cộng dồn	Khi wait_cnt = 2'd1 -> S_ACCUM; Ngược lại -> Giữ ở S_WAIT.	wait_cnt tăng dần; mac_valid_s1/accum_valid lần lượt lên mức cao theo pipeline.

	thực tế vào accum_reg (xảy ra ở chu kỳ thứ hai của state này).		
S_ACCUM	Tại thời điểm này, kết quả cộng dồn của batch hiện tại đã có sẵn trong accum_reg (hoàn tất từ chu kỳ trước). Vai trò chính của state là kiểm tra đã hoàn tất 6 lượt hay chưa, và cập nhật chỉ số cho lượt kế tiếp.	Nếu last_pass = 1 - > S_OUTPUT; Nếu last_pass = 0 - > S_ISSUE.	pass_idx tăng thêm 1 khi chưa hoàn thành tất cả các lượt tính toán.
S_OUTPUT	Xuất kết quả cuối cùng sau khi đã xử lý xong toàn bộ 6 lượt. Thực hiện cộng bias, làm tròn, bão hòa và	Chuyển ngay về S_IDLE ở chu kỳ kế tiếp.	out_valid = 1, đồng thời cập nhật out_class_0, out_class_1, out_class_2; pass_idx reset về 0.

	ghi ra các đầu ra phân loại.		
--	---------------------------------	--	--

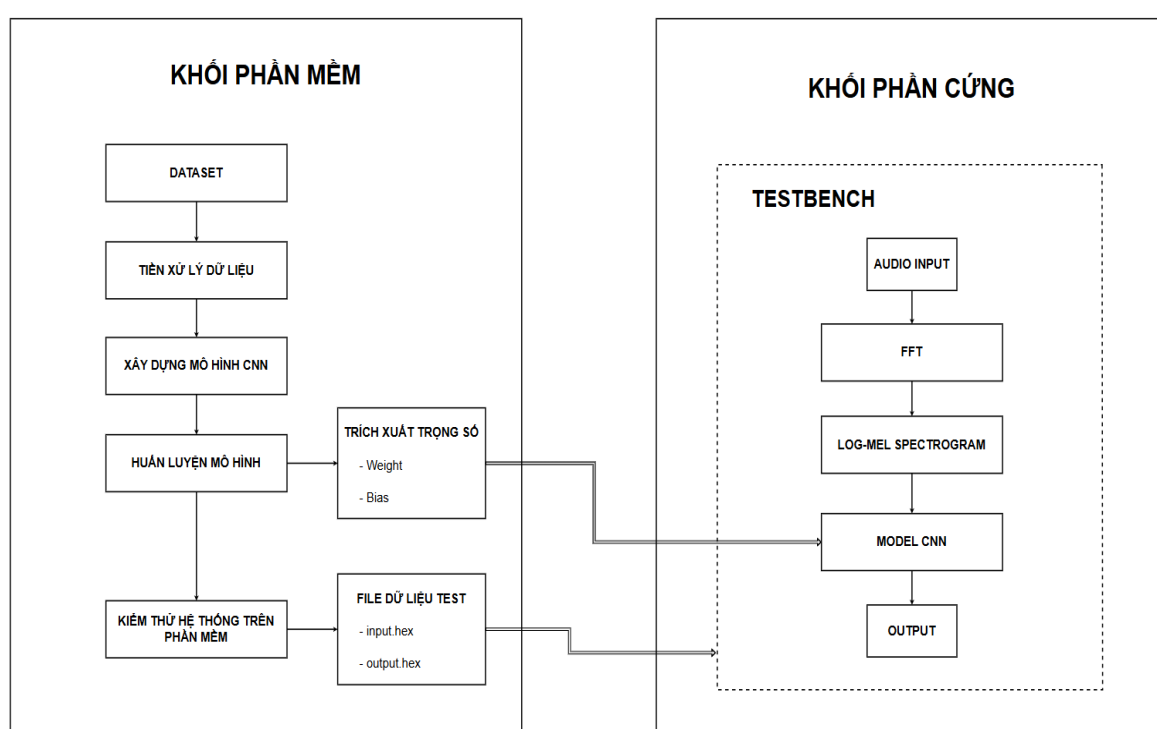


Hình 3.20: Lưu đồ FSM của FC2

Chương 4. THIẾT KẾ KIỂM THỬ

4.1. Tổng quan kiểm thử trên phần mềm

Chương này trình bày quá trình huấn luyện, kiểm thử và đánh giá model CNN cho bài toán phát hiện âm thanh khẩn cấp. Khác với Chương 2 tập trung vào lý thuyết toán học và Chương 3 tập trung vào hiện thực phần cứng, Chương 4 tập trung vào kết quả thực nghiệm, bao gồm chuẩn bị dữ liệu, huấn luyện mô hình, lượng tử hóa, xuất trọng số, kiểm thử bit-true và so sánh giữa phần mềm với phần cứng.



Hình 4.1: Quy trình thiết kế kiểm thử

Quy trình đánh giá gồm các bước chính:

- Bước 1: Chuẩn bị dataset âm thanh.
- Bước 2: Trích xuất đặc trưng Log-Mel theo pipeline mô phỏng phần cứng.
- Bước 3: Huấn luyện model CNN bằng PyTorch.
- Bước 4: Lưu model tốt nhất thành file audio_cnn.pth.
- Bước 5: Gộp BatchNorm vào Conv và trích xuất weight/bias sang file .hex.
- Bước 6: Chạy kiểm thử bit-true bằng Python.

- Bước 7: Chạy testbench Verilog cho model CNN.
- Bước 8: So sánh output Python/PyTorch với output phần cứng.

4.2. Thiết kế mô phỏng phần mềm

4.2.1. Chuẩn bị dataset

Dữ liệu âm thanh được thu âm trực tiếp bằng thiết bị di động trong nhiều điều kiện môi trường thực tế (không gian yên tĩnh, phòng làm việc, ngoài trời có tiếng ồn), nhằm mô phỏng nhiều thực tế cho mô hình. Các mẫu âm thanh khẩn cấp được thu từ nhiều đối tượng khác nhau để làm phong phú đặc trưng tần số giọng nói.

Tập gốc ở định dạng nén .m4a (AAC, 44.1/48 kHz, Stereo) không tương thích trực tiếp với pipeline phần cứng trên FPGA. Nhóm xây dựng script Python dùng thư viện pydub (kết hợp FFmpeg) để tự động xử lý hàng loạt, gồm 3 bước:

- Chuyển định dạng: .m4a $\rightarrow$.wav (PCM), giúp khối phần cứng đọc trực tiếp dữ liệu số nguyên.
- Chuyển mono: `set_channels(1)` — giảm dung lượng và tài nguyên tính toán.
- Downsampling: `set_frame_rate(16000)` — đưa về 16 kHz, đủ giữ dải tần giọng nói. Đây là tần số tối ưu giúp giữ trọn vẹn dải năng lượng giọng nói con người (theo định lý Nyquist, đáp ứng dải tần lên tới 8 kHz).

Các tệp sau xử lý có thời lượng ổn định từ 1–2 giây, sẵn sàng làm đầu vào cho huấn luyện mô hình. Tập dữ liệu gồm 3.000 tệp .wav (16 kHz, Mono), chia cân bằng thành 3 lớp:

- Cứu (1.000 mẫu): tiếng kêu cứu khẩn cấp.
- Cướp (1.000 mẫu): tiếng hô báo động an ninh.
- Unknown (1.000 mẫu): tạp âm nền (xe cộ, gió, nói chuyện bình thường, âm thanh trắng, ...) hoặc từ khóa không khẩn cấp, giúp giảm tỷ lệ báo động giả.

Bảng 4.1: Thống kê tập dữ liệu

STT	Phân lớp	Thời lượng	Định dạng	Tần số lấy mẫu	Số kênh	Số lượng
1	Cứu	1–2s	WAV (PCM)	16 kHz	Mono	1.000
2	Cướp	1–2s	WAV (PCM)	16 kHz	Mono	1.000
3	Unknown	1–2s	WAV (PCM)	16 kHz	Mono	1.000

4.2.2. Tiền xử lý dữ liệu

Khối tiền xử lý dữ liệu được xây dựng nhằm chuyển đổi tín hiệu âm thanh đầu vào từ miền thời gian sang đặc trưng Log-Mel Spectrogram phục vụ cho mô hình CNN. Điểm đặc biệt của khối này là không chỉ thực hiện các phép biến đổi tín hiệu thông thường, mà còn mô phỏng sát hành vi của phần cứng bằng các bước lượng tử hóa, cắt bit, dịch bit, tra bảng LUT và sử dụng ma trận hệ số Mel được nạp từ ROM. Nhờ đó, dữ liệu kiểm thử trên phần mềm có cùng đặc tính với dữ liệu tạo ra từ kiến trúc phần cứng, giúp đánh giá mô hình CNN một cách nhất quán giữa mô phỏng và triển khai thực tế.

Bảng 4.2: Thông số của khối tiền xử lý

Thông số	Giá trị	Vai trò
SR	16 kHz	Tần số lấy mẫu của tín hiệu âm thanh
N_FFT	1024	Độ dài mỗi frame dùng cho FFT
HOP_LENGTH	256	Bước dịch giữa hai frame liên tiếp
MEL_BINS	40	Số dải tần Mel đầu ra
OWIDTH	22 bit	Độ rộng biểu diễn sau khi scale FFT

FFT_SHIFT	6 bit	Số bit dịch phải để mô phỏng scale phần cứng
Q_SHIFT	15 bit	Độ dịch để đưa năng lượng về miền fixed-point trước khi log
LUT_BITS	12 bit	Số bit địa chỉ của bảng log LUT
LUT_SIZE	4096	Kích thước bảng LUT log
FFT_CORE_SCALE	16.0	Hệ số scale đầu ra FFT lõi phần cứng
MAX_FRAMES	64	Số frame đầu ra cố định sau padding/cropping
HW_SCALE_FACTOR	1024	Hệ số chuẩn hóa tensor đầu vào CNN

Quy trình xử lý bắt đầu bằng việc đọc các tệp âm thanh định dạng .wav, sau đó chuẩn hóa về tần số lấy mẫu 16 kHz và chuyển sang biểu diễn PCM 16 bit. Tín hiệu tiếp tục được chia thành các frame có độ dài 1024 mẫu với bước nhảy 256 mẫu để chuẩn bị cho khối FFT. Sau FFT, kết quả phổ được scale, làm tròn và giới hạn theo đúng độ rộng bit của phần cứng. Tiếp theo, hệ thống tính phổ công suất, nhân với ma trận Mel được tạo từ file ROM, rồi thực hiện phép log bằng bảng tra LUT thay cho hàm logarit trực tiếp. Cuối cùng, đặc trưng Log-Mel được chuẩn hóa kích thước bằng padding hoặc cropping để phù hợp với đầu vào cố định của CNN.

4.2.3. Mô hình CNN

Mô hình CNN trên phần mềm được xây dựng có cấu trúc các lớp chính tương tự như trên phần cứng, gồm ba khối tích chập (Convolution), mỗi khối bao gồm lớp Convolution, Batch Normalization, hàm kích hoạt ReLU và MaxPooling để trích xuất đặc trưng của tín hiệu âm thanh. Sau khi làm phẳng (Flatten), đặc trưng được đưa qua lớp Fully Connected 128 neuron, hàm kích hoạt ReLU, lớp Dropout và lớp Fully Connected cuối cùng để phân loại thành ba lớp. Ngoài ra, các lớp lượng tử hóa được chèn vào mô hình nhằm mô phỏng phép tính số cố định trên phần cứng, giúp giảm sai khác giữa mô hình huấn luyện và hệ thống FPGA.

Bảng 4.3: Các thành phần của mô hình

Thành phần	Cấu hình	Chức năng
Input	Log-Mel Spectrogram	Dữ liệu đầu vào
Quantization	Fixed-point	Mô phỏng dữ liệu phần cứng
Conv1 + BatchNorm + ReLU	16 kernel, 3×3	Trích xuất đặc trưng mức thấp
MaxPooling	2×2	Giảm kích thước đặc trưng
Conv2 + BatchNorm + ReLU	32 kernel, 3×3	Trích xuất đặc trưng trung gian
MaxPooling	2×2	Giảm kích thước đặc trưng
Conv3 + BatchNorm + ReLU	64 kernel, 3×3	Trích xuất đặc trưng mức cao
MaxPooling	2×2	Giảm kích thước đặc trưng
Flatten	-	Chuyển đặc trưng thành vector
Fully Connected + ReLU	128 neuron	Kết hợp đặc trưng
Quantization	Fixed-point	Mô phỏng phép tính phần cứng
Dropout	0,4	Giảm hiện tượng overfitting
Fully Connected	3 neuron	Phân loại 3 lớp

4.2.4. Huấn luyện mô hình

Mô hình được huấn luyện bằng PyTorch trên tập dữ liệu âm thanh gồm ba lớp nhận dạng: "CÚU" (nhãn 0), "CƯỚP" (nhãn 1) và "UNKNOWN" (nhãn 2). Dữ liệu

được nạp qua lớp AudioDataset tùy chỉnh, sau đó chia ngẫu nhiên theo tỷ lệ 80/20 (train/val) với seed cố định bằng torch.Generator().manual_seed(42) nhằm đảm bảo tính tái lập của thực nghiệm.

Quá trình huấn luyện áp dụng các kỹ thuật sau:

- Hàm mất mát: Cross-Entropy Loss
- Bộ tối ưu: Adam, learning rate = 0.001, kết hợp gradient clipping (norm = 1.0) để ổn định quá trình hội tụ
- Dừng sớm (Early Stopping): theo dõi val_acc qua từng epoch; nếu không cải thiện sau 6 epoch liên tiếp thì dừng huấn luyện
- Lưu checkpoint: chỉ lưu trọng số mô hình tốt nhất (theo val_acc) vào file audio_cnn.pth

Bảng 4.4: Các thông số giá trị

Thông số	Giá trị	Ghi chú
Framework	PyTorch	Tự động chọn GPU/CPU
Số lớp phân loại	3	CỬU, CUỐP, UNKNOWN
Tỷ lệ tập validation	20%	Seed = 42, phân chia ngẫu nhiên
Batch size	16	pin_memory=True, num_workers=2
Learning rate	0.001	Adam optimizer
Số epoch tối đa	30	Kết thúc sớm nếu hội tụ
Patience (Early Stopping)	6 epoch	Theo dõi val_acc
Gradient clipping	norm = 1.0	Ổn định huấn luyện
Random seed	42	torch, numpy, random
File checkpoint	audio_cnn.pth	Lưu model tốt nhất

```

Epoch 1/30 | Train Loss: 0.4709 | Train Acc: 0.8029 | Val Loss: 0.1096 | Val Acc: 0.9583
✅ Saved best model!
Epoch 2/30 | Train Loss: 0.1806 | Train Acc: 0.9333 | Val Loss: 0.0895 | Val Acc: 0.9667
✅ Saved best model!
Epoch 3/30 | Train Loss: 0.1167 | Train Acc: 0.9625 | Val Loss: 0.2408 | Val Acc: 0.9300
Epoch 4/30 | Train Loss: 0.0847 | Train Acc: 0.9738 | Val Loss: 0.1582 | Val Acc: 0.9550
Epoch 5/30 | Train Loss: 0.0772 | Train Acc: 0.9738 | Val Loss: 0.1466 | Val Acc: 0.9583
Epoch 6/30 | Train Loss: 0.0613 | Train Acc: 0.9817 | Val Loss: 0.0627 | Val Acc: 0.9833
✅ Saved best model!
Epoch 7/30 | Train Loss: 0.0423 | Train Acc: 0.9862 | Val Loss: 0.0639 | Val Acc: 0.9800
Epoch 8/30 | Train Loss: 0.0482 | Train Acc: 0.9838 | Val Loss: 0.1168 | Val Acc: 0.9700
Epoch 9/30 | Train Loss: 0.0462 | Train Acc: 0.9833 | Val Loss: 0.0283 | Val Acc: 0.9933
✅ Saved best model!
Epoch 10/30 | Train Loss: 0.0261 | Train Acc: 0.9912 | Val Loss: 0.0240 | Val Acc: 0.9900
Epoch 11/30 | Train Loss: 0.0245 | Train Acc: 0.9933 | Val Loss: 0.0511 | Val Acc: 0.9883
Epoch 12/30 | Train Loss: 0.0314 | Train Acc: 0.9921 | Val Loss: 0.0453 | Val Acc: 0.9900
Epoch 13/30 | Train Loss: 0.0094 | Train Acc: 0.9967 | Val Loss: 0.0413 | Val Acc: 0.9867
Epoch 14/30 | Train Loss: 0.0105 | Train Acc: 0.9971 | Val Loss: 0.0762 | Val Acc: 0.9867
Epoch 15/30 | Train Loss: 0.0075 | Train Acc: 0.9975 | Val Loss: 0.0527 | Val Acc: 0.9883
Early stopping triggered!
Training finished! Best Val Acc: 0.9933

```

Hình 4.2: Kết quả training mô hình CNN

Bảng 4.5: Các chỉ số huấn luyện

Chỉ số	Ý nghĩa	Giá trị tốt	Kết quả đạt được
Train Loss	Giá trị hàm mất mát (Cross-Entropy) trên tập huấn luyện. Phản ánh mức độ sai lệch giữa dự đoán và nhãn thực tế trong quá trình học.	Càng nhỏ càng tốt, xu hướng giảm đều theo epoch	0,4709 → 0,0075 (giảm ~63 lần)
Train Acc	Tỷ lệ phân loại đúng trên tập huấn luyện. Cho thấy mô hình đã học được đặc trưng từ dữ liệu ở mức độ nào.	Càng cao càng tốt, thường đạt > 95% sau vài epoch	80,29% → 99,75%

Val Loss	Giá trị hàm mất mát trên tập validation (không tham gia huấn luyện). Dùng để phát hiện overfitting: nếu Val Loss tăng trong khi Train Loss giảm thì mô hình bị overfit.	Càng nhỏ càng tốt, không tăng ngược chiều Train Loss	Min = 0,0240 (epoch 10)
Val Acc	Tỷ lệ phân loại đúng trên tập validation. Là chỉ số chính để đánh giá khả năng tổng quát hóa và làm tiêu chí lưu checkpoint tốt nhất.	Càng cao càng tốt, gần với Train Acc	Max = 99,33% (epoch 9)

4.3. Nền tảng phần cứng thực nghiệm (Kit FPGA ZCU104)

Để thực nghiệm và đánh giá lõi IP, đề tài sử dụng kit phát triển Xilinx Zynq UltraScale+ MPSoC ZCU104. Trọng tâm của kit là chip SoC XCZU7EV, kết hợp linh hoạt giữa Hệ thống xử lý phần mềm (PS - gồm các lõi ARM Cortex-A53 và Cortex-R5F để điều hành) và Logic lập trình phần cứng (PL - cấu trúc FPGA FinFET 16nm). Ngoài ra, kit tích hợp sẵn chip Codec âm thanh chuyên dụng, đóng vai trò nhận tín hiệu thô từ môi trường qua cổng Mic/Line-In và thực hiện biến đổi tương tự - số (ADC) để cung cấp luồng dữ liệu liên tục cho hệ thống.



Hình 4.3: Ảnh kit Zynq UltraScale+ MPSoC ZCU104

Giới hạn tài nguyên vật lý của chip XCZU7EV trên kit ZCU104 — cơ sở để tính toán tỷ lệ sử dụng tài nguyên hệ thống ở chương thực nghiệm — được tóm tắt ngắn gọn trong Bảng 4.6.

Bảng 4.6: Thông số tài nguyên của chip FPGA SoC XCZU7EV

Thành phần	Số lượng tài nguyên
LUT	230,400
LUTRAM	101,760
Flip-Flop (FF)	460,800
Block RAM (BRAM 36 Kb)	312
DSP Slice	1,728
User I/O	360
Global Clock Buffer (BUFG)	544

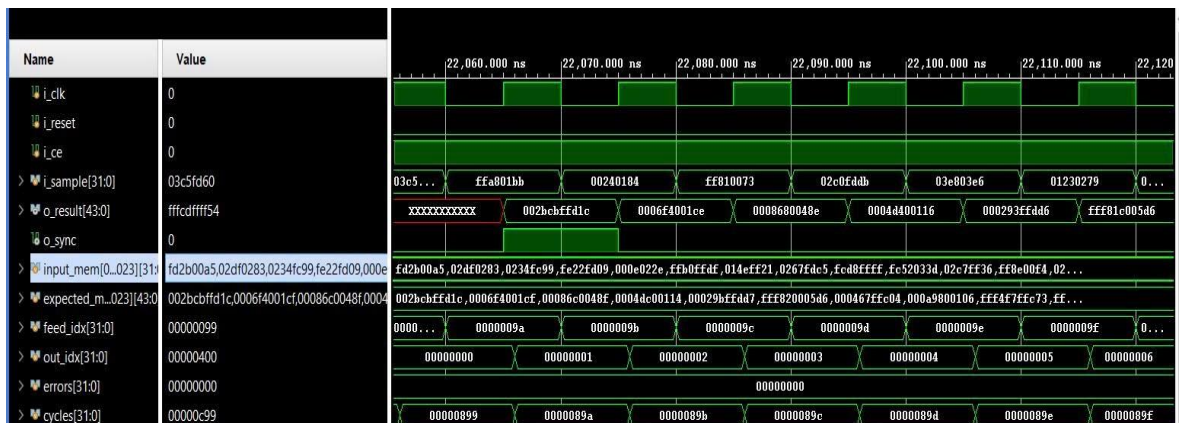
Giá trị logit thu được từ mô hình phần cứng Verilog có độ tương đồng cao với kết quả của mô hình golden trên Python. Sai lệch giữa hai kết quả chỉ ở mức một số đơn vị LSB và không làm thay đổi thứ tự độ lớn giữa ba giá trị logit. Trong các trường hợp kiểm thử, logit tương ứng với lớp nhãn thực tế vẫn đạt giá trị lớn nhất, vì vậy kết quả phân loại của phần cứng thống nhất với phần mềm. Kết quả này cho thấy quá trình lượng tử hóa, xuất trọng số và hiện thực mô hình CNN fixed-point trên phần cứng có độ chính xác phù hợp.

Chương 5. KẾT QUẢ MÔ PHỎNG VÀ KIỂM THỬ

5.1. Mô phỏng Behavioral/RTL (pre-synthesis)

5.1.1. Khối FFT

Khối kiểm thử cho FFT được xây dựng nhằm xác minh chức năng của mô-đun FFT 1024 điểm. Testbench đọc dữ liệu đầu vào từ tệp input.mem và kết quả tham chiếu từ expected.mem, sau đó liên tục cấp mẫu dữ liệu cho khối FFT thông qua tín hiệu i_ce. Khi tín hiệu o_sync xuất hiện, các mẫu đầu ra được thu nhận và so sánh với kết quả tham chiếu theo từng bin. Sai số phần thực và phần ảo được đánh giá với ngưỡng cho phép ± 8 . Sai số này xảy ra do quá trình lượng tử hóa. Sau khi hoàn tất 1024 điểm FFT, testbench thống kê số lỗi và đưa ra kết luận PASS hoặc FAIL cho toàn bộ phép biến đổi.



Hình 5.1: Kết quả mô phỏng FFT

Bảng 5.1: Thông số kiểm thử FFT

Thông số	Giá trị
Số điểm FFT	1024
Ngưỡng sai số	± 8
Số bin kiểm tra	1024
Số bin PASS	1024

Số bin FAIL	0
Sai lệch cực đại phần thực	2
Sai lệch cực đại phần ảo	2

```

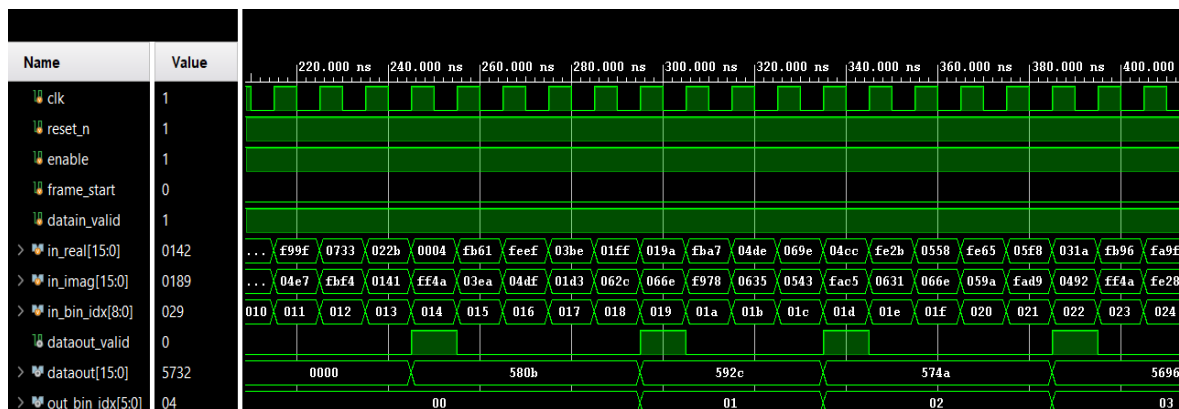
bin 995: python=(-1681,-135) verilog=(-1679,-136) diff=(2,-1) PASS
bin 996: python=(-1632,1121) verilog=(-1634,1121) diff=(-2,0) PASS
bin 997: python=(212,145) verilog=(213,145) diff=(1,0) PASS
bin 998: python=(1629,1594) verilog=(1630,1594) diff=(1,0) PASS
bin 999: python=(-1517,637) verilog=(-1518,637) diff=(-1,0) PASS
bin 1000: python=(-2394,473) verilog=(-2394,471) diff=(0,-2) PASS
bin 1001: python=(1756,-890) verilog=(1758,-889) diff=(2,1) PASS
bin 1002: python=(1160,1168) verilog=(1160,1167) diff=(0,-1) PASS
bin 1003: python=(2145,1451) verilog=(2144,1450) diff=(-1,-1) PASS
bin 1004: python=(-908,-3839) verilog=(-907,-3838) diff=(1,1) PASS
bin 1005: python=(1694,49) verilog=(1692,48) diff=(-2,-1) PASS
bin 1006: python=(1171,-860) verilog=(1172,-860) diff=(1,0) PASS
bin 1007: python=(1296,-1333) verilog=(1295,-1334) diff=(-1,-1) PASS
bin 1008: python=(332,-854) verilog=(334,-852) diff=(2,2) PASS
bin 1009: python=(-220,-1562) verilog=(-219,-1562) diff=(1,0) PASS
bin 1010: python=(-272,-617) verilog=(-272,-616) diff=(0,1) PASS
bin 1011: python=(-913,-1415) verilog=(-913,-1414) diff=(0,1) PASS
bin 1012: python=(708,2777) verilog=(707,2778) diff=(-1,1) PASS
bin 1013: python=(-1799,-1216) verilog=(-1799,-1216) diff=(0,0) PASS
bin 1014: python=(147,862) verilog=(146,861) diff=(-1,-1) PASS
bin 1015: python=(-1782,-1140) verilog=(-1780,-1142) diff=(2,-2) PASS
bin 1016: python=(-1213,-1430) verilog=(-1213,-1429) diff=(0,1) PASS
bin 1017: python=(-1420,-2084) verilog=(-1419,-2084) diff=(1,0) PASS
bin 1018: python=(-684,211) verilog=(-684,212) diff=(0,1) PASS
bin 1019: python=(-1806,1439) verilog=(-1806,1438) diff=(0,-1) PASS
bin 1020: python=(-1032,392) verilog=(-1034,394) diff=(-2,2) PASS
bin 1021: python=(-3563,-140) verilog=(-3564,-139) diff=(-1,1) PASS
bin 1022: python=(2194,483) verilog=(2193,485) diff=(-1,2) PASS
bin 1023: python=(-201,-172) verilog=(-201,-172) diff=(0,0) PASS
FFT TEST PASS: 1024 bins compared, tolerance=8
$finish called at time : 32300 ns : File "D:/NAM4/DOAN2/fft1024_1in/tb_fftmain.v" Line 123

```

Hình 5.2: Kết quả kiểm thử FFT

5.1.2. Khối Log-Mel

Khối kiểm thử cho khối Log-Mel được xây dựng nhằm xác minh chức năng của mô-đun Log-Mel Spectrogram. Testbench đọc dữ liệu phổ FFT từ tệp spectrogram_in_stimulus.mem, cấp liên tiếp hai khung dữ liệu gồm 512 bin và so sánh 40 giá trị Log-Mel đầu ra với kết quả tham chiếu trong tệp spectrogram_out_expected.mem. Sau khi bỏ qua khung khởi tạo, các giá trị đầu ra của phần cứng được đối chiếu với kết quả sinh từ Python theo từng kênh Mel. Kết quả mô phỏng cho thấy toàn bộ 40 kênh Mel đều được tính toán chính xác.



Hình 5.3: Kết quả mô phỏng Log-Mel

```

-> VERILOG Hardware: Chuoi ghep (Hex) = 236230 | Idx_out = 35, Log_out = 6230
-> PYTHON Golden:   Chuoi ghep (Hex) = 236230 | Idx_exp = 35, Log_exp = 6230
[=> CHUAN XAC]

-----
[OUTPUT @Time 9465000 ps] Kiem tra ngo ra thu: 36
-> VERILOG Hardware: Chuoi ghep (Hex) = 246285 | Idx_out = 36, Log_out = 6285
-> PYTHON Golden:   Chuoi ghep (Hex) = 246285 | Idx_exp = 36, Log_exp = 6285
[=> CHUAN XAC]

-----
[OUTPUT @Time 9785000 ps] Kiem tra ngo ra thu: 37
-> VERILOG Hardware: Chuoi ghep (Hex) = 256314 | Idx_out = 37, Log_out = 6314
-> PYTHON Golden:   Chuoi ghep (Hex) = 256314 | Idx_exp = 37, Log_exp = 6314
[=> CHUAN XAC]

-----
[OUTPUT @Time 10135000 ps] Kiem tra ngo ra thu: 38
-> VERILOG Hardware: Chuoi ghep (Hex) = 2662f5 | Idx_out = 38, Log_out = 62f5
-> PYTHON Golden:   Chuoi ghep (Hex) = 2662f5 | Idx_exp = 38, Log_exp = 62f5
[=> CHUAN XAC]

-----
[OUTPUT @Time 10505000 ps] Kiem tra ngo ra thu: 39
-> VERILOG Hardware: Chuoi ghep (Hex) = 27633c | Idx_out = 39, Log_out = 633c
-> PYTHON Golden:   Chuoi ghep (Hex) = 27633c | Idx_exp = 39, Log_exp = 633c
[=> CHUAN XAC]

=====
PASS: Da kiem tra thanh cong 40 Mel Channels!
=====
$finish called at time : 10505 ns : File "D:/NAM4/DOAN2/code/FFT/FFT.srcs/sources_1/new/tb_log_mel_spectrogram.v" Line 197
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_log_mel_spectrogram_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 30000ns
launch_simulation: Time (s): cpu = 00:00:02 ; elapsed = 00:00:06 . Memory (MB): peak = 1526.016 ; gain = 0.000

```

Hình 5.4: Kết quả kiểm thử Log-Mel

Bảng 5.2: Thông số kiểm thử Log-Mel

Thông số	Giá trị
Số bin FFT đầu vào	512
Số kênh Mel đầu ra	40
Số kênh kiểm tra	40
Số kênh PASS	40

Số kênh FAIL	0
Độ sai lệch	0

5.1.3. Khối CNN

Khối kiểm thử cho khối CNN được xây dựng nhằm xác minh chức năng của toàn bộ mô hình CNN. Testbench đọc dữ liệu đầu vào từ tệp model_cnn_input.hex, lần lượt đưa 2560 mẫu vào mạng và chờ tín hiệu out_valid. Ba giá trị phân loại đầu ra (out_class_0, out_class_1, out_class_2) được so sánh với kết quả tham chiếu trong tệp model_cnn_output_gold.hex. Kết quả mô phỏng cho thấy các giá trị phân loại của phần cứng hoàn toàn trùng khớp với kết quả tham chiếu, xác nhận tính đúng đắn của mô hình CNN.



Hình 5.5: Kết quả mô phỏng CNN (nạp dữ liệu)



Hình 5.6: Kết quả mô phỏng CNN (xuất kết quả)


```

=====
CHECK TIME: 4541085000 ns
Expected: C0=fad3 C1=0772 C2=f9ae
Actual   : C0=fad3 C1=0772 C2=f9ae
[PASS] model_CNN output matches golden.

```

Hình 5.7: Kết quả kiểm thử CNN

Bảng 5.3: Thông số kiểm thử model CNN

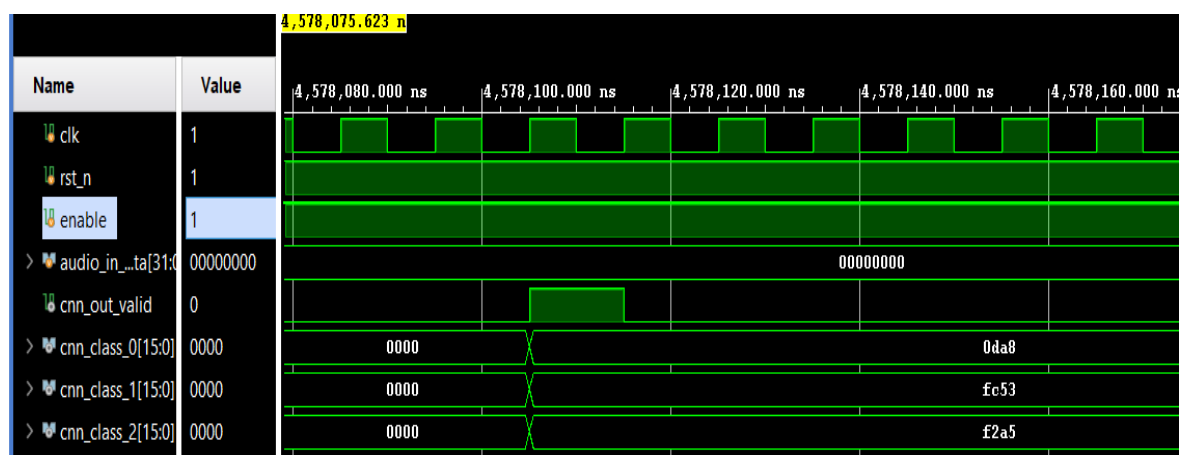
Thông số	Giá trị
Số lớp đầu ra	3
Class 0 (Expected / Actual)	FAD3 / FAD3
Class 1 (Expected / Actual)	0772 / 0772
Class 2 (Expected / Actual)	F9AE / F9AE
Số lớp khớp	3/3

5.1.4. Kết quả mô phỏng của toàn hệ thống (FFT + Log-Mel + CNN)

Khối kiểm thử toàn hệ thống được xây dựng nhằm xác minh hoạt động của toàn bộ hệ thống nhận dạng âm thanh. Qua kiểm thử trên Python, kết quả hệ thống nhận diện được đúng 30 testcase. Sau đó từ kết quả kiểm thử trên Python trích xuất ra dữ liệu để kiểm thử phần cứng. Testbench đọc 65536 mẫu/testcase, đưa dữ liệu qua chuỗi xử lý FFT, Log-Mel Spectrogram và CNN, sau đó so sánh ba giá trị phân loại đầu ra với kết quả tham chiếu từ Python. Kết quả mô phỏng cho thấy các đầu ra phân loại của phần cứng phù hợp với kết quả từ Python trong giới hạn sai số cho phép, xác nhận tính đúng đắn của toàn bộ hệ thống.

#	FILE	NHÃN THẬT	DỰ ĐOÁN	LOGIT [CỬU, CƯỚP, UNK]	STATUS
1	test_cuu_1.wav	CỬU	CỬU	[3495, -944, -3417]	✓
2	test_cuu_2.wav	CỬU	CỬU	[5325, -1517, -4359]	✓
3	test_cuu_3.wav	CỬU	CỬU	[5111, -1989, -3821]	✓
4	test_cuu_4.wav	CỬU	CỬU	[3992, -582, -4061]	✓
5	test_cuu_5.wav	CỬU	CỬU	[2603, -1031, -1787]	✓
6	test_cuu_6.wav	CỬU	CỬU	[3806, -1240, -3170]	✓
7	test_cuu_7.wav	CỬU	CỬU	[2998, 51, -3537]	✓
8	test_cuu_8.wav	CỬU	CỬU	[4944, -375, -5704]	✓
9	test_cuu_9.wav	CỬU	CỬU	[4668, -941, -4150]	✓
10	test_cuu_10.wav	CỬU	CỬU	[4479, -654, -4708]	✓
11	test_cuop_1.wav	CƯỚP	CƯỚP	[-501, 2642, -4340]	✓
12	test_cuop_2.wav	CƯỚP	CƯỚP	[-1164, 2048, -2348]	✓
13	test_cuop_3.wav	CƯỚP	CƯỚP	[-2377, 3586, -3932]	✓
14	test_cuop_4.wav	CƯỚP	CƯỚP	[-2068, 3418, -4078]	✓
15	test_cuop_5.wav	CƯỚP	CƯỚP	[-2356, 2721, -2307]	✓
16	test_cuop_6.wav	CƯỚP	CƯỚP	[-437, 1608, -2265]	✓
17	test_cuop_7.wav	CƯỚP	CƯỚP	[-5131, 4404, -3266]	✓
18	test_cuop_8.wav	CƯỚP	CƯỚP	[-2631, 4365, -4972]	✓
19	test_cuop_9.wav	CƯỚP	CƯỚP	[-2548, 3500, -3883]	✓
20	test_cuop_10.wav	CƯỚP	CƯỚP	[-3728, 5249, -6200]	✓
21	test_unk_1.wav	UNKNOWN	UNKNOWN	[-1403, -999, 1358]	✓
22	test_unk_2.wav	UNKNOWN	UNKNOWN	[-1058, -1828, 1960]	✓
23	test_unk_3.wav	UNKNOWN	UNKNOWN	[-1139, -995, 1269]	✓
24	test_unk_4.wav	UNKNOWN	UNKNOWN	[-1851, -1956, 2511]	✓
25	test_unk_5.wav	UNKNOWN	UNKNOWN	[-818, -819, 989]	✓
26	test_unk_6.wav	UNKNOWN	UNKNOWN	[-1357, -2018, 2295]	✓
27	test_unk_7.wav	UNKNOWN	UNKNOWN	[-1297, -2013, 2257]	✓
28	test_unk_8.wav	UNKNOWN	UNKNOWN	[-1231, -2104, 2343]	✓
29	test_unk_9.wav	UNKNOWN	UNKNOWN	[-1758, -1830, 2338]	✓
30	test_unk_10.wav	UNKNOWN	UNKNOWN	[-1399, -1294, 1705]	✓

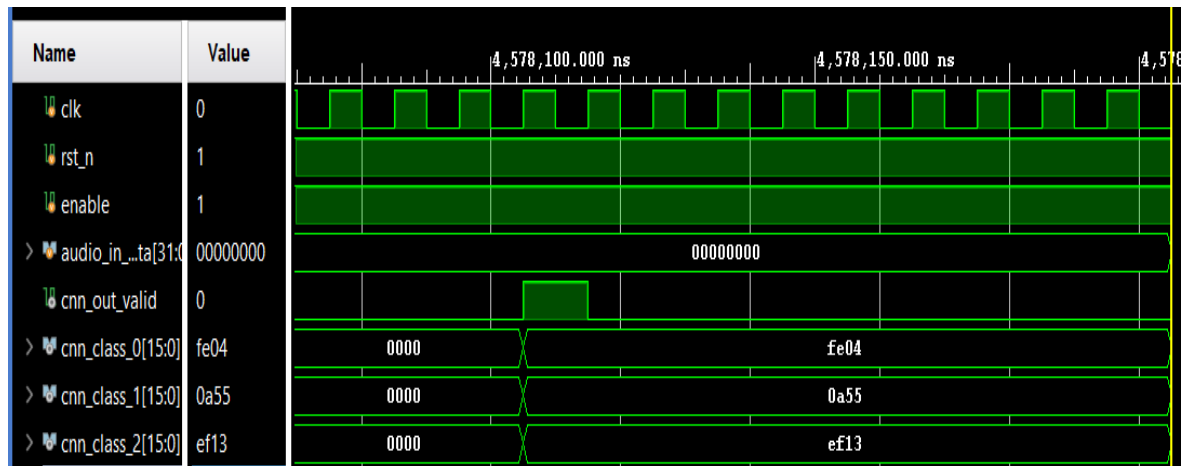
Hình 5.8: Kết quả nhận diện của hệ thống trên phần mềm



Hình 5.9: Kết quả mô phỏng hệ thống

[Nguon]	CUU (Class 0)	CUOP (Class 1)	UNK (Class 2)
Python	3495	-944	-3417
Verilog	3496	-941	-3419
Do lech	1	3	2

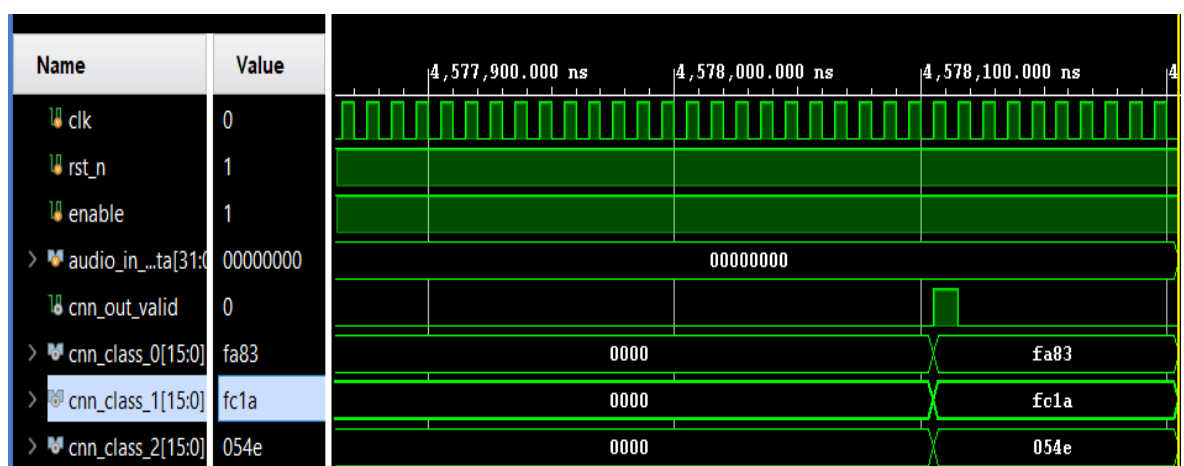
Hình 5.10: Kết quả kiểm thử hệ thống



Hình 5.11: Kết quả mô phỏng hệ thống

[Nguon]	CUU (Class 0)	CUOP (Class 1)	UNK (Class 2)
Python	-501	2642	-4340
Verilog	-508	2645	-4333
Do lech	7	3	7

Hình 5.12: Kết quả kiểm thử hệ thống

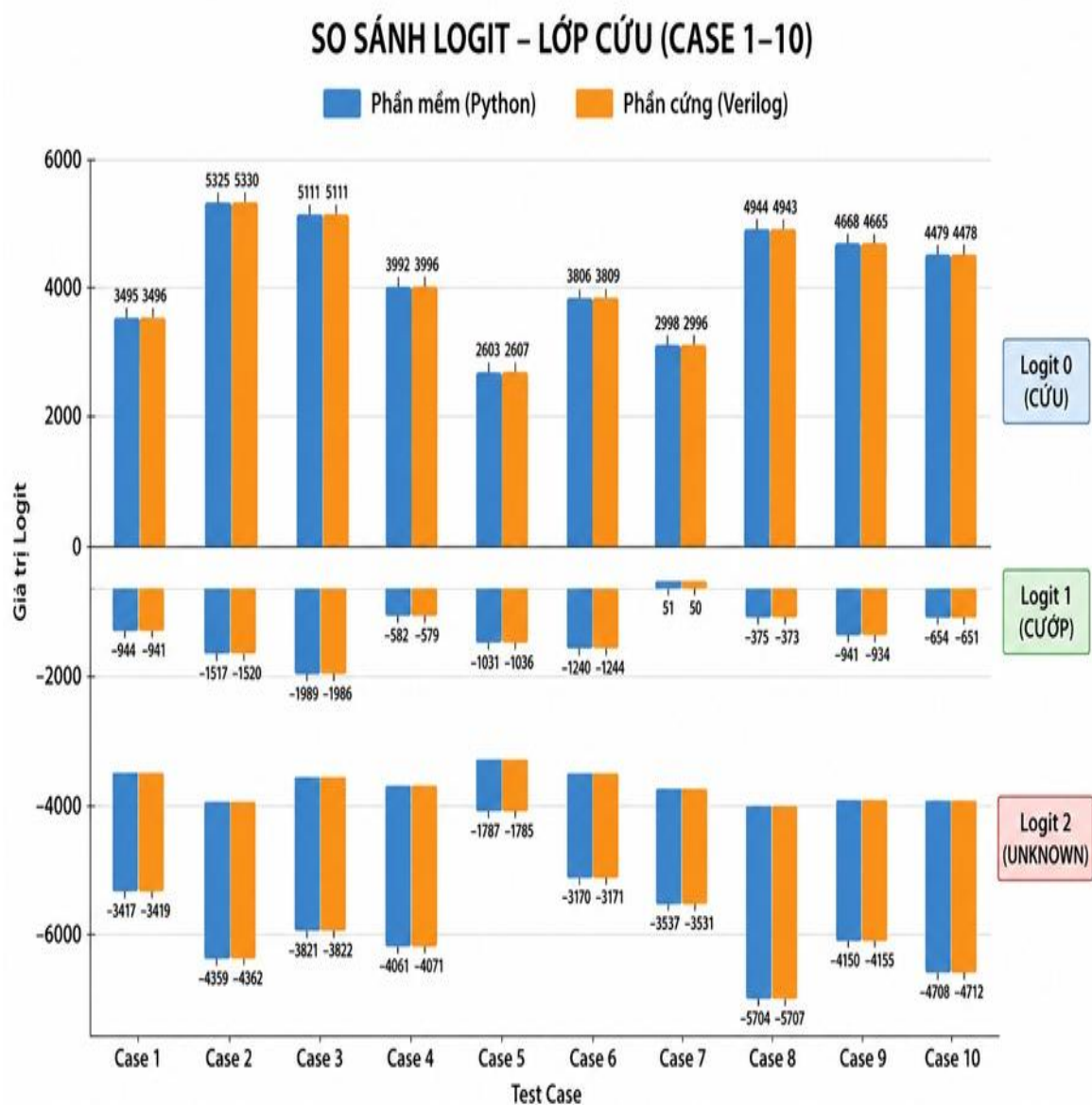


Hình 5.13: Kết quả mô phỏng hệ thống

[Nguon]	CUU (Class 0)	CUOP (Class 1)	UNK (Class 2)
Python	-1403	-999	1358
Verilog	-1405	-998	1358
Do lech	2	1	0

Hình 5.14: Kết quả kiểm thử hệ thống

Các ảnh mô phỏng và kiểm thử trên biểu thị cho giá trị của mỗi testcase đầu tiên thuộc lớp tương ứng. Qua quá trình kiểm thử hoàn tất 30 testcase, các giá trị logit của phần mềm và phần cứng được trình bày qua các biểu đồ so sánh.



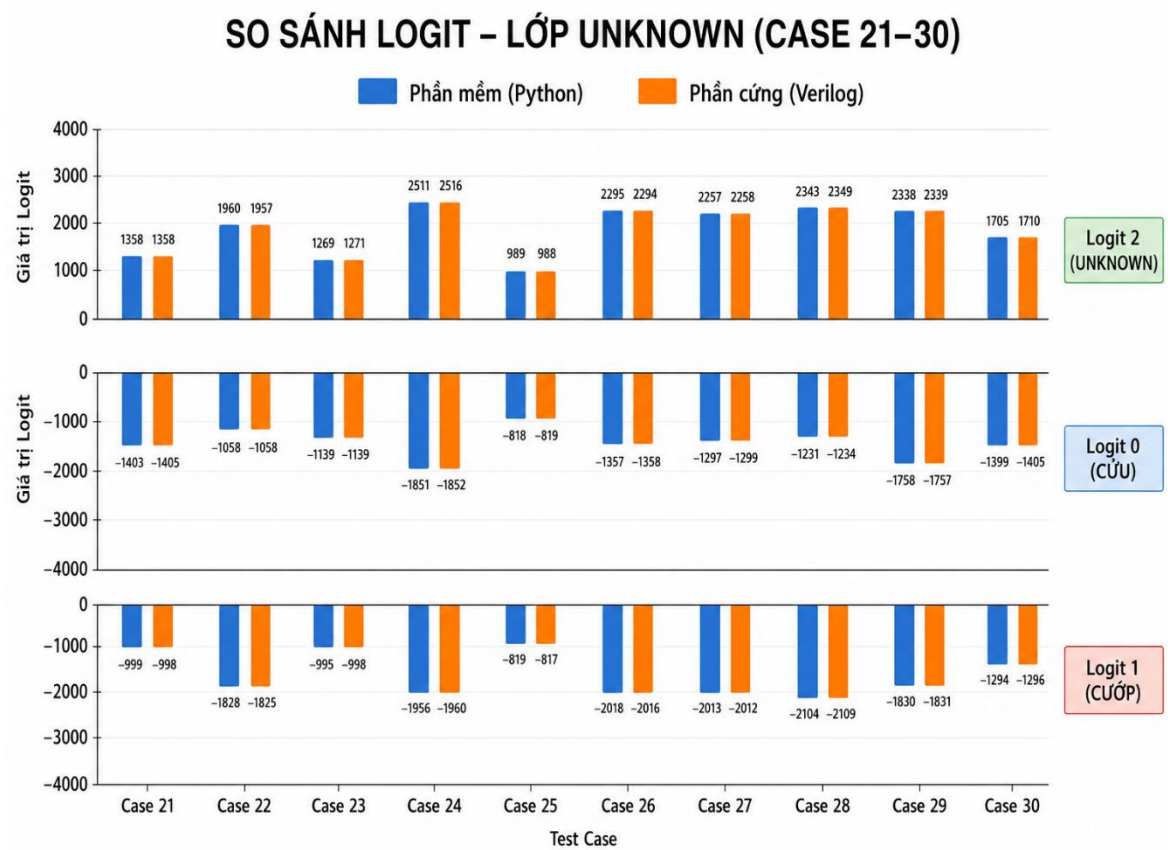
Hình 5.15: Biểu đồ so sánh giá trị logit các testcase lớp CỬU

SO SÁNH LOGIT - LỚP CƯỚP (CASE 11-20)



Hình 5.16: Biểu đồ so sánh giá trị logit các testcase lớp CƯỚP

Qua kết quả kiểm thử, có thể thấy được có giá trị sai số giữa kết quả trên phần mềm và phần cứng. Độ lệch này là do sai số tích trữ từ FFT sau quá trình lượng tử hóa gây ra, với độ lệch < 30 là sai số có thể chấp nhận được và không ảnh hưởng nhiều tới kết quả hệ thống.



Hình 5.17: Biểu đồ so sánh giá trị logit các testcase của lớp UNKNOWN

5.2. Tổng hợp thiết kế (Synthesis)

5.2.1. Ước lượng mức sử dụng tài nguyên phần cứng

Resource	Estimation	Available	Utilization...
LUT	131937	230400	57.26
LUTRAM	1427	101760	1.40
FF	166134	460800	36.05
BRAM	181.50	312	58.17
DSP	161	1728	9.32
IO	84	360	23.33
BUFG	1	544	0.18

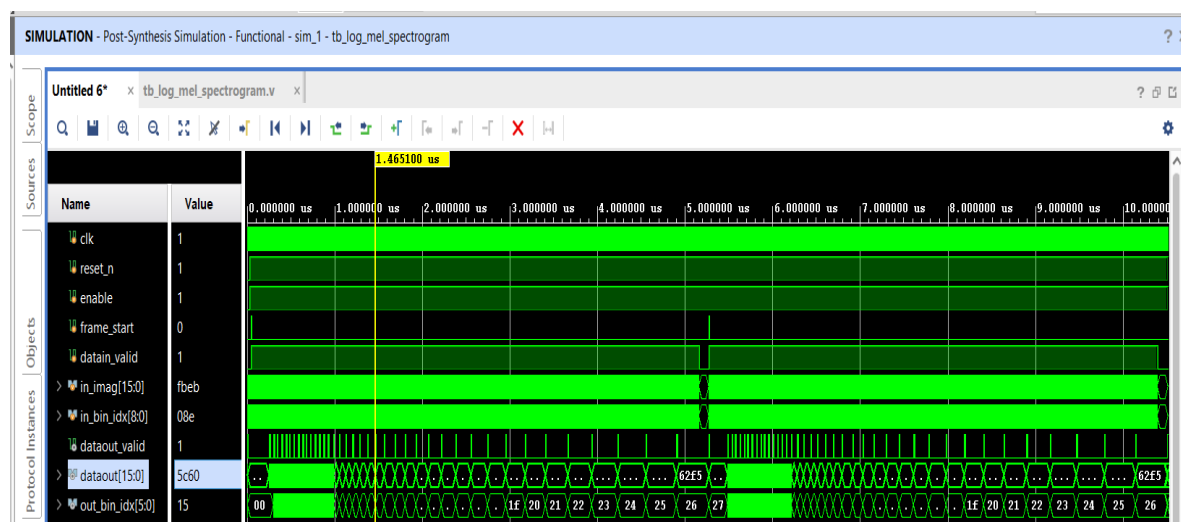
Hình 5.18: Bảng kết quả ước lượng mức sử dụng tài nguyên (Synthesis)

Kết quả cho thấy thiết kế được ước lượng mức sử dụng khoảng 57.26% LUT và 36.05% Flip-Flop trên tổng tài nguyên khả dụng của chip, nằm trong giới hạn cho phép của kit ZCU104. Tỷ lệ sử dụng BRAM tương đối cao (58.17%), chủ yếu do các khối lưu trữ trọng số trong mô hình CNN và FIFO tự thiết kế. Số lượng DSP được sử dụng (161/1728, tương ứng 9.32%) phản ánh các phép nhân trong Log-Mel và CNN.

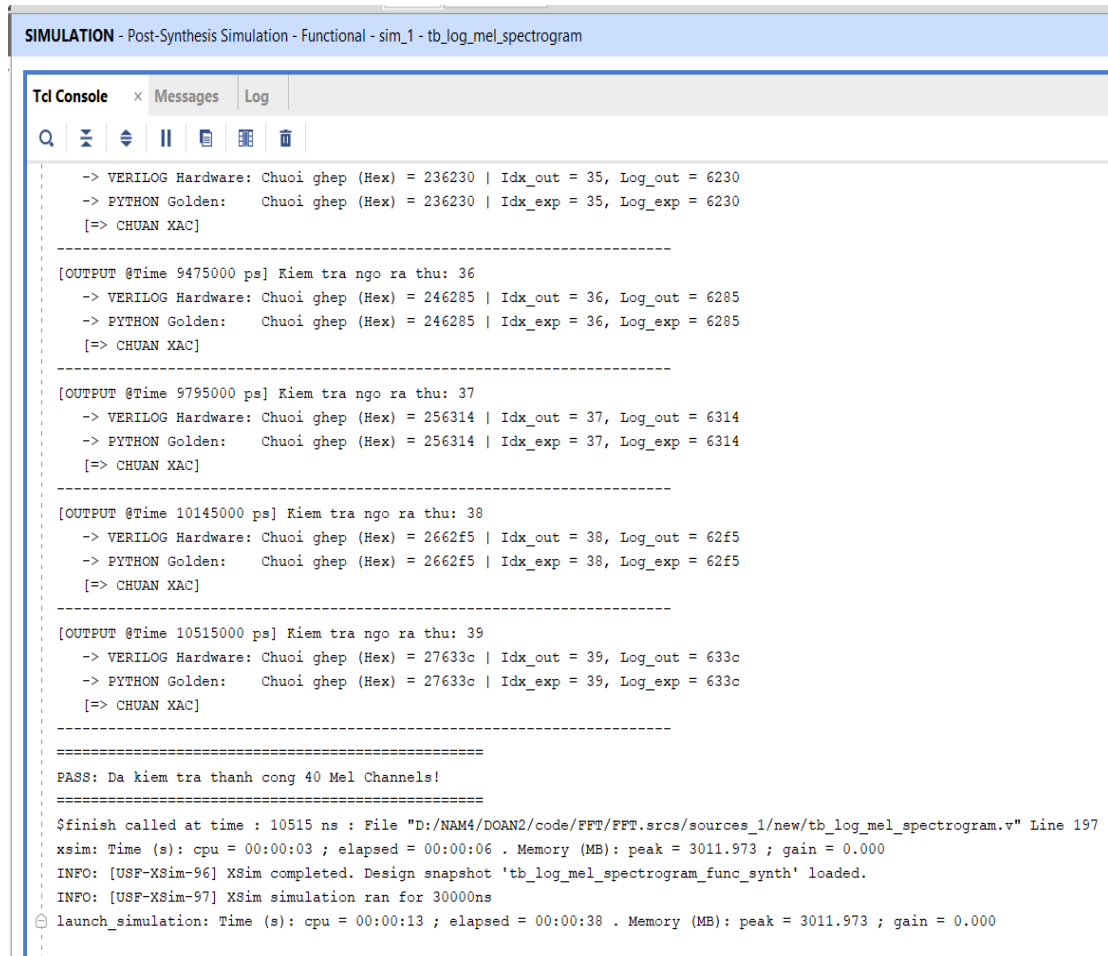
5.2.2. Mô phỏng chức năng sau tổng hợp

Sau khi hoàn thành quá trình tổng hợp (Synthesis), thiết kế được mô phỏng lại bằng Post-Synthesis Functional Simulation nhằm xác minh rằng chức năng của mạch sau khi được ánh xạ sang các phần tử phần cứng (LUT, DSP, BRAM, FF,...) vẫn tương đương với thiết kế RTL ban đầu.

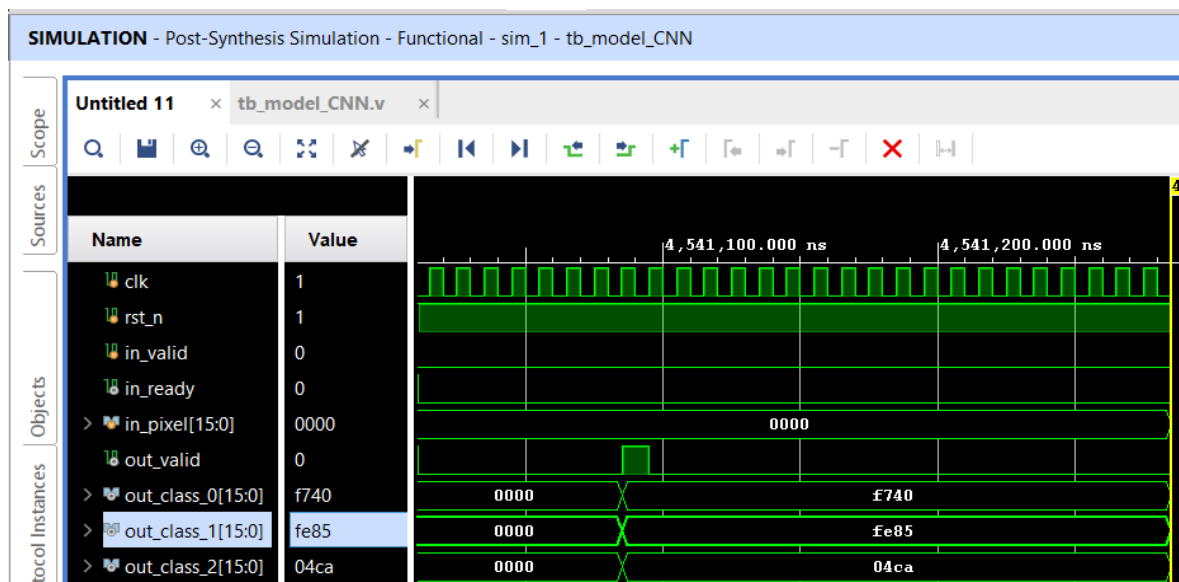
Kết quả mô phỏng sau tổng hợp của từng khối chức năng cho thấy các khối Log-Mel Spectrogram và CNN đều hoạt động đúng theo đặc tả thiết kế, với kết quả đầu ra phù hợp với mô hình tham chiếu. Đối với khối FFT, do được tích hợp từ lõi IP mã nguồn mở dbclockfft của ZipCPU, kết quả mô phỏng xuất hiện một số sai số so với giá trị tham chiếu. Sai số này được xác định có thể bắt nguồn từ quá trình hiện thực các bộ nhớ Twiddle ROM chưa hoàn toàn chính xác, dẫn đến sự sai lệch của các hệ số twiddle trong quá trình tính toán FFT và làm ảnh hưởng đến kết quả đầu ra.



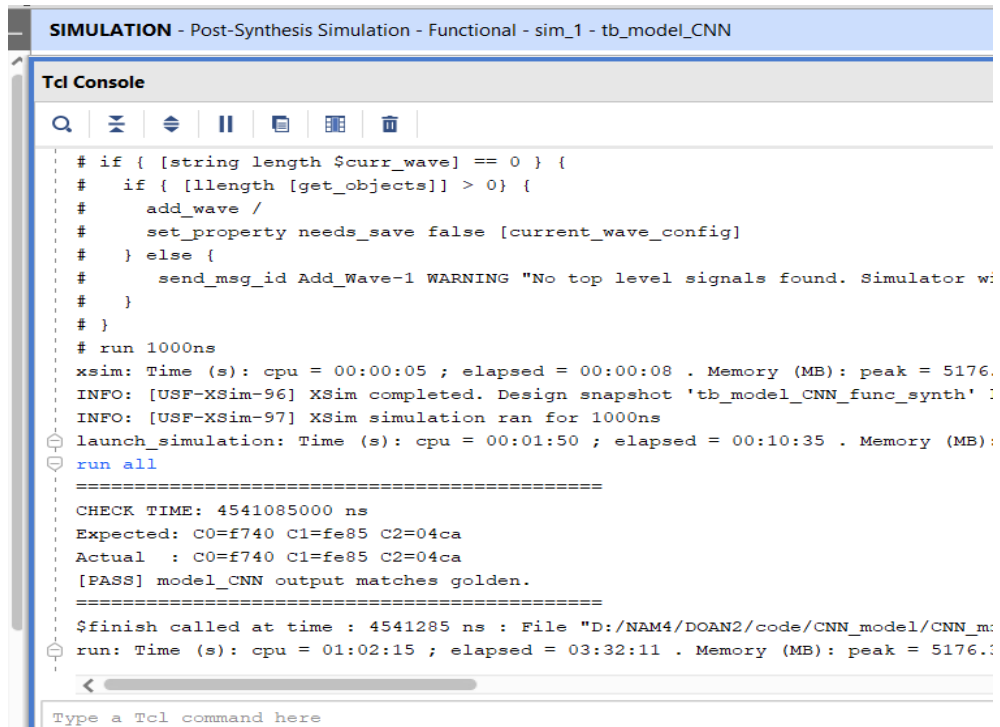
Hình 5.19: Kết quả mô phỏng sau tổng hợp của khối Log-Mel



Hình 5.20: Kết quả kiểm thử sau tổng hợp của khối Log-Mel



Hình 5.21: Kết quả mô phỏng khối CNN sau tổng hợp



```

SIMULATION - Post-Synthesis Simulation - Functional - sim_1 - tb_model_CNN

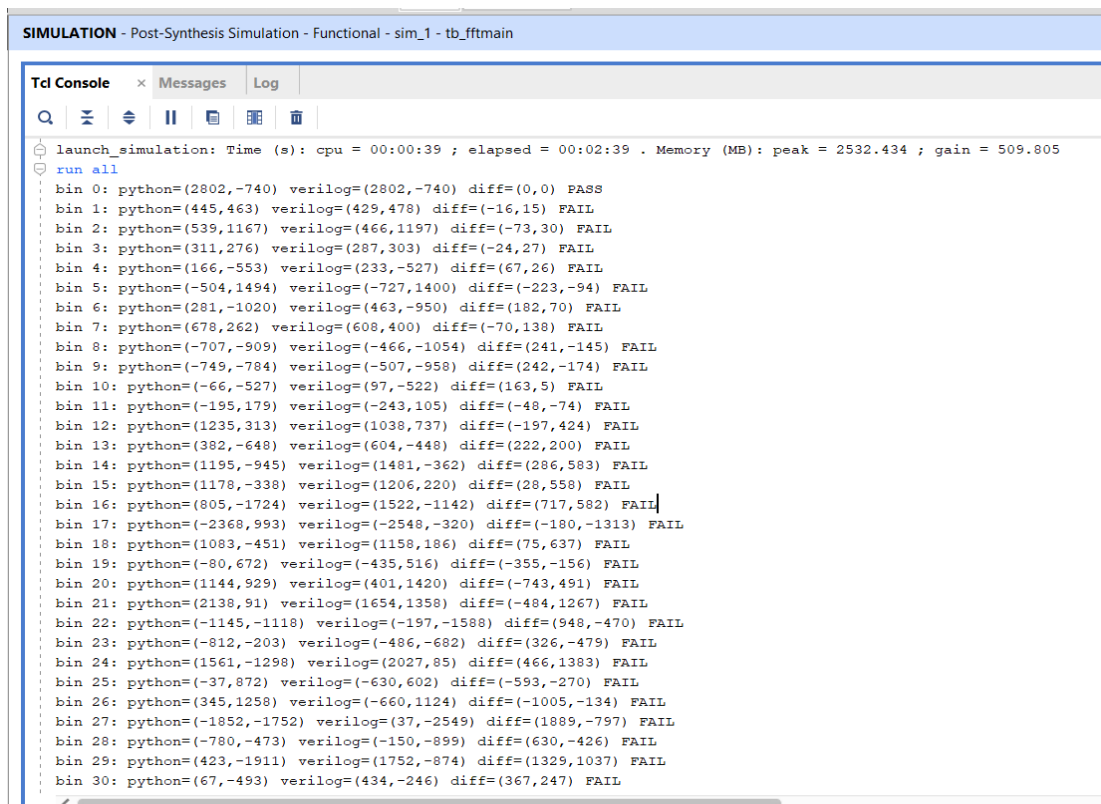
Tcl Console

# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0 } {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#     send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator w:
#   }
# }
# run 1000ns
xsim: Time (s): cpu = 00:00:05 ; elapsed = 00:00:08 . Memory (MB): peak = 5176.
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_model_CNN_func_synth' :
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:01:50 ; elapsed = 00:10:35 . Memory (MB):
run all
=====
CHECK TIME: 4541085000 ns
Expected: C0=f740 C1=fe85 C2=04ca
Actual   : C0=f740 C1=fe85 C2=04ca
[PASS] model_CNN output matches golden.
=====
$finish called at time : 4541285 ns : File "D:/NAM4/DOAN2/code/CNN_model/CNN_m:
run: Time (s): cpu = 01:02:15 ; elapsed = 03:32:11 . Memory (MB): peak = 5176.:

Type a Tcl command here

```

Hình 5.22: Kết quả kiểm thử sau tổng hợp của khối CNN



```

SIMULATION - Post-Synthesis Simulation - Functional - sim_1 - tb_fftmain

Tcl Console x Messages Log

launch_simulation: Time (s): cpu = 00:00:39 ; elapsed = 00:02:39 . Memory (MB): peak = 2532.434 ; gain = 509.805
run all
bin 0: python=(2802,-740) verilog=(2802,-740) diff=(0,0) PASS
bin 1: python=(445,463) verilog=(429,478) diff=(-16,15) FAIL
bin 2: python=(539,1167) verilog=(466,1197) diff=(-73,30) FAIL
bin 3: python=(311,276) verilog=(287,303) diff=(-24,27) FAIL
bin 4: python=(166,-553) verilog=(233,-527) diff=(67,26) FAIL
bin 5: python=(-504,1494) verilog=(-727,1400) diff=(-223,-94) FAIL
bin 6: python=(281,-1020) verilog=(463,-950) diff=(182,70) FAIL
bin 7: python=(678,262) verilog=(608,400) diff=(-70,138) FAIL
bin 8: python=(-707,-909) verilog=(-466,-1054) diff=(241,-145) FAIL
bin 9: python=(-749,-784) verilog=(-507,-958) diff=(242,-174) FAIL
bin 10: python=(-66,-527) verilog=(97,-522) diff=(163,5) FAIL
bin 11: python=(-195,179) verilog=(-243,105) diff=(-48,-74) FAIL
bin 12: python=(1235,313) verilog=(1038,737) diff=(-197,424) FAIL
bin 13: python=(382,-648) verilog=(604,-448) diff=(222,200) FAIL
bin 14: python=(1195,-945) verilog=(1481,-362) diff=(286,583) FAIL
bin 15: python=(1178,-338) verilog=(1206,220) diff=(28,558) FAIL
bin 16: python=(805,-1724) verilog=(1522,-1142) diff=(717,582) FAIL
bin 17: python=(-2368,993) verilog=(-2548,-320) diff=(-180,-1313) FAIL
bin 18: python=(1083,-451) verilog=(1158,186) diff=(75,637) FAIL
bin 19: python=(-80,672) verilog=(-435,516) diff=(-355,-156) FAIL
bin 20: python=(1144,929) verilog=(401,1420) diff=(-743,491) FAIL
bin 21: python=(2138,91) verilog=(1654,1358) diff=(-484,1267) FAIL
bin 22: python=(-1145,-1118) verilog=(-197,-1588) diff=(948,-470) FAIL
bin 23: python=(-812,-203) verilog=(-486,-682) diff=(326,-479) FAIL
bin 24: python=(1561,-1298) verilog=(2027,85) diff=(466,1383) FAIL
bin 25: python=(-37,872) verilog=(-630,602) diff=(-593,-270) FAIL
bin 26: python=(345,1258) verilog=(-660,1124) diff=(-1005,-134) FAIL
bin 27: python=(-1852,-1752) verilog=(37,-2549) diff=(1889,-797) FAIL
bin 28: python=(-780,-473) verilog=(-150,-899) diff=(630,-426) FAIL
bin 29: python=(423,-1911) verilog=(1752,-874) diff=(1329,1037) FAIL
bin 30: python=(67,-493) verilog=(434,-246) diff=(367,247) FAIL

```

Hình 5.23: Kết quả kiểm thử sau tổng hợp của khối FFT

5.3. Hiện thực hóa phần cứng (Implementation)

5.3.1. Tài nguyên sử dụng thực tế

Kết quả sử dụng tài nguyên sau implementation gần như không thay đổi nhiều so với post-synthesis, ngoại trừ số lượng LUT và LUTRAM giảm nhẹ (do công cụ tối ưu logic trong quá trình mapping vật lý) và số lượng BUFG tăng từ 1 lên 24 (do Vivado tự động chèn thêm các bộ đệm clock buffer nhằm đảm bảo chất lượng phân phối tín hiệu clock trên toàn chip sau khi định tuyến).

Resource	Utilization	Available	Utilization...
LUT	130601	230400	56.68
LUTRAM	1315	101760	1.29
FF	166134	460800	36.05
BRAM	181.50	312	58.17
DSP	161	1728	9.32
IO	84	360	23.33
BUFG	24	544	4.41

Hình 5.24: Bảng kết quả sử dụng tài nguyên (Implementation)

Bảng 5.4: So sánh kết quả sử dụng tài nguyên

Resource	Post-Synthesis	Post-Implementation
LUT	131937	130601
LUTRAM	1427	1315
FF	166134	166134
BRAM	181.50	181.50
DSP	161	161

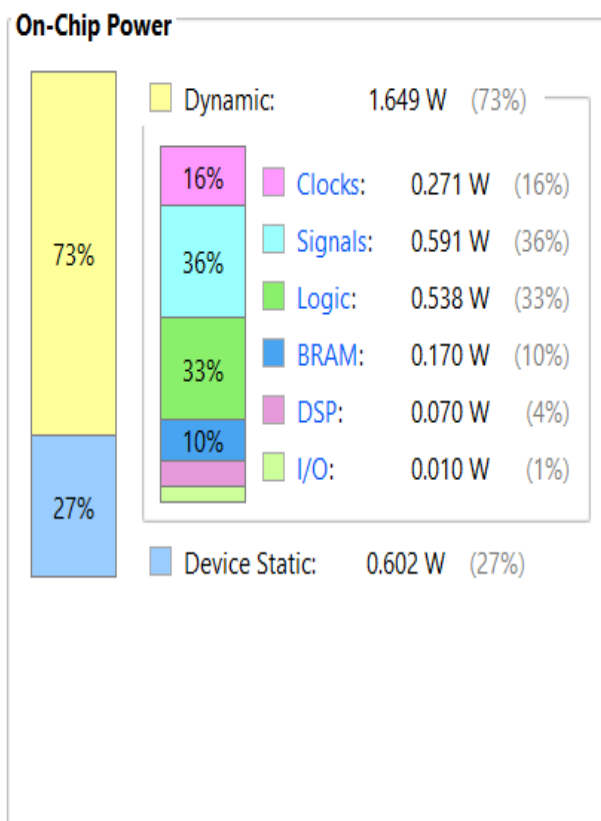
IO	84	84
BUFG	1	24

5.3.2. Phân tích công suất tiêu thụ (Power)

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 2.251 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: NA
Junction Temperature: 27.2°C
Thermal Margin: 72.8°C (72.9 W)
Ambient Temperature: 25.0 °C
Effective θ_{JA} : 1.0°C/W
Power supplied to off-chip devices: 0 W
Confidence level: Low

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity



Hình 5.25: Ảnh báo cáo công suất tiêu thụ

Kết quả phân tích công suất sau bước implementation cho thấy tổng công suất tiêu thụ của thiết kế đạt 2.251 W, trong đó công suất động chiếm 1.649 W (73%) và công suất tĩnh chiếm 0.602 W (27%). Thành phần công suất động chủ yếu tập trung ở Signal (0.591 W, 36%), Logic (0.538 W, 33%), và Clocks (0.271 W, 16%), trong khi BRAM, DSP và I/O chiếm tỷ lệ nhỏ hơn.

Kết quả này phản ánh đặc trưng tiêu thụ năng lượng của thiết kế sau khi đã được tổng hợp và định tuyến vật lý, trong đó hoạt động chuyển mạch của tín hiệu và logic là thành phần chi phối chính. Công suất tổng thể được xác định dựa trên netlist

sau implementation, do đó phản ánh tương đối chính xác hành vi tiêu thụ năng lượng trong điều kiện hoạt động thực tế.

5.3.3. Phân tích thời gian và tần số hoạt động (Timing)

Để đảm bảo thiết kế đáp ứng yêu cầu về thời gian, hệ thống được ràng buộc với chu kỳ xung clock 10 ns, tương ứng với tần số hoạt động mục tiêu 100 MHz.

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.411 ns	Worst Hold Slack (WHS): 0.010 ns	Worst Pulse Width Slack (WPWS): 4.458 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 328320	Total Number of Endpoints: 328320	Total Number of Endpoints: 168279

Hình 5.26: Ảnh báo cáo phân tích thời gian

Kết quả phân tích timing cho thấy thiết kế đã đáp ứng đầy đủ các ràng buộc thời gian. Cụ thể, Worst Negative Slack (WNS) đạt 1.411 ns, cho thấy ràng buộc setup time được thỏa mãn với biên độ slack dương. Worst Hold Slack (WHS) đạt 0.010 ns và không ghi nhận bất kỳ endpoint vi phạm nào, đảm bảo ràng buộc hold time được đáp ứng. Đồng thời, ràng buộc về độ rộng xung clock cũng được thỏa mãn với Worst Pulse Width Slack (WPWS) đạt 4.458 ns và không có vi phạm tại các endpoint liên quan.

Từ giá trị WNS = 1.411 ns và chu kỳ clock thiết kế 10 ns, có thể xác định độ trễ của đường truyền tới hạn (Critical Path Delay) như sau:

$$T_{critical} = T_{clock} - WNS = 10 - 1.411 = 8.589 \text{ ns} \quad (4.1)$$

Suy ra tần số hoạt động cực đại của thiết kế là:

$$F_{max} = \frac{1}{T_{critical}} = \frac{1}{8.589 \text{ ns}} \approx 116.4 \text{ MHz} \quad (4.2)$$

Kết quả này cho thấy mặc dù hệ thống được thiết kế để hoạt động ở 100 MHz, thiết kế vẫn còn biên dự trữ timing (timing margin) khoảng 16.4 MHz, tương đương

khoảng 16.4% so với tần số mục tiêu. Biên dự trữ này góp phần nâng cao độ tin cậy khi vận hành, đồng thời tạo điều kiện thuận lợi cho việc mở rộng hoặc tối ưu thêm thiết kế trong tương lai mà vẫn đảm bảo đáp ứng các ràng buộc thời gian.

5.4. So sánh tài nguyên hệ thống với các công trình nghiên cứu liên quan

Bảng 5.5: So sánh tài nguyên hệ thống giữa các công trình nghiên cứu

Chỉ số tài nguyên	Vandendriessche et al. (2021) [1]	da Silva et al. (2026) [2]	Khóa Luận
FPGA (Nền tảng)	ZCU104 (XCZU7EV)	ZU3CG (Ultra96-V2)	ZCU104 (XCZU7EV)
Mô hình / Phương pháp	CNN 2D (hls4ml & Vitis AI DPU)	CNN-RNN (GRU) HLS	CNN KWS 3 lớp (RTL thủ công)
LUT	21,609 – 523,000 (9.38% – 227.29%)	33,961 (48.10%)	130,601 (56.68%)
FF	32,269 – 165,381 (7.00% – 35.89%)	5,977 (4.30%)	166,134 (36.05%)
BRAM	69.50 – 311.00 (22.28% – 99.68%)	108.00 (50.00%)	181.50 (58.17%)
DSP	66 – 1,277 (3.82% – 73.90%)	64 (18.10%)	161 (9.32%)

Dựa trên bảng so sánh hiệu năng sử dụng tài nguyên phần cứng, có thể thấy rõ sự khác biệt trong chiến lược thiết kế của đồ án so với các công trình nghiên cứu quốc tế:

Thứ nhất, về tối ưu hóa tài nguyên: Công trình của Vandendriessche et al. [1] cho thấy sự bùng nổ tài nguyên không kiểm soát khi sử dụng các công cụ dịch tự động (như hls4ml chiếm tới 227.29% LUT), dẫn đến việc thiết kế không thể thực thi trên phần cứng thực tế. Ngược lại, nhờ phương pháp thiết kế thủ công cấp RTL kết

hợp với kỹ thuật đa hợp chia sẻ phần cứng (time-multiplexing) cho tầng phân lớp, khóa luận của nhóm đã kiểm soát tài nguyên logic (LUT) ở mức 57.26% và tài nguyên tính toán (DSP) chỉ 9.32%. Kết quả này minh chứng rằng việc tự thiết kế mạch điều khiển chuyên dụng cho phép tận dụng tối đa tài nguyên sẵn có mà không gây lãng phí như các mô hình DPU tổng quát hoặc các công cụ dịch tự động.

Thứ hai, về tính toàn vẹn của hệ thống: So với công trình của da Silva et al. [2] triển khai trên dòng chip ZU3CG có quy mô tài nguyên nhỏ hơn, khóa luận đạt được sự cân bằng ổn định trên nền tảng ZCU104 cao cấp. Mặc dù tiêu thụ tài nguyên Flip-Flops (FF) ở mức 36.05% để duy trì cấu trúc Pipeline xử lý đồng bộ từ khâu tiền xử lý (FFT, Log-Mel) đến mô hình CNN, hệ thống của khóa luận đã hoàn thiện được một chu trình nhận diện khép kín, hoạt động mà không phụ thuộc vào các thư viện HLS.

Tổng kết lại, kết quả thực nghiệm khẳng định rằng cách tiếp cận thiết kế RTL thủ công mang lại hiệu quả cao trong việc cân bằng giữa diện tích phần cứng và khả năng xử lý thời gian thực. Khóa luận không chỉ dừng lại ở mức mô hình thuật toán mà đã hiện thực hóa thành công một lõi IP tối ưu, phù hợp để tích hợp vào các hệ thống nhúng cảnh báo âm thanh thông minh trong thực tế.

Chương 6. TỔNG KẾT

6.1. Kết luận

Khóa luận đã nghiên cứu, thiết kế và hiện thực một pipeline xử lý âm thanh phục vụ bài toán phát hiện âm thanh khẩn cấp trong môi trường công cộng. Hệ thống được xây dựng theo hướng kết hợp giữa xử lý tín hiệu số và mạng nơ-ron tích chập CNN, trong đó tín hiệu âm thanh đầu vào được chuẩn hóa ở tần số lấy mẫu 16 kHz, sau đó được trích xuất đặc trưng Log-Mel Spectrogram và đưa vào mô hình CNN để phân loại các lớp âm thanh mục tiêu.

Về mặt lý thuyết, khóa luận đã trình bày các nội dung nền tảng liên quan đến tín hiệu âm thanh số, chia frame, biến đổi Fourier nhanh FFT, phổ công suất, Mel filterbank, Log-Mel Spectrogram, mạng CNN và biểu diễn số fixed-point trong phần cứng. Đây là cơ sở quan trọng để xây dựng pipeline xử lý âm thanh từ miền thời gian sang miền đặc trưng dùng cho mô hình học sâu.

Về mặt thiết kế phần cứng, khóa luận đã xây dựng kiến trúc RTL gồm các khối chính: FFT, Log-Mel và mô hình CNN. Khối FFT thực hiện biến đổi tín hiệu từ miền thời gian sang miền tần số. Khối Log-Mel Spectrogram thực hiện tính phổ công suất, tích lũy Mel filterbank và biến đổi logarithm để tạo ra đặc trưng Log-Mel 16 bit. Bên cạnh đó, kiến trúc CNN được thiết kế theo hướng gồm khối trích xuất đặc trưng và khối phân loại, phù hợp với dữ liệu đầu vào dạng ảnh Log-Mel kích thước 40×64 .

Về mặt kiểm thử, khóa luận đã thực hiện mô phỏng từng IP và chuỗi xử lý hoàn chỉnh FFT – Log-Mel – CNN trên Vivado. Kết quả mô phỏng và báo cáo tài nguyên cho thấy hệ thống đã hoàn chỉnh về mặt thiết kế và chức năng.

Nhìn chung, khóa luận đã hoàn thành được nền tảng thiết kế và kiểm thử. Kết quả đạt được cho thấy hướng tiếp cận sử dụng FFT – Log-Mel – CNN là phù hợp với bài toán phát hiện âm thanh khẩn cấp, đồng thời tạo cơ sở để tiếp tục tích hợp toàn bộ hệ thống và triển khai trên FPGA trong các bước phát triển tiếp theo.

6.2. Khó khăn

Trong quá trình thực hiện khóa luận, nhóm gặp một số khó khăn liên quan đến cả phần thuật toán, phần cứng và kiểm chứng hệ thống.

Về mặt kỹ thuật, khó khăn nhóm đang gặp là việc hiện thực hóa chuỗi thuật lượng tử hóa từ số thực sang số cố định (fixed-point). Quá trình này đòi hỏi sự cân bằng khắt khe giữa độ chính xác và tài nguyên FPGA, dẫn đến việc kết quả RTL vẫn tồn tại sai số nhỏ so với mô hình tham chiếu.

Về hạn chế, hệ thống hiện mới được xác minh chủ yếu qua môi trường mô phỏng và chưa triển khai thực tế trên bo mạch FPGA để đánh giá hiệu năng thời gian thực. Đồng thời, tập dữ liệu huấn luyện còn giới hạn ở một số từ khóa khẩu cấp phổ biến.

6.3. Hướng phát triển

Trong tương lai nhóm sẽ tiếp tục hiệu chỉnh pipeline fixed-point để giảm sai số giữa RTL và mô hình Python. Tối ưu kiến trúc CNN để cân bằng giữa độ chính xác và tài nguyên phần cứng.

Mở rộng thêm tập dữ liệu âm thanh. Có thể xem xét bổ sung thêm dữ liệu ở nhiều điều kiện khác nhau, đồng thời áp dụng kỹ thuật tăng cường dữ liệu như thêm nhiễu, thay đổi âm lượng,..... để cải thiện khả năng tổng quát hóa của mô hình.

Hệ thống có thể được mở rộng từ bài toán nhận dạng một số từ khóa khẩu cấp sang bài toán phát hiện đa dạng các loại âm thanh bất thường khác như tiếng kính vỡ, tiếng va chạm mạnh, tiếng la hét, tiếng còi báo động hoặc tiếng nổ. Điều này giúp nâng cao khả năng ứng dụng của hệ thống trong các mô hình camera công cộng thông minh và hệ thống giám sát an ninh.

TÀI LIỆU THAM KHẢO

- [1] J. Vandendriessche, N. Wouters, B. da Silva, M. Lamrini, M. Y. Chkouri, and A. Touhafi, “Environmental Sound Recognition on Embedded Systems: From FPGAs to TPUs,” *Electronics*, vol. 10, no. 21, p. 2622, Oct. 2021.
- [2] R. L. da Silva, G. Jacinto, M. Véstias, and R. P. Duarte, “Real-Time Bird Audio Detection with a CNN-RNN Model on a SoC-FPGA,” *Electronics*, vol. 15, no. 4, p. 354, Feb. 2026.
- [3] P. Fonseca et al., “Urban Sound Classification using Convolutional Neural Networks and Log-Mel Spectrograms,” *IEEE Access*, 2020.
- [4] K. J. Piczak, “Environmental Sound Classification with Convolutional Neural Networks,” *IEEE ICASSP Workshop*, 2019.
- [5] J. Salamon and J. P. Bello, “Deep Convolutional Neural Networks and Data Augmentation for Environmental Sound Classification,” *IEEE Signal Processing Letters*, 2019.
- [6] H. Phan et al., “Audio Event Detection with Weak Labeling using CNNs,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2021.
- [7] S. Kong and D. A. Bansal, “Acoustic Event Detection using Deep Learning and Log-Mel Features,” *Applied Acoustics*, 2022.
- [8] Y. Chen et al., “Emergency Sound Recognition using Deep Convolutional Neural Networks,” *IEEE Access*, 2023.
- [9] E. Lee et al., “Lightweight CNN for Real-Time Audio Event Detection on Edge Devices,” *Sensors*, 2024.
- [10] N. Meseguer-Brocal et al., “DCASE Challenge Task: Audio Event Detection and Classification Narration,” *DCASE Workshop Proceedings*, 2022–2024.
- [11] R. Goyal et al., “Fixed-Point Quantization of Neural Networks for Embedded Inference,” *arXiv preprint*, 2021.

- [12] T. Kim, J. Lee, and H. Park, "Scream and Shouting Detection for Emergency Situation Awareness Using Convolutional Neural Networks," *IEEE Transactions on Consumer Electronics*, vol. 68, no. 2, pp. 145–153, May 2022.
- [13] M. S. Hossain, M. R. Islam, and A. Ahmed, "Hardware Acceleration of Mel-Frequency Cepstral Coefficients (MFCC) for Speech and Audio Processing on FPGA," *Microprocessors and Microsystems*, vol. 92, p. 104567, Jul. 2023.
- [14] A. Kumar and S. R. Chowdhury, "Design of a High-Throughput Pipeline FFT Processor for Real-Time Audio Feature Extraction," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 4, pp. 589–598, Apr. 2023.
- [15] L. Bai, Y. Han, and X. Zhang, "An Efficient Hardware Accelerator for Convolutional Neural Network-Based Audio Classification on FPGA," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 70, no. 8, pp. 3014–3018, Aug. 2023.
- [16] G. Sharon, A. Gupta, and V. Sharma, "Deep Learning-Based Automated Violence and Distress Audio Detection in Urban Environments," *Applied Sciences*, vol. 14, no. 3, p. 1102, Jan. 2024.
- [17] X. Wang, Y. Liu, and Z. Wang, "Edge-Computing-Based Lightweight CNN Accelerator on FPGA for Real-Time Acoustic Event Recognition," *IEEE Internet of Things Journal*, vol. 11, no. 5, pp. 8234–8245, Mar. 2024.
- [18] J. Zhang, H. Wu, and L. Tan, "Co-Design of Hardware and Software for Real-Time Sound Classification Using Log-Mel Spectrogram and CNN on SoC-FPGA," *Journal of Systems Architecture*, vol. 148, p. 103062, Mar. 2024.
- [19] H. Nguyen, T. Tran, and V. Le, "FPGA-Based Hardware Intellectual Property (IP) Core Design for Real-Time Emergency Audio Recognition," *International Conference on Advanced Technologies for Communications (ATC)*, pp. 112–117, Oct. 2025.